Contents

第一册《Transformer 推导 30 讲》 **5**
本册目录 . . . 5
1. 数学工具/预备知识 . . . 6
2. 完整推导 . . . 6
3. 直觉理解 . . . 9
4. 常见误区 . . . 9
5. 代码实现 . . . 9
6. 小结 . . . 10
7. 延伸阅读/进一步思考 . . . 10
8. 思考题 . . . 10
1. 数学工具/预备知识 . . . 11
2. 完整推导 . . . 11
3. 直觉理解 . . . 13
4. 常见误区 . . . 13
5. 代码实现 . . . 13
6. 小结 . . . 14
7. 延伸阅读/进一步思考 . . . 14
8. 思考题 . . . 15
1. 数学工具/预备知识 . . . 15
2. 完整推导 . . . 15
3. 直觉理解 . . . 17
4. 常见误区 . . . 17
5. 代码实现 . . . 18
6. 小结 . . . 19
7. 延伸阅读/进一步思考 . . . 19
8. 思考题 . . . 19
1. 数学工具/预备知识 . . . 20
2. 完整推导 . . . 20
3. 直觉理解 . . . 22
4. 常见误区 . . . 22
5. 代码实现 . . . 22
6. 小结 . . . 23
7. 延伸阅读/进一步思考 . . . 23
8. 思考题 . . . 23
1. 数学工具/预备知识 . . . 24
2. 完整推导 . . . 24
3. 直觉理解 . . . 26
4. 常见误区 . . . 26
5. 代码实现 . . . 26
6. 小结 . . . 27
7. 延伸阅读/进一步思考 . . . 27
8. 思考题 . . . 27
1. 数学工具/预备知识 . . . 28
2. 完整推导 . . . 28
3. 直觉理解 . . . 29
4. 常见误区 . . . 29
5. 代码实现 . . . 29
6. 小结 . . . 30
7. 延伸阅读/进一步思考 . . . 30
8. 思考题 . . . 30
1. 数学工具/预备知识 . . . 31
2. 完整推导 . . . 31
3. 直觉理解 . . . 32

4. 常见误区 32
5. 代码实现 33
6. 小结 33
7. 延伸阅读/进一步思考 34
8. 思考题 34
1. 数学工具/预备知识 34
2. 完整推导 34
3. 直觉理解 36
4. 常见误区 36
5. 代码实现 36
6. 小结 37
7. 延伸阅读/进一步思考 37
8. 思考题 37
1. 数学工具/预备知识 38
2. 完整推导 38
3. 直觉理解 39
4. 常见误区 39
5. 代码实现 39
6. 小结 40
7. 延伸阅读/进一步思考 40
8. 思考题 40
1. 数学工具/预备知识 41
2. 完整推导 41
3. 直觉理解 42
4. 常见误区 42
5. 代码实现 42
6. 小结 43
7. 延伸阅读/进一步思考 43
8. 思考题 43
1. 数学工具/预备知识 44
2. 完整推导 44
3. 直觉理解 45
4. 常见误区 45
5. 代码实现 45
6. 小结 46
7. 延伸阅读/进一步思考 46
8. 思考题 46
1. 数学工具/预备知识 47
2. 完整推导 47
3. 直觉理解 48
4. 常见误区 48
5. 代码实现 48
6. 小结 49
7. 延伸阅读/进一步思考 49
8. 思考题 49
1. 数学工具/预备知识 50
2. 完整推导 50
3. 直觉理解 51
4. 常见误区 51
5. 代码实现 51
6. 小结 52
7. 延伸阅读/进一步思考 52
8. 思考题 52
1. 数学工具/预备知识 52
2. 完整推导 52

3. 直觉理解 . . . 53
4. 常见误区 . . . 53
5. 代码实现 . . . 53
6. 小结 . . . 54
7. 延伸阅读/进一步思考 . . . 54
8. 思考题 . . . 54
1. 数学工具/预备知识 . . . 55
2. 完整推导 . . . 55
3. 直觉理解 . . . 56
4. 常见误区 . . . 56
5. 代码实现 . . . 56
6. 小结 . . . 57
7. 延伸阅读/进一步思考 . . . 57
8. 思考题 . . . 57
1. 数学工具/预备知识 . . . 57
2. 完整推导 . . . 58
3. 直觉理解 . . . 59
4. 常见误区 . . . 60
5. 代码实现 . . . 60
6. 小结 . . . 61
7. 思考题 . . . 61
8. 延伸阅读 / 进一步思考 . . . 61
1. 数学工具/预备知识 . . . 61
2. 完整推导 . . . 62
3. 直觉理解 . . . 64
4. 常见误区 . . . 64
5. 代码实现 . . . 64
6. 小结 . . . 65
7. 思考题 . . . 65
8. 延伸阅读 / 进一步思考 . . . 65
1. 数学工具/预备知识 . . . 65
2. 完整推导 . . . 66
3. 直觉理解 . . . 67
4. 常见误区 . . . 67
5. 代码实现 . . . 68
6. 小结 . . . 68
7. 思考题 . . . 68
8. 延伸阅读 / 进一步思考 . . . 68
1. 数学工具/预备知识 . . . 69
2. 完整推导 . . . 69
3. 直觉理解 . . . 71
4. 常见误区 . . . 71
5. 代码实现 . . . 71
6. 小结 . . . 72
7. 思考题 . . . 72
8. 延伸阅读 / 进一步思考 . . . 72
1. 数学工具/预备知识 . . . 73
2. 完整推导 . . . 73
3. 直觉理解 . . . 74
4. 常见误区 . . . 74
5. 代码实现 . . . 75
6. 小结 . . . 75
7. 思考题 . . . 75
8. 延伸阅读 / 进一步思考 . . . 76
1. 数学工具/预备知识 . . . 76

2. 完整推导 . . . 76
3. 直觉理解 . . . 77
4. 常见误区 . . . 78
5. 代码实现 . . . 78
6. 小结 . . . 78
7. 思考题 . . . 78
8. 延伸阅读 / 进一步思考 . . . 79
1. 数学工具/预备知识 . . . 79
2. 完整推导 . . . 79
3. 直觉理解 . . . 80
4. 常见误区 . . . 81
5. 代码实现 . . . 81
6. 小结 . . . 82
7. 思考题 . . . 82
8. 延伸阅读 / 进一步思考 . . . 82
1. 数学工具/预备知识 . . . 82
2. 完整推导 . . . 82
3. 直觉理解 . . . 84
4. 常见误区 . . . 84
5. 代码实现 . . . 84
6. 小结 . . . 85
7. 思考题 . . . 85
8. 延伸阅读 / 进一步思考 . . . 85
1. 数学工具/预备知识 . . . 85
2. 完整推导 . . . 85
3. 直觉理解 . . . 87
4. 常见误区 . . . 87
5. 代码实现 . . . 87
6. 小结 . . . 88
7. 思考题 . . . 88
8. 延伸阅读 / 进一步思考 . . . 88
1. 数学工具/预备知识 . . . 88
2. 完整推导 . . . 89
3. 直觉理解 . . . 90
4. 常见误区 . . . 90
5. 代码实现 . . . 90
6. 小结 . . . 91
7. 思考题 . . . 91
8. 延伸阅读 / 进一步思考 . . . 91
1. 数学工具/预备知识 . . . 92
2. 完整推导 . . . 92
3. 直觉理解 . . . 93
4. 常见误区 . . . 93
5. 代码实现 . . . 93
6. 小结 . . . 94
7. 思考题 . . . 94
8. 延伸阅读 / 进一步思考 . . . 94
1. 数学工具/预备知识 . . . 94
2. 完整推导 . . . 95
3. 直觉理解 . . . 95
4. 常见误区 . . . 95
5. 代码实现 . . . 95
6. 小结 . . . 97
7. 思考题 . . . 97
8. 延伸阅读 / 进一步思考 . . . 97

1. 数学工具/预备知识 . . . 97
2. 完整推导 . . . 97
3. 直觉理解 . . . 98
4. 常见误区 . . . 98
5. 代码实现 . . . 98
6. 小结 . . . 100
7. 思考题 . . . 100
8. 延伸阅读 / 进一步思考 . . . 100
1. 数学工具/预备知识 . . . 100
2. 完整推导 . . . 100
3. 直觉理解 . . . 101
4. 常见误区 . . . 101
5. 代码实现 . . . 101
6. 小结 . . . 102
7. 思考题 . . . 103
8. 延伸阅读 / 进一步思考 . . . 103
1. 数学工具/预备知识 . . . 103
2. 完整推导 . . . 103
3. 直觉理解 . . . 104
4. 常见误区 . . . 104
5. 代码实现 . . . 104
6. 小结 . . . 105
7. 思考题 . . . 105
8. 延伸阅读 / 进一步思考 . . . 105

第一册《Transformer 推导 30 讲》

从张量与矩阵乘法出发，完整推导 Scaled Dot-Product Attention、Multi-Head、位置编码、RoPE、KV Cache、FlashAttention，最终手写 mini-Transformer。

共 30 讲·合订版

本册目录

1. 第 1 讲 张量与矩阵乘法：Transformer 的运算语言
2. 第 2 讲 概率与信息论：从交叉熵到 KL 散度
3. 第 3 讲 梯度下降与反向传播：链式法则的矩阵形式
4. 第 4 讲 Softmax 与归一化：从 logits 到概率分布
5. 第 5 讲 序列建模简史：RNN 与 LSTM 的能力边界
6. 第 6 讲 Seq2Seq 与注意力机制的萌芽
7. 第 7 讲 Query-Key-Value：注意力的一般化抽象
8. 第 8 讲 Scaled Dot-Product Attention：完整推导
9. 第 9 讲 Multi-Head Attention：多头的本质与维度分析
10. 第 10 讲 Transformer 总体架构：Encoder-Decoder 数据流全览
11. 第 11 讲 位置编码：正余弦编码的完整推导
12. 第 12 讲 Add & Norm：残差连接与 LayerNorm 的原理
13. 第 13 讲 前馈网络 FFN：逐位置 MLP 的作用
14. 第 14 讲 Mask 机制：Padding Mask 与 Causal Mask
15. 第 15 讲 参数量与计算量：一层 Transformer 的 FLOPs 分析
16. 第 16 讲 复杂度对比：Transformer vs RNN vs CNN
17. 第 17 讲 损失函数：交叉熵、困惑度与 Label Smoothing
18. 第 18 讲 学习率调度：Warmup 与衰减的设计原理
19. 第 19 讲 Adam 优化器：从动量到自适应学习率
20. 第 20 讲 初始化与梯度稳定性：Xavier 与 He 初始化推导

21. 第 21 讲 混合精度训练：FP16 与 BF16 的数学与工程
22. 第 22 讲 RoPE 旋转位置编码：复数域的优雅推导
23. 第 23 讲 KV Cache：自回归推理的加速本质
24. 第 24 讲 FlashAttention：IO 感知的精确注意力
25. 第 25 讲 MoE 混合专家：稀疏激活的条件计算
26. 第 26 讲 量化与蒸馏：模型压缩的核心机理
27. 第 27 讲 手写 Attention：50 行 PyTorch 实现
28. 第 28 讲 手写 mini-Transformer：完整模型搭建
29. 第 29 讲 从零训练：小语料上的语言模型与注意力可视化
30. 第 30 讲 面试与进阶：高频考点与论文阅读路线

第 1 讲 张量与矩阵乘法：Transformer 的运算语言

> **本讲问题**：为什么 Transformer 里几乎所有运算都能写成”两个三维张量相乘再 softmax”？理解张量和矩阵乘法的多重视角，是后面 29 讲所有推导的地基。这一讲不追求快，追求把”为什么可以这么算”讲到底。

1. 数学工具/预备知识

这一讲会用到下面几样工具。第一次见的术语先在这里交代清楚，后面不再解释。

标量、向量、矩阵、张量。一个数叫标量 $s \in \mathbb{R}$；一排数叫向量 $x \in \mathbb{R}^d$；一张二维数表叫矩阵 $X \in \mathbb{R}^{m\times n}$；把数表推广到三维及以上，就叫张量。三维张量 $\mathcal{X} \in \mathbb{R}^{b\times s\times d}$ 可以理解成”一摞 b 张二维表，每张表有 s 行 d 列”。

记法约定。标量用小写斜体 x，向量用小写粗体 x，矩阵用大写粗体 X，三维张量沿用 $\mathcal{X}$ 或直接写形状。向量默认写成列向量，转置 $x^\top$ 是行向量。

矩阵乘法的定义。 $A \in \mathbb{R}^{m\times k}$ 乘 $B \in \mathbb{R}^{k\times n}$，结果 $C = AB \in \mathbb{R}^{m\times n}$，第 i 行第 j 列元素为:

$$C_{ij} = \sum_{p=1}^{k} A_{ip} B_{pj}$$

能不能乘，先看”中间维度是否对齐”：A 的列数 k 必须等于 B 的行数 k。结果形状是”左行右列” $m \times n$。

迹（trace）。方阵 $M \in \mathbb{R}^{n\times n}$ 的迹是对角线元素之和 $\mathrm{tr}(M) = \sum_i M_{ii}$。它有两个马上要用的性质:

- 线性：$\mathrm{tr}(aM + bN) = a\,\mathrm{tr}(M) + b\,\mathrm{tr}(N)$。
- 循环交换性：只要 AB 和 BA 都有定义，$\mathrm{tr}(AB) = \mathrm{tr}(BA)$。把它套到三个矩阵上，$\mathrm{tr}(ABC) = \mathrm{tr}(BCA) = \mathrm{tr}(CAB)$——可以把”乘积里最右边的矩阵”搬到最左边。

内积与逐元素积。两个维度相同的向量 $u, v \in \mathbb{R}^d$ 的内积是 $u^\top v = \sum_i u_i v_i$，结果是一个标量。两个形状相同的矩阵逐元素相乘叫哈达玛积，记作 $\odot$，$(A \odot B)_{ij} = A_{ij} B_{ij}$。

偏导数与雅可比（先建立直觉）。把一个向量 $x \in \mathbb{R}^n$ 映射到另一个向量 $y \in \mathbb{R}^m$ 的函数 f，它的”导数”不是一个数，而是一个 $m \times n$ 的矩阵，叫雅可比矩阵，第 (i, j) 个元素是 $\partial y_i / \partial x_j$。本讲 2.3 先用更省事的”微分法”，避开显式写雅可比。

2. 完整推导

2.1 从标量到张量：轴的语义

标量是一个数：$s \in \mathbb{R}$。**向量**是一列数：$x \in \mathbb{R}^d$，写成 $x = (x_1, x_2, \dots, x_d)^\top$。**矩阵**是一个二维数表：$X \in \mathbb{R}^{m\times n}$，第 i 行第 j 列元素记作 X_{ij}。**张量**是多维数组的泛化。三维张量 $\mathcal{X} \in \mathbb{R}^{b\times s\times d}$ 在 Transformer 里的标准含义是：一批 b 个样本（batch），每个样本是一条长度为 s 的序列，序列里每个 token 用一个 d 维向量表示。输入形状永远写成 `(batch_size, seq_len, d_model)`。

张量的**轴**（axis）是后面所有操作的命门。对 $\mathcal{X} \in \mathbb{R}^{b\times s\times d}$:

- 第 0 轴是 batch——“这是这批里的第几个样本”；
- 第 1 轴是序列长度——“这是序列里的第几个 token”；
- 第 2 轴是特征维度——“这是 token 向量里的第几维”。

后面所有 `transpose`、`reshape`、`sum(dim=...)`、`softmax(dim=-1)` 都要先在脑子里定位到具体哪根轴。为什么要强调这点？因为”把序列轴和特征轴搞反”是写 Transformer 代码时最常见的维度 bug——程序不报错，只是结果 silently 错了。

广播（broadcasting）。把一个 $\mathbb{R}^d$ 的向量”加到”一个 $\mathbb{R}^{b\times s\times d}$ 的张量上，PyTorch 会自动把向量沿着 batch 维和序列维复制 $b\times s$ 份。这不是数学上的矩阵乘法，而是工程上的对齐约定：形状从右往左对齐，维度为 1 的那一维自动扩展。理解这一点，后面理解 LayerNorm 的 γ,β 加到每个 token 上时就不会困惑。

2.2 矩阵乘法的四个视角

设 $A\in\mathbb{R}^{m\times k}$，$B\in\mathbb{R}^{k\times n}$，乘积 $C=AB\in\mathbb{R}^{m\times n}$。同一个乘积可以从四个角度看，每个角度后面都会对应到注意力的某一步。

视角一：元素视角（点积视角）

$$C_{ij}=\sum_{p=1}^{k}A_{ip}B_{pj}$$

C 的第 (i,j) 个元素等于 A 的第 i 行与 B 的第 j 列的**点积**。几何上，点积衡量两个向量的相似度：方向越一致点积越大（同方向为 $|a||b|$，垂直为 0，反方向为负）。

这个视角是 Scaled Dot-Product Attention 中”打分矩阵”的来源：$QK^\top$ 的每个元素就是一个 query 向量和一个 key 向量的点积，数值越大表示”这个 query 和这个 key 越匹配”。ch08 会在这个视角上做缩放。

视角二：列视角（列线性组合）

把 B 按列分块：$B=[b_1,b_2,\dots,b_n]$，其中 $b_j\in\mathbb{R}^k$。则：

$$C=[Ab_1,Ab_2,\dots,Ab_n]$$

C 的第 j 列 $c_j=Ab_j$。如果把 A 的列看作一组”基向量”，那么 c_j 就是这组基以 b_j 的分量为系数的**线性组合**。也就是说：A 是一个固定的”混合规则”，b_j 是第 j 个混合权重。

这个视角对应 Transformer 中”输出向量是 value 向量的加权和”：注意力 softmax 出来的权重 α_{ij} 就是混合系数，V 的列就是被混合的基。

视角三：行视角（行线性组合）

把 A 按行分块：$A=[a_1^\top;a_2^\top;\dots;a_m^\top]$，其中 $a_i^\top\in\mathbb{R}^{1\times k}$。则：

$$C=\begin{bmatrix}a_1^\top B\\ a_2^\top B\\ \vdots\\ a_m^\top B\end{bmatrix}$$

C 的第 i 行是 B 的行向量以 a_i 的分量为权重的线性组合。和列视角对偶：列视角看”输出列怎么来”，行视角看”输出行怎么来”。

视角四：外积叠加视角

把 A 按列分块 $A=[a_1,\dots,a_k]$（每列 $a_p\in\mathbb{R}^m$），把 B 按行分块 $B=[b_1^\top;\dots;b_k^\top]$（每行 $b_p^\top\in\mathbb{R}^{1\times n}$）。则：

$$AB=\sum_{p=1}^{k}a_pb_p^\top$$

每一项 $a_p b_p^\top$ 是一个 $m \times n$ 的外积矩阵：列 a_p 乘行 $b_p^\top$，它的秩严格为 1（所有列都是 a_p 的倍数）。把 k 张秩 1 的”小海报”全部叠起来，就是 C。

这个视角的推论很重要：AB **的秩不超过** k，因为它是 k 个秩 1 矩阵的和。它对应 MHA 中”每个 head 独立做注意力再叠加”的分解思路，也对应 einsum 里沿某条轴求和缩并。

2.3 矩阵求导：微分法

机器学习经常要算”标量损失 L 对整个矩阵 W 的梯度”。我们用**微分法**（dimension-matching），而不是死记公式表。

核心原则一句话：先写出微分 dL，再把它凑成 $\mathrm{tr}(G^\top dW)$ 的形式，那么凑出来的 G 就是梯度 $\partial L/\partial W$。为什么这样可以？因为对标量 L 和矩阵变量 W，微分有唯一分解：

$$dL = \sum_{ij} \frac{\partial L}{\partial W_{ij}} dW_{ij} = \mathrm{tr}\Big((\frac{\partial L}{\partial W})^\top dW \Big)$$

右边是迹形式，左边是定义。所以只要把 dL 整理成迹的形式，括号里那个矩阵就是答案。

例子： $L = f(Y)$，$Y = XW$。设 $X \in \mathbb{R}^{n\times d}$，$W \in \mathbb{R}^{d\times h}$，$Y \in \mathbb{R}^{n\times h}$。

第一步，对 $Y = XW$ 两边取微分。微分是线性算子，和求导同法则：$d(XW) = X\,dW + dX\,W$。求 $\partial L/\partial W$ 时把 X 当常数（$dX = 0$），所以 $dY = X\,dW$。

第二步，记上游梯度 $G_Y = \partial L/\partial Y$。由上面的唯一分解，$dL = \mathrm{tr}(G_Y^\top dY)$。

第三步，代入 $dY = X\,dW$：

$$dL = \mathrm{tr}(G_Y^\top X\,dW)$$

用迹的循环性质把 dW 搬到最右边（也就是把 $G_Y^\top X$ 看作一个整体），再用 $\mathrm{tr}(M) = \mathrm{tr}(M^\top)$ 调整形状：

$$\mathrm{tr}(G_Y^\top X\,dW) = \mathrm{tr}(dW\,G_Y^\top X) = \mathrm{tr}\Big((X^\top G_Y)^\top dW \Big)$$

第四步，对比 $\mathrm{tr}(G_W^\top dW)$，读出：

$$\frac{\partial L}{\partial W} = X^\top G_Y$$

同理，固定 W 对 X 求导，$dY = dX\,W$：

$$dL = \mathrm{tr}(G_Y^\top dX\,W) = \mathrm{tr}(W G_Y^\top dX) = \mathrm{tr}\Big((G_Y W^\top)^\top dX \Big)$$

所以 $\partial L/\partial X = G_Y W^\top$。

这两个公式就是反向传播里最常用的”前向乘谁、反向就转置乘谁”：前向 $Y = XW$，反向传回 X 的梯度要右乘 $W^\top$，传回 W 的梯度要左乘 $X^\top$。ch03 会反复用到。

2.4 einsum：爱因斯坦求和约定

`einsum` 把矩阵乘法的四个视角压缩成一行字符串。规则：写出每个张量的下标，箭头左边是输入下标，右边是输出下标；**只出现在左边、不出现在右边的下标被求和缩并掉**。

- $C = AB$：$C_{ij} = \sum_k A_{ik} B_{kj}$，写成 `'ik,kj->ij'`。
- $C = AB^\top$：$C_{ij} = \sum_k A_{ik} B_{jk}$，写成 `'ik,jk->ij'`。这正是注意力打分 $QK^\top$。
- 批量矩阵乘法 $\mathcal{C} = \mathcal{A}\mathcal{B}$（batch 维 b 不参与求和）：`'bik,bkj->bij'`。

- 加权求和 $\mathcal{O} = \alpha\mathcal{V}$：$O_{ij} = \sum_k \alpha_{ik} V_{kj}$，写成 `'bij,bkj->bij'`（和打分同形，只是语义不同）。

为什么要用 einsum？因为 Transformer 里张量操作经常要同时做多件事（投影、切头、打分、加权），反复 `transpose + reshape` 容易把维度搞错。一行 einsum 把”沿哪根轴缩并、保留哪根轴”写得明明白白，出错率低很多。

3. 直觉理解

直觉：矩阵乘法不是”一个公式”，而是四种同时成立的看法。

- 点积视角告诉你”怎么算一个数”——就像拿两个向量比对一下，方向越像数越大。
- 列视角告诉你”输出的每一列是输入列的混合”——就像调鸡尾酒，B 的每列是一个配方，A 是调酒台。
- 行视角和列视角是一回事，只是站在行的角度看。
- 外积视角告诉你”每一对（行向量, 列向量）都贡献一张秩 1 海报，全部叠起来就是结果”——海报张数等于中间维度 k。

在注意力里，$QK^\top$ 是点积视角（算相似度分数），softmax 后的权重乘 V 是列视角（用 value 向量拼出输出）。一个注意力公式，前半段用视角一，后半段用视角二。

4. 常见误区

误区一：把”张量”神秘化。张量就是多维数组，没有玄学。唯一要警惕的是**轴的语义**——`(batch, seq, d)` 里的 seq 轴和 d 轴不能随便 transpose。transpose 完不检查形状就写下一行代码，是维度 bug 的第一来源。

误区二：以为矩阵乘法”必须左乘行右乘列”。死记这个教条会让你在理解多头注意力时卡住——外积视角告诉我们 AB 完全可以理解成 k 张秩 1 海报的叠加，和”行列”无关。

误区三：把迹的循环性质用错方向。$\mathrm{tr}(AB) = \mathrm{tr}(BA)$ 要求 AB 和 BA 都有定义。$A \in \mathbb{R}^{n\times d}, B \in \mathbb{R}^{d\times h}$ 时，$\mathrm{tr}(G_Y^\top X\, dW)$ 能循环，是因为三个矩阵连乘的迹可以把最右的 dW 搬到最左，搬完形状仍然合法。

5. 代码实现

下面这段代码把矩阵乘法的四个视角在 PyTorch 里逐一对照验证。注意几个工程细节：用 `atol` 做容差比较而不是 `==`（浮点运算有舍入误差）；`transpose(1,2)` 交换的是第 1、2 轴，不要写成 `.T`（`.T` 会转置整个三维张量，把 batch 维也翻转）。

```
import torch

# 演示：一个最小化的" 注意力打分" 全过程，用矩阵乘法四个视角对照
torch.manual_seed(0)
b, s, d = 2, 3, 4  # batch=2, seq_len=3, dim=4

# 随机生成 Q, K, V。形状 (b, s, d)：batch 轴在外，特征轴在最后。
Q = torch.randn(b, s, d)
K = torch.randn(b, s, d)
V = torch.randn(b, s, d)

# 视角一：点积视角 —— scores[b, i, j] = Q[b,i,:] · K[b,j,:]
# K.transpose(1, 2) 把 (b, s, d) 变成 (b, d, s)，这样矩阵乘出来是 (b, s, s)。
scores_matmul = torch.matmul(Q, K.transpose(1, 2))  # shape: (b, s, s)
# einsum 版本：bid 下标和 bjd 下标缩并掉 d，保留 b,i,j。
scores_einsum = torch.einsum('bid,bjd->bij', Q, K)
print(" 点积视角 scores 形状:", tuple(scores_matmul.shape))
assert torch.allclose(scores_matmul, scores_einsum, atol=1e-5)

# 视角二：列视角 —— output[b,i,:] = sum_j alpha[b,i,j] * V[b,j,:]
# softmax 在最后一维（j 维，即 key 维）上归一化，保证每个 query 对所有 key 权重和为 1。
```

```
alpha = torch.softmax(scores_matmul, dim=-1)
# alpha: (b,s,s) 乘 V: (b,s,d)，对每个 b,i 得到 d 维加权和。
out_colview = torch.matmul(alpha, V)
out_einsum = torch.einsum('bij,bjd->bid', alpha, V)
print(" 列视角 out 形状:", tuple(out_colview.shape))
assert torch.allclose(out_colview, out_einsum, atol=1e-5)

# 视角四：外积叠加 ——对单个 batch 单个 query，手动把每个 value 向量按权重加起来。
# 这等价于 out[b,i,:] = sum_j alpha[b,i,j] * V[b,j,:]（外积视角退化为向量加权和）。
b0, i0 = 0, 0
manual = torch.zeros(d)
for j in range(s):
    manual += alpha[b0, i0, j] * V[b0, j, :]
print(" 外积叠加手动验证 vs 矩阵乘法:",
      torch.allclose(manual, out_colview[b0, i0], atol=1e-5))

# 顺带验证：注意力权重每行和为 1（softmax 性质）
print(" 每行权重和应 ≈1:", alpha.sum(dim=-1).flatten().tolist())
```

6. 小结

1. 张量是多维数组，Transformer 的核心数据形状是 `(batch, seq_len, d_model)`，每根轴都有明确语义；跨轴 `transpose` 前先想清楚在动哪根轴。
2. 矩阵乘法有四个等价视角：元素点积（算相似度）、列线性组合（加权混合 value）、行线性组合、外积叠加（秩不超过中间维度）。注意力同时用到了视角一和视角二。
3. 矩阵求通用”微分法”：凑成 $\mathrm{tr}(G^\top dW)$ 的形式即可读出梯度，配合迹的循环性质不用死记公式。
4. einsum 用一个字符串统一表达所有张量缩并，避免反复 transpose 出错。

7. 延伸阅读/进一步思考

- 矩阵求导的完整规则可参考 Magnus & Neudecker, *Matrix Differential Calculus with Applications in Statistics and Econometrics* 的微分法体系；反向传播里所有”转置乘回来”的公式都能从这套方法推出，不必背表。
- 想深入”外积叠加 = 低秩分解”的工程意义，可参考 SVD / 低秩近似：把 AB 的 k 项外积只保留前 r 项（$r < k$），就是低秩矩阵分解，模型压缩和 LoRA 微调里常用。
- einsum 在 GPU 上的高效实现见 `opt-einsum`（Numpy 生态）和 PyTorch 2.0 后的 `torch.compile` 算子融合。

8. 思考题

基础题

1. 若 $Q \in \mathbb{R}^{b\times s\times d_k}$，$K \in \mathbb{R}^{b\times s\times d_k}$，写出 $QK^\top$ 的 einsum 表达式，并说明输出形状和每个元素的含义。

进阶题

2. 用微分法推导：若 $Y = XW + b$，其中 $b \in \mathbb{R}^h$ 是偏置向量（广播加到 Y 的每一行），求 $\partial L/\partial b$。提示：$dY = dX\,W + db$，注意 db 是逐列加上去的，要把它按样本维求和。
3. 外积叠加视角下 $AB = \sum_p a_p b_p^\top$。若 A 的秩为 r_A、B 的秩为 r_B，AB 的秩最大是多少？为什么”两个高瘦矩阵相乘不会让秩暴涨”？

开放题

4. 如果把注意力写成 einsum `'bid,bjd->bij'`，然后又写 `'bij,bjd->bid'`，整个过程里哪几维被缩并了？对比”先投影成 Q/K/V 再算注意力”和”直接在原始 embedding 上算注意力”，从外积视角看参数量差在哪？

第 2 讲 概率与信息论：从交叉熵到 KL 散度

> **本讲问题**：为什么语言模型的损失函数是交叉熵，而不是 MSE？为什么”最大化似然”和”最小化交叉熵”是同一件事？本讲把这两个问题从公理推到底，并且顺手证明 KL 散度为什么非负。

1. 数学工具/预备知识

这一讲用到的工具先交代清楚。

离散概率分布。随机变量 X 取有限个值 $x_1, \dots, x_k$，概率分布 $p(x)$ 满足两条公理：非负 $p(x) \geq 0$，归一 $\sum_x p(x) = 1$。所有可能分布构成的集合叫概率单纯形。

期望。函数 $f(X)$ 对分布 p 的期望是”按概率加权平均”：

$$\mathbb{E}_{x\sim p}[f(x)] = \sum_x p(x) f(x)$$

期望是线性的：$\mathbb{E}[af + bg] = a\mathbb{E}[f] + b\mathbb{E}[g]$。

自然对数 ln。它是 e^x 的反函数。本讲用到三条性质：$\ln 1 = 0$；$\ln(ab) = \ln a + \ln b$；$\ln$ 在 $(0, 1]$ 上取负值，在 $x \to 0^+$ 时趋向 $-\infty$。为什么用自然对数而不是以 2 为底？因为自然对数和微积分里 $\frac{d}{dx}\ln x = 1/x$ 配套，求导最干净；以 2 为底时信息量单位是 bit，自然对数单位是 nat，两者只差一个常数 $\ln 2$，不影响优化方向。

凸函数与 Jensen 不等式（预告）。f 是凸函数时，$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$。2.5 节会用它证明 KL 散度非负。

2. 完整推导

2.1 信息量的三公理

我们想给”事件 x 发生带来多少信息量”定义一个函数 $I(x)$，它必须满足三条直观公理：

1. **单调性**：越不可能发生的事件，信息量越大。$p(x)$ 越小，$I(x)$ 越大。
2. **非负性**：$I(x) \geq 0$。必然事件（$p = 1$）信息量为 0，不可能事件（$p = 0$）信息量无穷大。
3. **可加性**：两个独立事件同时发生的信息量等于各自信息量之和。若 x, y 独立，$p(x, y) = p(x)p(y)$，则 $I(x, y) = I(x) + I(y)$。

由公理 3，I 必须是概率的对数形式。把 $p(x, y) = p(x)p(y)$ 代入可加性要求：

$$I(p(x)p(y)) = I(p(x)) + I(p(y))$$

满足这个函数方程 $f(ab) = f(a) + f(b)$ 的连续解只有 $f(p) = c \ln p$。再由公理 2（$p = 1$ 时 $I = 0$，p 越小 I 越大），取负号并令 $c = 1$（单位 nat）：

$$I(x) = -\ln p(x)$$

验证三条公理：$p = 1$ 时 $I = 0$；$p \to 0$ 时 $I = -\ln p \to +\infty$；独立事件 $p(x, y) = p(x)p(y)$ 时 $I(x, y) = -\ln p(x)p(y) = -\ln p(x) - \ln p(y) = I(x) + I(y)$。三条全部满足。

2.2 熵：信息量的期望

熵是信息量的期望，也就是”平均而言一个事件能带来多少惊讶”：

$$H(p) = \mathbb{E}_{x\sim p}[I(x)] = -\sum_x p(x) \ln p(x)$$

熵衡量分布 p 的不确定度。两个极端：

- 确定性分布 $p(x_0) = 1$：$H = -\ln 1 = 0$，没有任何不确定性。
- 均匀分布 $p(x) = 1/k$：$H = -\sum_x \frac{1}{k} \ln \frac{1}{k} = \ln k$，不确定性最大。

为什么均匀分布熵最大? 用拉格朗日乘子证明。约束 $\sum_i p_i = 1$，最大化 $H = -\sum_i p_i \ln p_i$。构造拉格朗日函数:

$$\mathcal{L} = -\sum_i p_i \ln p_i + \lambda \left(\sum_i p_i - 1 \right)$$

对 p_j 求偏导并令其为 0:

$$\frac{\partial \mathcal{L}}{\partial p_j} = -(\ln p_j + 1) + \lambda = 0 \quad \Longrightarrow \quad \ln p_j = \lambda - 1$$

右边与 j 无关，所以所有 p_j 相等。结合归一约束得 $p_j = 1/k$。二阶导 $\partial^2 H / \partial p_j^2 = -1/p_j < 0$，确实是最大值。

2.3 交叉熵

交叉熵回答:“用分布 q 去编码服从分布 p 的事件，平均需要多少 nat”:

$$H(p, q) = -\sum_x p(x) \ln q(x)$$

注意它的不对称性: p 是真实分布，q 是模型预测分布。把它和熵对比: $H(p, q)$ 里 $-\ln q(x)$ 是”用 q 估计的信息量”，再用 p 加权平均。当 $q = p$ 时交叉熵就退化成熵 $H(p)$。$H(p, q) \neq H(q, p)$，这一点后面会反复用到。

2.4 KL 散度与三者关系

KL 散度（相对熵）定义为:

$$D_{\mathrm{KL}}(p \| q) = \sum_x p(x) \ln \frac{p(x)}{q(x)}$$

把对数拆开: $\ln \frac{p(x)}{q(x)} = \ln p(x) - \ln q(x)$，代入:

$$D_{\mathrm{KL}}(p \| q) = \sum_x p(x) \ln p(x) - \sum_x p(x) \ln q(x) = -H(p) + H(p, q)$$

移项得到整个深度学习训练目标的核心恒等式:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$$

为什么 KL 散度非负? 用 Jensen 不等式。把 $D_{\mathrm{KL}}(p \| q) = \mathbb{E}_p \left[\ln \frac{p(X)}{q(X)} \right] = -\mathbb{E}_p \left[\ln \frac{q(X)}{p(X)} \right]$。ln 是凹函数，由 Jensen:

$$\mathbb{E}_p \left[\ln \frac{q(X)}{p(X)} \right] \leq \ln \mathbb{E}_p \left[\frac{q(X)}{p(X)} \right] = \ln \sum_x p(x) \frac{q(x)}{p(x)} = \ln \sum_x q(x) = \ln 1 = 0$$

所以 $D_{\mathrm{KL}}(p \| q) \geq 0$，等号成立当且仅当 $p = q$（分布完全匹配）。这就是说交叉熵永远不小于真实分布的熵，q 越接近 p 交叉熵越小。

由于 $H(p)$ 是真实数据分布的熵，和模型参数 θ **无关**，所以:

$$\min_\theta H(p, q_\theta) \quad \Longleftrightarrow \quad \min_\theta D_{\mathrm{KL}}(p \| q_\theta)$$

优化交叉熵等价于让模型分布 q_θ 在 KL 意义上逼近真实分布 p。

2.5 最小化交叉熵 ⟺ 最大化似然

语言模型的真实分布 $p(x)$ 是什么？对 next-token 预测，给定前文 $x_{<t}$，真实分布是数据里观察到的下一个 token 的经验分布。在训练语料里，每个位置 t 对应的真实 token 是一个具体观测 x_t^*，相当于一个 one-hot 分布：$p_{\text{data}}(x = x_t^*) = 1$，其余为 0。

把 one-hot 代入交叉熵：

$$H(p_{\text{data}}, q_\theta) = -\sum_x p_{\text{data}}(x) \ln q_\theta(x) = -\ln q_\theta(x_t^*)$$

对整个数据集 N 个 token 取平均：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{t=1}^{N} \ln q_\theta(x_t^* \mid x_{<t})$$

而语料的似然（在”给定前文后各 token 条件独立”这个分解下）是 $q_\theta(\text{整个语料}) = \prod_t q_\theta(x_t^* \mid x_{<t})$，取负对数：

$$-\ln q_\theta(\text{语料}) = -\sum_t \ln q_\theta(x_t^* \mid x_{<t})$$

两者只差一个常数因子 $1/N$。所以：**最小化平均交叉熵 = 最大化对数似然 = 最小化负对数似然（NLL）**。这就是为什么所有语言模型都用同一个损失函数。

3. 直觉理解

直觉：交叉熵就是”你以为的概率分布”和”真实发生的概率分布”之间的编码成本差。真实答案只有一个（one-hot），模型却把概率摊薄在多个 token 上——你为”猜错”付出的代价就是 $-\ln q(x^*)$。

代入几个数感受一下：$q(x^*) = 0.9$ 时代价约 0.105；$q(x^*) = 0.5$ 时代价约 0.69；$q(x^*) = 0.1$ 时代价约 2.3；$q(x^*) \to 0$ 时代价趋向无穷。代价函数在”猜错”时惩罚极重，在”猜对但不自信”时惩罚较轻。这就是模型被”逼”着把概率集中到正确答案上的原因。

4. 常见误区

误区一：以为交叉熵和 MSE 差不多。对分类任务，MSE 在错答案离正确答案很远时梯度极小（因为 softmax 饱和区，$p_i(1-p_i) \to 0$，见 ch04），而交叉熵的复合梯度直接等于”预测概率减真实概率”——无论对错都有正常量级的梯度。这就是分类任务必须用 CE 而不是 MSE 的根本原因。

误区二：以为 KL 散度是距离。它不对称：$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$，也不满足三角不等式。它衡量”用 q 编码 p 多出的成本”，方向反了含义就反了。

误区三：以为 $q(x^*) = 0$ 时损失只是”很大”。实际上 $-\ln 0 = +\infty$，模型会被这个无穷梯度直接炸掉。工程上用 log_softmax 数值稳定实现（ch04），并且禁止把任何输出概率精确置 0。

5. 代码实现

下面这段代码手写交叉熵并对照 PyTorch 官方实现。注意 `F.cross_entropy` 内部是 `log_softmax + NLLLoss` 的融合，不是”先 softmax 再 log”——后者在 logits 很大时会下溢。

```python
import torch
import torch.nn.functional as F

# 手写交叉熵，对照 PyTorch 官方实现
logits = torch.tensor([[2.0, 1.0, 0.1, -1.0]])  # 4 类 logits
target = torch.tensor([0])  # 正确类别是第 0 类

# 第一步：手动 softmax 得到概率
probs = torch.softmax(logits, dim=-1)
print("softmax 概率:", probs.detach().round(decimals=4))

# 第二步：取正确类别的概率，取 -log，即 NLL
manual_nll = -torch.log(probs[0, target])
print(" 手动交叉熵:", manual_nll.item())

# 对照 PyTorch 官方：F.cross_entropy = log_softmax + NLLLoss
official_ce = F.cross_entropy(logits, target)
print(" 官方交叉熵:", official_ce.item())
assert torch.allclose(manual_nll, official_ce, atol=1e-6)

# 数值稳定版：log_softmax 直接算 log(q)，避免先 softmax 再 log 下溢
log_probs = F.log_softmax(logits, dim=-1)
stable_nll = -log_probs[0, target]
print(" 数值稳定版交叉熵:", stable_nll.item())
assert torch.allclose(stable_nll, official_ce, atol=1e-6)

# 验证：最小化 CE = 最大化正确类概率。增大正确类 logit，loss 应单调下降。
print("\n增大正确类 logit 后的 loss:")
for delta in [0, 1, 2, 5]:
    new_logits = logits.clone()
    new_logits[0, 0] += delta
    print(f"  +{delta}: loss = {F.cross_entropy(new_logits, target).item():.4f}")

# 验证 KL 非负：构造 p, q，计算 D_KL(p||q)，应 >= 0
p = torch.tensor([0.5, 0.3, 0.2])
q = torch.tensor([0.6, 0.2, 0.2])
kl = (p * (torch.log(p) - torch.log(q))).sum()
print(f"\nD_KL(p||q) = {kl.item():.4f} (应 >= 0)")
# 当 q=p 时 KL 应为 0
kl_self = (p * (torch.log(p) - torch.log(p))).sum()
print(f"D_KL(p||p) = {kl_self.item():.6f} (应 = 0)")
```

6. 小结

1. 信息量 $I(x) = -\ln p(x)$ 由三条公理（单调、非负、可加）唯一确定；熵是信息量的期望，均匀分布时最大。
2. 交叉熵 $H(p, q)$ 是用 q 编码 p 的平均成本；核心恒等式 $H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$，KL 散度由 Jensen 不等式证明非负。
3. 由于 $H(p)$ 与模型无关，最小化 CE 等价于最小化 KL 散度。
4. 对 one-hot 标签，CE 退化为 $-\ln q(x^*)$，即负对数似然。最小化 CE = 最大化似然。

7. 延伸阅读/进一步思考

- 信息论的源头是 Shannon 1948 年的《A Mathematical Theory of Communication》，信息量三公理和熵的定义都出自那里；想理解”为什么是对数”可以读 Shannon 原文第一节。
- 交叉熵和 KL 散度的方向问题在无监督学习里很关键：VAE 用 $D_{\mathrm{KL}}(q \| p)$（反向 KL），倾向”mode-covering”；GAN/对比学习用另一种方向，倾向”mode-seeking”。后面学到概率模型时会再遇到。
- 本讲的”one-hot 标签”在实践中会被 Label Smoothing 软化，这是 ch17 的主题。

8. 思考题

基础题

1. 两个分布 $p=(0.5,0.5)$ 和 $q=(0.9,0.1)$，分别计算 $D_{\mathrm{KL}}(p\|q)$ 和 $D_{\mathrm{KL}}(q\|p)$，验证它们不相等、且都非负。

进阶题

2. 为什么分类任务用 CE 而不用 MSE? 从梯度角度推导: 若最后一层是 softmax + MSE（$L=\frac{1}{2}\sum_i(y_i-t_i)^2$），求 $\partial L/\partial z_j$，并说明为什么在 softmax 饱和时这个梯度会消失。提示：先用 ch04 的 softmax 雅可比 $J_{ij}=p_i(\delta_{ij}-p_j)$。
3. 若模型对正确答案的预测概率为 0，交叉熵是多少？工程上 log_softmax 是怎么避免 $-\ln 0$ 的?

开放题

4. KL 散度不对称：$D_{\mathrm{KL}}(p\|q)$ 和 $D_{\mathrm{KL}}(q\|p)$ 在最优解附近的行为有什么不同?（提示：一个会让 q 试图覆盖 p 的所有峰，另一个会让 q 只聚焦 p 最高的那个峰。）这对生成模型的训练目标选择意味着什么?

第 3 讲 梯度下降与反向传播：链式法则的矩阵形式

> **本讲问题**：一个两层 MLP 有几十万个参数，我们怎么可能在毫秒内算出损失对每个参数的梯度？反向传播为什么比”逐个求导”快几个数量级？这一讲把两层 MLP 的每一步梯度都推到底，并解释梯度消失/爆炸的代数根源。

1. 数学工具/预备知识

链式法则。一元情形: 若 $L=f(g(x))$, 则 $\frac{dL}{dx}=f'(g(x))\cdot g'(x)$。多元向量情形: 若 $z=g(x)$、$L=f(z)$, 则 $\frac{\partial L}{\partial x}=\frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$，其中 $\frac{\partial z}{\partial x}$ 是雅可比矩阵。

ch01 的矩阵求导微分法。记住结论即可：前向 $Y=XW$，反向对 W 的梯度是 $X^\top G_Y$，对 X 的梯度是 $G_Y W^\top$。本讲直接用。

逐元素激活函数的雅可比。若 $h=\sigma(z)$ 且 σ 逐元素作用（$h_i=\sigma(z_i)$，各维独立），则雅可比是对角矩阵 $\frac{\partial h}{\partial z}=\mathrm{diag}(\sigma'(z))$，非对角元全为 0。因此向量形式的链式法则简化为逐元素乘：$\frac{\partial L}{\partial z}=\frac{\partial L}{\partial h}\odot\sigma'(z)$。

计算图。把前向计算拆成一串基本运算节点（矩阵乘、加偏置、逐元素激活、softmax、loss），每个节点缓存自己的输入和输出。反向传播就是在这张图上从 loss 节点逆序走一遍。

2. 完整推导

2.1 计算图与反向传播的一般形式

前向传播是把输入 x 通过一串运算 $f_1,f_2,\dots,f_L$ 得到损失 L。每个节点 i 的输出是 $z_i=f_i(z_{i-1})$。

反向传播的目标是求 $\frac{\partial L}{\partial z_i}$。定义**上游梯度** $g_i=\frac{\partial L}{\partial z_i}$——它表示”最终损失 L 对这个节点输出有多敏感”。最终节点 $z_L=L$，所以 $g_L=1$（标量对自己求导）。

对任意节点 $z_i=f_i(z_{i-1})$，由链式法则:

$$g_{i-1}=\frac{\partial L}{\partial z_{i-1}}=\frac{\partial L}{\partial z_i}\frac{\partial z_i}{\partial z_{i-1}}=g_i\cdot J_i$$

其中 $J_i=\frac{\partial z_i}{\partial z_{i-1}}$ 是该节点的雅可比。反向传播就是从 $g_L=1$ 出发，逐个节点右乘雅可比，一路传回到输入。

为什么比逐个求导快? 如果对每个参数独立地”前向算一次、数值差分一次”求梯度，N 个参数就要前向 N 次，总复杂度 $O(N\cdot\text{前向})$。而反向传播只前向一次（缓存中间结果），再逆序算一次，总复杂度和前向同阶 $O(\text{前向})$。差别来自：前向算过的 z_i 全部缓存复用，每个节点的梯度只算一次，所有参数共享这次反向。

2.2 两层 MLP 完整推导

设输入 $x \in \mathbb{R}^d$, 隐藏层 $h \in \mathbb{R}^h$, 输出 $y \in \mathbb{R}^c$, 损失 L 为交叉熵。参数: $W_1 \in \mathbb{R}^{h\times d}$, $b_1 \in \mathbb{R}^h$, $W_2 \in \mathbb{R}^{c\times h}$, $b_2 \in \mathbb{R}^c$。

前向传播:

$$z_1 = W_1 x + b_1, \quad h = \sigma(z_1)$$

$$z_2 = W_2 h + b_2, \quad y = \text{softmax}(z_2)$$

$$L = -\ln y_k \quad (k \text{ 是正确类别，标签 one-hot: } t_k = 1, \text{ 其余为 } 0)$$

反向传播（从后往前五步）:

第一步: 求 $\frac{\partial L}{\partial z_2}$。这是 softmax 加交叉熵的复合梯度。ch04 会完整推导，这里给出结论并复述要点: $L = -\sum_j t_j \ln p_j$ 与 softmax 复合后，$p_j = \text{softmax}(z_2)_j$，链式法则里 softmax 雅可比的 $p_i(\delta_{ij} - p_j)$ 项恰好和 $-\frac{t_i}{p_i}$ 约掉，得到:

$$\frac{\partial L}{\partial z_{2,j}} = p_j - t_j$$

写成向量: $g_2 = \frac{\partial L}{\partial z_2} = y - t \in \mathbb{R}^c$。对 one-hot 标签，就是”预测概率减去第 k 位为 1 的 one-hot 向量”。这个结果干净的关键: 没有 $p_j(1-p_j)$ 饱和因子。

第二步: 求 $\frac{\partial L}{\partial W_2}$ **和** $\frac{\partial L}{\partial b_2}$。由 ch01 微分法, $z_2 = W_2 h + b_2$, $dz_2 = dW_2\, h + db_2$。$dL = g_2^\top dz_2$。对 W_2:

$$dL = g_2^\top dW_2\, h = \text{tr}(g_2^\top dW_2\, h) = \text{tr}(h g_2^\top dW_2) = \text{tr}((g_2 h^\top)^\top dW_2)$$

读出 $\frac{\partial L}{\partial W_2} = g_2 h^\top$。对 b_2，db_2 直接加在 z_2 上（本例是单样本，无广播），$dL = g_2^\top db_2$，读出 $\frac{\partial L}{\partial b_2} = g_2$。

第三步: 求 $\frac{\partial L}{\partial h}$。 $dz_2 = W_2 dh$（固定 W_2），$dL = g_2^\top W_2 dh$，读出:

$$\frac{\partial L}{\partial h} = W_2^\top g_2$$

第四步: 求 $\frac{\partial L}{\partial z_1}$。 $h = \sigma(z_1)$ 是逐元素激活，雅可比是对角阵，用逐元素乘:

$$g_1 = \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \sigma'(z_1) = (W_2^\top g_2) \odot \sigma'(z_1)$$

第五步: 求 $\frac{\partial L}{\partial W_1}$ **和** $\frac{\partial L}{\partial b_1}$。同第二步:

$$\frac{\partial L}{\partial W_1} = g_1 x^\top, \qquad \frac{\partial L}{\partial b_1} = g_1$$

把五步连起来看: g_2 是最末端的误差信号，它经过 $W_2^\top$ 传回隐藏层，被激活导数 σ' 逐元素缩放，再经过 $W_1^\top$ 传回输入。这就是”梯度从出口倒灌回入口”。

2.3 批量情形下偏置的梯度

实际训练一个 batch 有 B 个样本，$Z_2 = XW_2^\top + b_2$，其中 b_2 被广播加到每一行。此时 $\frac{\partial L}{\partial b_2}$ 要对样本维求和：

$$\frac{\partial L}{\partial b_2} = \sum_{b=1}^{B} g_2^{(b)}$$

直觉：同一个偏置被 B 个样本共用，每个样本都对它有一份贡献，梯度要全部加起来。而 $\frac{\partial L}{\partial W_2} = \sum_b g_2^{(b)} (h^{(b)})^\top$ 也是逐样本贡献累加。

2.4 梯度消失与爆炸的范数分析

考虑 L 层全连接网络，每层权重矩阵为 W_l，激活雅可比为对角阵 $D_l = \mathrm{diag}(\sigma'(z_l))$。反向传播时，梯度要从第 L 层一路穿过 L 层矩阵乘法传到第 1 层：

$$g_{\text{第 1 层}} = W_1^\top D_1 \, W_2^\top D_2 \cdots W_L^\top g_L$$

取范数并反复用三角不等式 $\|AB\| \le \|A\|\|B\|$：

$$\|g_1\| \le \|W_1^\top\| \cdot \|D_1\| \cdot \|W_2^\top\| \cdot \|D_2\| \cdots \|W_L^\top\| \cdot \|g_L\|$$

记每层的”放大系数” $c_l = \|W_l^\top\| \cdot \|D_l\|$。若所有 $c_l \approx c$，则 $\|g_1\| \lesssim c^L \|g_L\|$。

- $c < 1$：c^L 指数衰减 → **梯度消失**，前面层学不到东西。
- $c > 1$：c^L 指数增长 → **梯度爆炸**，参数直接 NaN。

对 sigmoid，$\sigma'(z) \le 0.25$（最大值在 $z = 0$ 处），若 $\|W_l\| \approx 1$，则 $c \lesssim 0.25$，三层就衰减到 $0.25^3 \approx 0.016$。这就是深网络/RNN 梯度消失的根源（ch05 沿时间维重复这个分析）。ReLU 的 $\sigma'(z) = \mathbf{1}[z > 0]$，约一半时间为 1，配合合理初始化（ch20）可以把每层 c_l 控制在 1 附近；残差连接（ch12）则直接在连乘里加了一个恒等项 1，从结构上根治消失。

3. 直觉理解

直觉：反向传播就是”把损失的误差像水一样从出口倒灌回去”，每经过一个阀门（矩阵乘法、激活函数），就告诉上游”你该为最终的错负责多少”。为什么比逐个求导快？因为前向算过的所有中间值 h, z_1 都被缓存下来了，反向时直接拿来用，不需要重新算。复杂度是 $O(L)$，和前向一样，而不是 $O(N \cdot L)$。

梯度消失的范数分析也很直白：每层阀门如果把水流拧小一半（$c = 0.5$），10 层后只剩千分之一；残差连接就是在每一层旁边修一条”旁路”，让水至少能原样流过去（系数 1）。

4. 常见误区

误区一：以为反向传播是”特殊算法”。它就是链式法则在计算图上的逆序执行，没有任何神秘成分。

误区二：把梯度消失归因于”网络太深”。真正的根因是**每层雅可比谱半径的连乘** c^L——如果每层 $c_l \approx 1$，再深也不消失；如果每层 $c_l \approx 0.5$，三层就消失到 $1/8$。初始化（ch20）、残差连接（ch12）、归一化（ch12）都是在控制这个连乘积。

误区三：以为梯度爆炸比梯度消失更难处理。其实爆炸好处理——梯度裁剪（把全局范数超过阈值的梯度整体缩放）是工程上的标准做法；消失是结构性的，靠裁剪救不回来，必须改架构。

5. 代码实现

下面手写两层 MLP 的反向传播，再和 PyTorch autograd 对照。注意几点：手动实现时 `g2 = probs.clone(); g2[y_true] -= 1` 就是 $y - e_k$；ReLU 导数是 `(z1 > 0).float()`，$z_1 = 0$ 处取 0（约定）；批量偏置梯度在单样本例子里就是 g_2 本身。

```
import torch
import torch.nn.functional as F

# 手写两层 MLP 的反向传播，对照 autograd
torch.manual_seed(42)
d, h, c = 4, 5, 3
x = torch.randn(d)
y_true = torch.tensor(1)  # 正确类别

# 参数
W1 = torch.randn(h, d) * 0.5
b1 = torch.zeros(h)
W2 = torch.randn(c, h) * 0.5
b2 = torch.zeros(c)

# ---- 前向 ----
z1 = W1 @ x + b1
h_act = torch.relu(z1)
z2 = W2 @ h_act + b2
probs = F.softmax(z2, dim=0)
loss = -torch.log(probs[y_true])
print(" 前向 loss:", loss.item())

# ---- 手写反向 ----
# 第 1 步：softmax+CE 复合梯度 g2 = y - onehot
g2 = probs.clone()
g2[y_true] -= 1.0

# 第 2 步：dL/dW2 = g2 h^T, dL/db2 = g2
gW2 = g2.unsqueeze(1) @ h_act.unsqueeze(0)  # (c, h)
gb2 = g2.clone()

# 第 3 步：dL/dh = W2^T g2
gh = W2.T @ g2  # (h,)

# 第 4 步：ReLU 导数 z1>0 处为 1，否则 0；逐元素乘
g1 = gh * (z1 > 0).float()

# 第 5 步：dL/dW1 = g1 x^T, dL/db1 = g1
gW1 = g1.unsqueeze(1) @ x.unsqueeze(0)  # (h, d)
gb1 = g1.clone()

# ---- 对照 autograd ----
# 重新建一份叶子参数，开启梯度，跑同一条前向
W1a = W1.clone().requires_grad_(True)
b1a = b1.clone().requires_grad_(True)
W2a = W2.clone().requires_grad_(True)
b2a = b2.clone().requires_grad_(True)
z1_a = W1a @ x + b1a
h_a = torch.relu(z1_a)
z2_a = W2a @ h_a + b2a
loss_a = F.cross_entropy(z2_a.unsqueeze(0), y_true.unsqueeze(0))
loss_a.backward()

print("\n手写 vs autograd 梯度差异:")
```

```
print("  W1:", torch.allclose(gW1, W1a.grad, atol=1e-5))
print("  b1:", torch.allclose(gb1, b1a.grad, atol=1e-5))
print("  W2:", torch.allclose(gW2, W2a.grad, atol=1e-5))
print("  b2:", torch.allclose(gb2, b2a.grad, atol=1e-5))

# 观察梯度消失：把 W_hh 谱半径压到 < 1，重复 100 层后梯度范数指数衰减
torch.manual_seed(0)
L = 100
grads_norm = []
x = torch.randn(16, requires_grad=True)
W = torch.eye(16) * 0.9  # 谱半径 0.9 < 1
h = x
for _ in range(L):
    h = torch.tanh(h @ W)
h.sum().backward()
print(f"\n100 层 tanh 网络（每层谱半径 0.9）后 x 的梯度范数: {x.grad.norm().item():.2e}")
print(f" 理论下界 ~ 0.9^100 ≈ {0.9**100:.2e}")
```

6. 小结

1. 反向传播是链式法则在计算图上的逆序执行，前向缓存中间值，反向复杂度与前向同阶，比逐参数数值差分快 N 倍。
2. 两层 MLP 关键梯度：$\frac{\partial L}{\partial W_2} = g_2 h^\top$，$\frac{\partial L}{\partial h} = W_2^\top g_2$，$\frac{\partial L}{\partial z_1} = (W_2^\top g_2) \odot \sigma'(z_1)$；批量时偏置梯度对样本维求和。
3. 梯度消失/爆炸的根源是每层雅可比谱半径的连乘 c^L，指数衰减或增长。
4. softmax+ 交叉熵的复合梯度 $g_2 = y - e_k$ 极其干净，没有饱和因子，这是分类网络训练稳定的关键。

7. 延伸阅读/进一步思考

- 反向传播的现代形式由 Rumelhart, Hinton, Williams 1986 年推广开来；想了解”为什么它没被更早发现”可读 LeCun 等人关于反向传播历史与高效实现的综述。
- 梯度消失的系统性解决方案：残差连接（ch12，He 等 2015）、归一化层（ch12）、合理初始化（ch20，Xavier/He）、门控结构（ch05 LSTM）。它们本质上都在控制 $c_l \to 1$。
- 梯度裁剪的标准做法是”按全局范数缩放”，见 Pascanu 等 2013 年关于 RNN 梯度爆炸的论文；它和 Adam 等自适应优化器（ch19）配合使用。

8. 思考题

基础题

1. 激活函数换成 ReLU 后，$\sigma'(z) = \mathbf{1}[z > 0]$。在反向传播中 $z_1 = 0$ 的位置梯度是多少？这个位置对应的神经元之后还能恢复激活吗？

进阶题

2. 推导：若最后一层损失是 MSE 而不是 CE，即 $L = \frac{1}{2}\sum_j (y_j - t_j)^2$，且 $y = \mathrm{softmax}(z_2)$，求 $\partial L / \partial z_{2,j}$。提示：先用 ch04 的雅可比 $J_{ij} = p_i(\delta_{ij} - p_j)$ 展开链式法则，会看到结果里残留 $p_i(1-p_j)$ 项，说明饱和时梯度会消失。
3. 为什么梯度爆炸在实践中比梯度消失好处理？（提示：一个是数值问题可裁剪，一个是结构问题。）

开放题

4. 范数分析给出的是上界 $\|g_1\| \le \prod_l c_l \|g_L\|$。如果权重矩阵的谱半径都略大于 1（比如 1.1），梯度会指数爆炸；但实践中 LayerNorm 会把每层激活重新拉回单位方差。从这个角度分析：为什么 LayerNorm 能把”每层 c_l 乘积”这个不受控的连乘变成近似 1 的稳定量？（预告 ch12。）

第 4 讲 Softmax 与归一化：从 logits 到概率分布

本讲问题：为什么注意力权重必须经过 Softmax 才能乘到 V 上？Softmax 有什么性质让它和交叉熵组成”梯度最干净”的组合？这一讲把 Softmax 的雅可比、复合梯度、温度系数全部推到底，并解释 ch08 缩放为什么必要。

1. 数学工具/预备知识

指数函数 e^x。单调递增、处处可导，$\frac{d}{dx}e^x = e^x$；$e^x > 0$ 恒成立，$x \to -\infty$ 时 $e^x \to 0$。这两条保证 Softmax 输出恒正。

概率单纯形（probability simplex）。k 维概率分布集合 $\{p \in \mathbb{R}^k : p_i \geq 0,\ \sum_i p_i = 1\}$。它是一个 $(k-1)$ 维的三角形（$k=3$ 时是平面上的三角形区域）。Softmax 的作用就是把任意实数向量 $z \in \mathbb{R}^k$ 映射到这个单纯形内部。

ch02 的交叉熵。$L = -\sum_k t_k \ln p_k$，t 是真实分布。

偏导数链式法则。对复合函数 $L(p(z))$，$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i}\frac{\partial p_i}{\partial z_j}$。

2. 完整推导

2.1 Softmax 的定义与平移不变性

对 k 维 logits 向量 $z \in \mathbb{R}^k$，Softmax 定义为：

$$p_i = \text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

分母是所有指数项之和，保证 $\sum_i p_i = 1$；分子是单项指数，保证 $p_i > 0$。输出落在概率单纯形内部。

平移不变性：给所有 z_i 同时加上常数 C：

$$\frac{e^{z_i+C}}{\sum_j e^{z_j+C}} = \frac{e^C e^{z_i}}{e^C \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

e^C 分子分母约掉，输出不变。也就是说 Softmax 只关心 logits 之间的**相对差**，不关心整体平移。这个性质是数值稳定的关键（2.5 节）。

2.2 Softmax 的雅可比矩阵

求 $\frac{\partial p_i}{\partial z_j}$。记 $S = \sum_m e^{z_m}$，则 $p_i = e^{z_i}/S$。分两种情况。

当 $i = j$： 用商法则，分子 e^{z_i} 对 z_i 求导是 e^{z_i}：

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i}\cdot S - e^{z_i}\cdot e^{z_i}}{S^2} = \frac{e^{z_i}}{S} - \frac{e^{2z_i}}{S^2} = p_i - p_i^2 = p_i(1-p_i)$$

当 $i \neq j$： 分子 e^{z_i} 对 z_j 求导是 0（$i \neq j$），但分母 S 对 z_j 求导是 e^{z_j}：

$$\frac{\partial p_i}{\partial z_j} = \frac{0\cdot S - e^{z_i}\cdot e^{z_j}}{S^2} = -\frac{e^{z_i}e^{z_j}}{S^2} = -p_i p_j$$

写成矩阵形式 $J \in \mathbb{R}^{k\times k}$：

$$J_{ij} = \frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

这里 δ_{ij} 是 Kronecker 符号（$i=j$ 时为 1，否则为 0）。

这个雅可比告诉我们什么？ 对角元 $p_i(1-p_i)$ 在 $p_i \to 0$ 或 $p_i \to 1$ 时都趋近 0。也就是说，一旦 Softmax 输出接近 one-hot（某个 $p_i \approx 1$，其余 ≈ 0），雅可比的所有元素都趋近 0——梯度消失。这就是 ch08 要对打分做缩放的根本原因。

2.3 Softmax + 交叉熵的复合梯度

设损失 $L = -\sum_k t_k \ln p_k$，t 是 one-hot 标签。求 $\frac{\partial L}{\partial z_j}$。

先用链式法则展开：

$$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i}\frac{\partial p_i}{\partial z_j}$$

第一步算 $\frac{\partial L}{\partial p_i} = -\frac{t_i}{p_i}$（对 $L = -\sum_k t_k \ln p_k$ 求导，只有 $k=i$ 项贡献）。

第二步代入雅可比 $J_{ij} = p_i(\delta_{ij} - p_j)$：

$$\frac{\partial L}{\partial z_j} = \sum_i \left(-\frac{t_i}{p_i}\right) \cdot p_i(\delta_{ij} - p_j) = -\sum_i t_i(\delta_{ij} - p_j)$$

p_i 约掉了——这是关键的一步。展开求和：

$$= -\sum_i t_i\delta_{ij} + \sum_i t_i p_j = -t_j + p_j \sum_i t_i$$

由于 $\sum_i t_i = 1$（one-hot 标签归一）：

$$\frac{\partial L}{\partial z_j} = p_j - t_j$$

结论：Softmax+CE 的复合梯度就是”预测概率减真实标签”。对比 2.2 节单独 Softmax 的雅可比 $p_i(\delta_{ij}-p_j)$——那些 p_i 因子在和 $-\frac{t_i}{p_i}$ 相乘时恰好约掉了。所以复合梯度里**没有饱和项**：无论 p_j 接近 0 还是 1，梯度都是 $p_j - t_j$，量级正常。这就是分类网络敢把 logits 放得很大、却不会梯度消失的原因。

2.4 温度系数

若在 Softmax 前除以温度 T：

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \to 0$：z_i/T 的差异被无限放大，分布趋向 one-hot（argmax）。
- $T = 1$：标准 Softmax。
- $T \to \infty$：$z_i/T \to 0$，所有 $e^{z_i/T} \to 1$，分布趋向均匀。

从方差角度看：logits 经过温度缩放后，有效 logit 差缩小 T 倍，输出分布的熵升高（更均匀）。推理时调高 T 让生成更多样，调低 T 让生成更确定。训练时 $T=1$。

2.5 数值稳定 Softmax

直接算 e^{z_i} 有两个风险：z_i 很大时 e^{z_i} 上溢到 ∞；z_i 很小时 e^{z_i} 下溢到 0（分母变成 0）。利用 2.1 节的平移不变性：令 $m = \max_i z_i$，令 $\tilde{z}_i = z_i - m$，则 $\tilde{z}_i \le 0$，$e^{\tilde{z}_i} \in (0, 1]$，最大的指数项是 $e^0 = 1$，不会上溢：

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}$$

分母至少有一项是 $e^0 = 1$,也不会下溢成 0。工程上进一步用 `log_softmax` 直接计算 $\log p_i = \tilde{z}_i - \log \sum_j e^{\tilde{z}_j}$，避免先 exp 再 log 的两次精度损失。

3. 直觉理解

直觉：Softmax 是一个”投票器”——每个 logit 是一个候选人的”得票指数”，然后归一化成百分比。它的平移不变性意味着”大家同时加 100 分”不改变排名，只改变自信程度。

单独 Softmax 的雅可比 $p_i(\delta_{ij} - p_j)$ 之所以在饱和区消失，是因为当一个候选人几乎全拿票时，你再微调任何人的分数，结果都几乎不变——函数平坦了。而 Softmax+CE 的复合梯度 $p_j - t_j$ 之所以干净，是因为交叉熵损失对”错得多”惩罚极重（ch02），两个函数的导数里那些饱和因子恰好互相约掉——这是两个函数天作之合的结果。

4. 常见误区

误区一：以为 Softmax 只是”把数归一化成概率”。关键陷阱是：**单独对 Softmax 求梯度会饱和**（$p_i \to 0$ 或 1 时 $p_i(1-p_i) \to 0$），但和 CE 组合后就不饱和了。如果你在中间层单独用 Softmax（比如中间特征归一化），梯度仍然会消失。

误区二：以为温度 T 是训练时调的超参。训练时 $T = 1$；温度只在推理（采样）时调，用来控制生成的锐度。

误区三：以为除以最大值是为了”提高精度”。它的首要目的是防止上溢/下溢；顺便它也不改变输出（平移不变性）。

5. 代码实现

下面手写数值稳定 Softmax 并验证复合梯度。注意 `log_softmax` 是稳定实现，不要手写 `log(softmax)`；温度缩放要在 softmax 之前做。

```
import torch
import torch.nn.functional as F

# 数值稳定 Softmax 手动实现
def stable_softmax(z, dim=-1):
    z_max, _ = z.max(dim=dim, keepdim=True)
    z_shifted = z - z_max  # 平移，防止上溢；平移不改变输出
    exp_z = torch.exp(z_shifted)
    return exp_z / exp_z.sum(dim=dim, keepdim=True)

z = torch.tensor([[1000.0, 1001.0, 999.0]])  # 大数值，直接 exp 会 inf
print(" 稳定 softmax:", stable_softmax(z).detach().round(decimals=4))
print(" 官方 softmax:", F.softmax(z, dim=-1).detach().round(decimals=4))
assert torch.allclose(stable_softmax(z), F.softmax(z, dim=-1), atol=1e-5)

# 验证平移不变性
z2 = z + 1000
print("\n加常数后 softmax 不变:",
      torch.allclose(stable_softmax(z), stable_softmax(z2), atol=1e-5))
```

```python
# 验证复合梯度: dL/dz = p - t
z3 = torch.tensor([1.0, 2.0, 0.5], requires_grad=True)
target = torch.tensor([0, 1, 0], dtype=torch.float32)  # one-hot

p = F.softmax(z3, dim=0)
loss = F.cross_entropy(z3.unsqueeze(0), target.argmax().unsqueeze(0))
loss.backward()
print("\n复合梯度 (autograd):", z3.grad.detach().round(decimals=4))
print("p - t (手动):       ", (p - target).detach().round(decimals=4))
assert torch.allclose(z3.grad, p - target, atol=1e-5)

# 温度系数效果: T 越大越均匀
logits = torch.tensor([3.0, 1.0, -1.0])
print("\n温度对分布的影响:")
for T in [0.5, 1.0, 2.0, 10.0]:
    p_T = F.softmax(logits / T, dim=0)
    ent = -(p_T * torch.log(p_T + 1e-12)).sum()
    print(f"  T={T:5.1f}: 分布 ={p_T.tolist()}  熵 ={ent:.3f}")

# 验证饱和区: 单独 softmax 的雅可比趋近 0
z_sat = torch.tensor([100.0, 0.0, 0.0], requires_grad=True)
p_sat = F.softmax(z_sat, dim=0)
p_sat.sum().backward()  # 对 p1 求梯度（单独 softmax，不加 CE）
print("\n单独 softmax 在饱和区的梯度 (应 ≈0):", z_sat.grad.detach().round(decimals=6))
```

6. 小结

1. Softmax 把任意实数向量映射到概率单纯形，具有平移不变性——只看相对差。
2. 单独 Softmax 的雅可比 $J_{ij} = p_i(\delta_{ij} - p_j)$，在饱和区梯度趋近 0。
3. Softmax+CE 的复合梯度 $\partial L/\partial z_j = p_j - t_j$，饱和因子被约掉，完全消除饱和问题。
4. 数值稳定做法：先减最大值再指数；温度系数在推理时控制生成锐度。

7. 延伸阅读/进一步思考

- Softmax 的饱和问题是 Ch08 引入 $\sqrt{d_k}$ 缩放的直接动机；想理解完整因果链，把本讲 2.2 和 Ch08 2.3 对照着读。
- Label Smoothing（ch17）把 one-hot 标签 t 软化成 $(1-\epsilon)\cdot\text{onehot} + \epsilon/k$，动机之一就是防止 Softmax 输出过度自信导致饱和。
- 若把单标签 Softmax 换成多标签 Sigmoid（$\sigma(z_i) = 1/(1+e^{-z_i})$），复合梯度不再是 $p-t$ 的干净形式，每个标签独立用 BCE 损失。

8. 思考题

基础题

1. 若 Softmax 输入 $z = (100, 0, 0)$，输出约等于什么？此时对 z_1 的单独 Softmax 雅可比 $\partial p_1/\partial z_1 = p_1(1-p_1)$ 约等于多少？

进阶题

2. 为什么推理时调高温度会让生成更随机？从输出分布的熵角度解释：$T \to \infty$ 时熵趋向多少？
3. 如果把 Softmax 换成 Sigmoid(多标签分类,每个 logit 独立过 σ,用 BCE 损失 $L = -\sum_i [t_i \ln \sigma(z_i) + (1-t_i)\ln(1-\sigma(z_i))]$，求 $\partial L/\partial z_i$，它还是 $p_i - t_i$ 这个形式吗？

开放题

4. 实验发现：当 logits 方差很大时，即使 Softmax+CE 复合梯度本身不饱和，训练仍然不稳。结合 ch08 的方差推导解释：大 logits 除了让 Softmax 饱和，还会通过什么途径影响训练？（提示：想想注意力权重一旦接近 one-hot，value 加权求和就退化成了只取一个 value，模型的”软检索”能力会发生什么变化？）

第 5 讲 序列建模简史：RNN 与 LSTM 的能力边界

> **本讲问题**：在 Transformer 出现之前，RNN 是序列建模的标准答案。为什么它训不深？LSTM 到底解决了什么问题？理解这些历史包袱，才能理解 Transformer 为什么要抛弃循环、改用全局注意力。

1. 数学工具/预备知识

序列数据的形式化。一条序列 $x_{1:T} = (x_1, x_2, \dots, x_T)$，每个 $x_t \in \mathbb{R}^d$ 是一个向量（比如 token embedding）。语言模型的目标是建模条件概率 $p(x_t \mid x_{<t})$。

ch03 的反向传播与梯度消失。本讲把”层间连乘导致指数衰减”这个分析搬到”时间维”上——同一套数学，只是连乘方向从”层”变成了”时间步”。

逐元素激活函数。$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$，导数 $\tanh'(z) = 1 - \tanh^2(z) \in (0, 1]$，在 $z = 0$ 处取最大值 1，在饱和区 $|z| \gg 1$ 时趋近 0。Sigmoid $\sigma(z) \in (0, 1)$，导数 $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le 0.25$。

门控直觉。Sigmoid 输出 $(0, 1)$，可以当成一个”软开关”：0 是完全关闭，1 是完全打开。门控机制就是用一个 Sigmoid 输出逐元素地控制”让多少信息通过”。

2. 完整推导

2.1 序列建模的形式化

序列模型要建模条件概率 $p(x_t \mid x_{<t})$。RNN 用一个隐藏状态 h_t 递归地压缩全部历史：

$$h_t = f(h_{t-1}, x_t), \quad y_t = g(h_t)$$

其中 f 是转移函数，g 是输出函数。对最简单的 Elman RNN：

$$h_t = \tanh(W_{ih} x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{ho} h_t + b_o$$

关键特点：**所有时间步共享同一套参数** W_{hh}, W_{ih}；当前隐藏状态只依赖上一步隐藏状态和当前输入——信息沿时间步单向流动。

2.2 BPTT：沿时间反向传播

总损失 $L = \sum_{t=1}^{T} L_t$，其中 L_t 是第 t 步预测的损失。求 $\frac{\partial L}{\partial W_{hh}}$，要把梯度沿时间传回所有历史步：

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W_{hh}}$$

而单个 L_t 对 W_{hh} 的梯度，要对从时刻 1 到时刻 t 的每一步求贡献：

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

关键项是雅可比连乘 $\frac{\partial h_t}{\partial h_k}$，它衡量”时刻 k 的隐藏状态变动，经过 $t-k$ 步后，对时刻 t 的影响”：

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$$

这就是沿时间展开的链式法则，叫 BPTT（Backpropagation Through Time）。

2.3 梯度消失的数学证明

对 Elman RNN，$h_j = \tanh(W_{hh} h_{j-1} + \cdots)$。逐元素求导：

$$\frac{\partial h_j}{\partial h_{j-1}} = \operatorname{diag}(\tanh'(z_j)) \cdot W_{hh}$$

其中 $z_j = W_{ih} x_j + W_{hh} h_{j-1} + b_h$。取谱范数上界：

$$\left\| \frac{\partial h_j}{\partial h_{j-1}} \right\| \le \max_i |\tanh'(z_{j,i})| \cdot \|W_{hh}\|$$

由于 $\tanh'(z) = 1 - \tanh^2(z) \le 1$，且在饱和区 $|\tanh(z)| \approx 1$ 时 $\tanh'(z) \approx 0$。记 $c = \max_i \tanh'(z_{j,i}) \cdot \|W_{hh}\|$。若 $\|W_{hh}\| < 1$ 且激活经常饱和，则 $c < 1$。经过 $t-k$ 步：

$$\left\| \frac{\partial h_t}{\partial h_k} \right\| \le c^{t-k} \to 0 \quad (t-k \to \infty)$$

遥远过去的梯度信号传到当前位置时已经指数级消失。RNN 实际只能记住”最近几步”的信息。这个分析和 ch03 层间梯度消失完全同构，只是把”层数 L“换成了”时间距离 $t-k$ “。

2.4 LSTM 的梯度高速公路

LSTM 的核心思想：引入一个**细胞状态** c_t，让梯度可以在不经过 tanh 饱和变换的情况下直接流过。六组公式：

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \quad \text{（遗忘门）}$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i) \quad \text{（输入门）}$$

$$\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c) \quad \text{（候选细胞）}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{（细胞状态更新）}$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \quad \text{（输出门）}$$

$$h_t = o_t \odot \tanh(c_t)$$

关键区别在细胞状态的梯度。对 $c_t = f_t \odot c_{t-1} + \ldots$ 两边关于 c_{t-1} 求导（固定其他项）：

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

遗忘门 f_t 是 Sigmoid 输出，取值 $(0,1)$。如果训练学得让 $f_t \approx \mathbf{1}$（遗忘门打开），那么梯度在细胞状态上的传递近似恒等映射，没有 $\tanh'$ 的饱和缩放，不会指数衰减。这就是”梯度高速公路”。

但 LSTM 并没有根治：f_t 仍然逐元素小于 1，长期连乘 $\prod_j f_j$ 仍可能缓慢衰减；而且某个维度的遗忘门若长期接近 0，那个维度的梯度仍然消失。更根本的是——LSTM 仍然是**串行**的：h_t 必须等 h_{t-1} 算完才能算，序列长度 T 决定了计算必须串行 T 步，并行度为 1。这在算力时代是不可接受的。

2.5 串行瓶颈：RNN vs 注意力

RNN 任意两个位置 i, j 之间的信息路径长度是 $|i-j|$（要沿时间一步步传）。Transformer 的自注意力里，任意两个位置直接连边，路径长度是 $O(1)$（ch16 会展开复杂度对比）。路径短意味着梯度不需要穿过长连乘，也意味着长距离依赖可以直接建模——这是 Transformer 取代 RNN 的两个根本原因。

3. 直觉理解

直觉：RNN 像传话游戏里站成一排的人——第一个人说的话传到第十个人时已经面目全非（梯度消失）。LSTM 给每个人发了一个笔记本，可以直接把上一个人的笔记原样传下去，不用经过”口头翻译”。但问题是——这一排人还是得一个接一个地传，没法同时开工。

Transformer 直接把所有人叫到一个房间里开会（注意力），谁需要谁直接对话，不用排队传话。训练时所有人同时发言，GPU 把整个矩阵一次算完。

4. 常见误区

误区一：以为 LSTM 彻底解决了梯度消失。它只是缓解——通过细胞状态的加法通路代替乘法通路。遗忘门接近 1 时梯度近似恒等，但连乘 $\prod f_t$ 仍可能缓慢衰减。

误区二：把 RNN 的失败只归因于梯度消失。即使梯度不消失，固定维度 $h_t \in \mathbb{R}^d$ 也是一个信息瓶颈——任意长的历史都要压进同一个 d 维向量，装不下。Transformer 的全局注意力让每个输出直接访问全部历史，绕开了这个瓶颈。

误区三：以为 LSTM 慢是因为层数多。慢的根源是**时间维串行**：第 t 步必须等第 $t-1$ 步，GPU 再大也只能一步一步算。

5. 代码实现

下面观察 RNN 和 LSTM 在不同序列长度下的梯度范数。工程注意：梯度范数要对所有参数求和，且每个模型实例要独立建、独立反向，否则梯度会累积；`sum().backward()` 对输出求和构造一个标量损失，方便观测。

```
import torch
import torch.nn as nn

# 直接观测" 长程梯度": 末端损失对第 0 步输入的梯度范数, 随序列长度 T 的变化
torch.manual_seed(0)

class VanillaRNN(nn.Module):
    def __init__(self, d=16, scale=0.9):
        super().__init__()
        self.d = d
        self.W_ih = nn.Linear(d, d)
        self.W_hh = nn.Linear(d, d)
        with torch.no_grad():
            self.W_hh.weight.mul_(scale)  # 循环权重谱半径 < 1, 制造梯度消失条件
    def forward(self, x):  # x: (T, d), 要求 x 保留梯度
        h = torch.zeros(self.d)
        for t in range(x.size(0)):
            h = torch.tanh(self.W_ih(x[t]) + self.W_hh(h))
        return h
```

```python
class LSTMWrap(nn.Module):
    def __init__(self, d=16):
        super().__init__()
        self.lstm = nn.LSTM(d, d, batch_first=False)
    def forward(self, x):
        out, _ = self.lstm(x.unsqueeze(1))
        return out[-1, 0]

# 关键观测对象：末端 h[T] 对 x[0] 的梯度范数。
# RNN 应随 T 增大明显缩小（每步乘 ~0.9，长程信号指数衰减）；
# LSTM 靠细胞加法通路衰减慢得多。
print("T  | RNN 末端对 x[0] 梯度范数 | LSTM 末端对 x[0] 梯度范数")
for T in [10, 20, 40, 80]:
    x = torch.randn(T, 16, requires_grad=True)
    rnn = VanillaRNN(scale=0.9)
    rnn(x).sum().backward()
    rnn_grad0 = x.grad[0].norm().item()

    x2 = torch.randn(T, 16, requires_grad=True)
    lstm = LSTMWrap()
    lstm(x2).sum().backward()
    lstm_grad0 = x2.grad[0].norm().item()

    print(f"{T:3d} | {rnn_grad0:22.6f} | {lstm_grad0:22.6f}")
```

6. 小结

1. RNN 通过隐藏状态递归压缩历史，BPTT 导致梯度沿时间维雅可比连乘。
2. 梯度消失根源是 $\tanh'$ 饱和性和 $\|W_{hh}\| < 1$ 的连乘效应 c^{t-k}。
3. LSTM 通过细胞状态加法通路 $c_t = f_t \odot c_{t-1} + \ldots$ 提供梯度高速公路，$\partial c_t / \partial c_{t-1} = f_t$，缓解但未根治。
4. RNN/LSTM 的致命伤是时间维串行无法并行，加上固定维度隐藏状态的信息瓶颈。

7. 延伸阅读/进一步思考

- 梯度消失的早期系统分析见 Hochreiter 1991 的论文和 Pascanu 等 2013 年《On the difficulty of training Recurrent Neural Networks》；LSTM 出自 Hochreiter & Schmidhuber 1997。
- 想理解”路径长度”和”长距离依赖”的关系，对照 He 等 2015 残差网络（ch12）：残差让前馈网络的有效路径也变成近似 $O(1)$。
- Transformer 的自注意力把串行 T 步变成一次 $O(n^2 d)$ 的矩阵乘法，这是它能并行训练的根本原因；ch16 会系统对比复杂度。

8. 思考题

基础题

1. 在 LSTM 中，如果遗忘门 $f_t = \mathbf{0}$（完全关闭），细胞状态 c_t 会变成什么？梯度还能沿细胞状态传过这一步吗？

进阶题

2. RNN 中位置 i 到位置 j 的最大路径长度是 $|i-j|$。对比 Transformer 自注意力中任意两位置路径长度为 $O(1)$。从梯度连乘 $c^{|i-j|}$ 的角度解释：为什么路径短对长距离依赖训练更友好？
3. 为什么双向 RNN 在训练时可以并行（双向同时算），但推理时仍然是串行的？

开放题

4. LSTM 的细胞状态是加法更新 $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$，这和残差连接 $x + F(x)$（ch12）在数学结构上有什么共通之处？为什么”加法通路”比”乘法通路”更利于梯度流动？

第 6 讲 Seq2Seq 与注意力机制的萌芽

本讲问题：把一句话压缩成一个固定维度向量再解码，信息瓶颈有多严重？Bahdanau 注意力是怎么打破这个瓶颈的？从”固定向量”到”动态回看”，这一步为什么是 Transformer 的思想源头？

1. 数学工具/预备知识

ch05 的 RNN/LSTM。 Encoder 和 Decoder 都是循环网络，状态沿时间步递推。

对齐（alignment）。在机器翻译里，生成目标语言第 s 个词时，应该重点参照源语言的哪个（哪些）词？这个对应关系就叫对齐。Bahdanau 之前对齐是隐式的（压在固定向量里），之后变成显式的概率分布。

前馈网络作为打分函数。一个小的 MLP 输入两个向量、输出一个标量分数，就是一个可学习的”相似度函数”。

softmax（ch04）。把一组分数变成和为 1 的权重。

2. 完整推导

2.1 Encoder-Decoder 的信息瓶颈

Seq2Seq 架构分两段。Encoder 把源序列 $x_{1:T}$ 编码成一个固定向量 c；Decoder 从 c 出发自回归生成目标序列 $y_{1:S}$：

$$h_t^{enc} = f_{enc}(h_{t-1}^{enc}, x_t), \quad c = h_T^{enc}$$

$$p(y_s \mid y_{<s}, c) = g_{dec}(h_s^{dec}, c), \quad h_s^{dec} = f_{dec}(h_{s-1}^{dec}, y_{s-1})$$

问题出在 c：无论源序列多长，全部信息都要塞进一个 $c \in \mathbb{R}^d$。当 T 很大时，c 容量不足，翻译质量随句子变长急剧下降。这就是**信息瓶颈**。打个比方：让人把一整篇文章读完后合上书背诵全文，文章越长背错越多。

2.2 Bahdanau 加性注意力

Bahdanau 的突破：不再用固定向量 c，而是让 Decoder 在每一步**动态地回看** Encoder 的全部隐藏状态，为每个目标位置生成一个专属上下文向量 c_s。

设 Encoder 用双向 RNN，隐藏状态为 $h_1, \dots, h_T$（每个 h_t 已包含左右文信息）。Decoder 在第 s 步的隐藏状态是 s_s。三步：

第一步：打分函数（加性）

$$e_{st} = v^\top \tanh(W_1 s_s + W_2 h_t)$$

其中 $W_1 \in \mathbb{R}^{d_a \times d_s}$，$W_2 \in \mathbb{R}^{d_a \times d_h}$，$v \in \mathbb{R}^{d_a}$。这是一个小型前馈网络：输入”当前 Decoder 状态 s_s “和”某个 Encoder 位置 h_t “，输出标量分数 e_{st}。为什么用 tanh 而不是直接点积？因为它能建模 s_s 和 h_t 之间更复杂的非线性匹配，代价是参数更多。

第二步：归一化

$$\alpha_{st} = \frac{\exp(e_{st})}{\sum_{t'=1}^{T} \exp(e_{st'})}$$

对固定的 s，$\alpha_{s\cdot}$ 是一个关于源位置 t 的概率分布（ch04 softmax）。

第三步：加权求和得到上下文向量

$$c_s = \sum_{t=1}^{T} \alpha_{st} h_t$$

这个 c_s 替代原来的固定 c，和 s_s 拼接后输入 Decoder 下一步。关键区别：c_s 随 s 变化——生成每个目标词时，注意力都重新分配一次。

2.3 Luong 乘性注意力

Luong 提出三种打分函数，比加性更省参数、更快：

1. **Dot product**：$e_{st} = s_s^\top h_t$（要求 $d_s = d_h$）。
2. **General**：$e_{st} = s_s^\top W h_t$，$W \in \mathbb{R}^{d_s \times d_h}$。
3. **Concat**：等价于 Bahdanau 加性注意力。

乘性注意力比加性快，因为点积可以写成一次矩阵乘法 $SH^\top$ 批量算出所有 (s,t) 对的分数，而加性要逐对过 MLP。这直接导向了 ch08 的 Scaled Dot-Product Attention。

2.4 注意力权重的概率解释

α_{st} 是一个关于 t 的概率分布（对固定 s）。它回答："Decoder 在生成第 s 个词时，应该把多少注意力放在源序列的位置 t?"这个软分配（soft alignment）和硬选择（hard selection，只挑一个位置）不同——所有位置都按权重参与，权重可以接近 one-hot 但永远是混合。

这个概率视角后来被抽象为 Query-Key-Value 框架（ch07）：s_s 是 query，h_t 既是 key 也是 value。

3. 直觉理解

直觉：传统 Seq2Seq 像让人把整本书背下来再默写——背不下来就丢内容。Bahdanau 注意力像让人看着原书一页一页地翻译：每翻到目标位置，眼睛扫一遍原文，把目光聚焦在最相关的那几行上。α_{st} 就是眼球的注视分布。

加性注意力像"两个人各说一句话，第三个人综合判断相关度"；乘性注意力像"直接把两句话的向量对齐算分"——后者省掉了中间人，更快，但要求两边在同一个向量空间里可比。

4. 常见误区

误区一：以为注意力是"新的网络层"。它本质是一个**带软检索的加权求和**，不是层、不是激活函数，而是一种计算范式。

误区二：以为 Bahdanau 注意力补的是"上下文"。它用双向 RNN，每个 h_t 本来就含左右文；注意力补的是"Decoder 每一步需要聚焦不同源位置"的能力。

误区三：以为 α_{st} 会均匀分布。实际训练好的模型，α_{st} 往往很尖锐——一个目标词只盯着一两个源词，这正是对齐关系。

5. 代码实现

下面实现一个迷你 Bahdanau 加性注意力。注意维度广播：`dec_proj` 是 `(1, d_a)`，`enc_proj` 是 `(T, d_a)`，要各自 unsqueeze 成 `(1,1,d_a)` 和 `(1,T,d_a)` 才能相加。`torch.tanh` 是函数式接口，不要用已废弃的 `F.tanh`。

```
import torch
import torch.nn.functional as F

# 迷你 Bahdanau 加性注意力
torch.manual_seed(0)

T, d_h, d_s, d_a = 5, 8, 6, 10  # 源长度 =5, encoder 维 =8, decoder 维 =6, 中间维 =10
```

```
# Encoder 隐藏状态 (batch=1, T, d_h)
encoder_out = torch.randn(1, T, d_h)
# Decoder 当前状态 (1, d_s)
decoder_state = torch.randn(1, d_s)

# 加性注意力参数
W1 = torch.randn(d_a, d_s)  # 作用于 decoder state
W2 = torch.randn(d_a, d_h)  # 作用于 encoder hidden
v = torch.randn(d_a)        # 最终打分向量

# 第一步：计算每个位置的分数 e_st
# decoder_state: (1, d_s) @ W1.T: (d_s, d_a) -> (1, d_a)
dec_proj = torch.tanh(decoder_state @ W1.T)          # (1, d_a)
# encoder_out[0]: (T, d_h) @ W2.T: (d_h, d_a) -> (T, d_a)
enc_proj = torch.tanh(encoder_out[0] @ W2.T)         # (T, d_a)

# 广播相加: dec_proj(1,1,d_a) + enc_proj(1,T,d_a) -> (1,T,d_a)
combined = torch.tanh(dec_proj.unsqueeze(1) + enc_proj.unsqueeze(0))  # (1, T, d_a)
scores = combined @ v                                 # (1, T)

# 第二步：softmax 归一化（沿 key 维 T）
alpha = F.softmax(scores, dim=-1)                     # (1, T)
print(" 注意力权重 alpha:", alpha.detach().round(decimals=4))
print(" 权重和:", alpha.sum().item())                 # 应为 1

# 第三步：加权求和得到上下文向量
# alpha.unsqueeze(1): (1,1,T) 乘 encoder_out: (1,T,d_h) -> (1,1,d_h)
context = torch.bmm(alpha.unsqueeze(1), encoder_out).squeeze(1)  # (1, d_h)
print(" 上下文向量 shape:", tuple(context.shape))
```

6. 小结

1. Encoder-Decoder 架构的固定向量 c 是信息瓶颈，长序列下性能崩塌。
2. Bahdanau 加性注意力：$e_{st} = v^\top \tanh(W_1 s_s + W_2 h_t)$，softmax 后加权求和得到动态上下文向量 c_s。
3. Luong 乘性注意力（Dot/General/Concat）是加性注意力的高效变体，可批量矩阵乘法，直接导向 SDPA。
4. 注意力权重 α_{st} 是概率分布，回答”当前步该关注源序列哪个位置”，是软检索而非硬选择。

7. 延伸阅读/进一步思考

- Bahdanau, Cho, Bengio 2014《Neural Machine Translation by Jointly Learning to Align and Translate》是注意力的开山之作;Luong 等 2015《Effective Approaches to Attention-based Neural Machine Translation》系统对比了三种打分函数。
- 注意力最初是为了解决”长句子的翻译瓶颈”，后来被证明可以脱离 RNN 独立使用（ch07 的自注意力），这是思想上的关键一跳。
- 软对齐 vs 硬对齐：Xu 等 2015 把注意力用于图像描述生成，提出了 hard/soft attention 的对比，是视觉 Transformer 的先声。

8. 思考题

基础题

1. 加性注意力对每个 (s, t) 对都要过一个小 MLP；乘性注意力直接矩阵乘。为什么乘性可以批量加速而加性不能?

进阶题

2. 如果 Encoder 用单向 RNN（而非双向），h_t 只含左侧信息。这对依赖”后文”的翻译会造成什么影响?
3. Bahdanau 注意力中 $\sum_t \alpha_{st} = 1$。若源序列有 1000 个词，均匀分布时每位置权重 0.001——softmax 后如果分数差异不大，权重会摊得很平，这对上下文向量 c_s 有什么坏处?

开放题

4. Bahdanau 注意力里 Decoder 状态是 query、Encoder 隐藏状态既是 key 又是 value。ch07 会把 key 和 value 拆成两个不同的投影。你觉得”key 和 value 共用同一个向量”在什么情况下是个限制?（提示：一个向量既要负责”好不好匹配”又要负责”匹配后输出什么”，这两个角色能解耦吗?）

第 7 讲 Query-Key-Value：注意力的一般化抽象

> **本讲问题**：Bahdanau 注意力里只有一个”查询”（Decoder 状态）和一堆”被查内容”（Encoder 隐藏状态）。怎么把它抽象成数据库检索的三要素 Q/K/V？为什么这个抽象是 Transformer 的灵魂?

1. 数学工具/预备知识

ch06 的 Bahdanau/Luong 注意力。已知打分 e_{st}、softmax 归一化、加权求和三步。

数据库检索直觉。检索系统三要素：query（你输入的搜索词）、key（每条文档的索引/标签）、value（文档正文）。匹配度由 query 和 key 决定，返回的是 value。

概率分布与期望。 softmax 输出 α_i 是概率分布，输出 $\sum_i \alpha_i v_i$ 是 value 的期望。

2. 完整推导

2.1 从数据库检索到数学抽象

数据库检索流程：输入 query，系统把每个文档的 key 和 query 匹配，匹配度高的排前面，返回这些文档的 value。翻译成数学：

- **Query** q：当前需要查询的信息。
- **Key** $k_1, \dots, k_n$：每个被检索对象的索引特征。
- **Value** $v_1, \dots, v_n$：每个被检索对象的实际内容。

匹配函数 $s(q, k_i)$ 衡量 query 和第 i 个 key 的相似度，经过 softmax 变成权重 α_i，输出是 value 的加权期望:

$$\text{Attn}(q, \mathcal{K}, \mathcal{V}) = \sum_{i=1}^{n} \alpha_i v_i, \quad \alpha_i = \frac{\exp(s(q, k_i))}{\sum_j \exp(s(q, k_j))}$$

这就是注意力的最一般形式。ch06 的 Bahdanau 是它的一个特例：$q = s_s$，$k_i = v_i = h_i$，s 是加性 MLP。

2.2 为什么需要三套不同的投影

在 Bahdanau 中，Decoder 状态直接当 query，Encoder 隐藏状态直接当 key 和 value。但这有个问题：query、key、value 若都来自同一个未变换的表征空间，表达能力受限——因为”该怎么匹配”和”匹配后输出什么”这两件事被绑死在同一个向量里。

Transformer 引入三个可学习的投影矩阵，把同一输入分别投到三个子空间:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

- Self-Attention：Q, K, V 都来自同一个输入序列 X。
- Cross-Attention：Q 来自 Decoder，K, V 来自 Encoder。

这个设计的好处:

- Q/K 投影学”该怎么匹配”，V 投影学”匹配后提取什么内容”，三者解耦。

- 同一个 token 向量，在不同投影下扮演不同角色（它既是 query 又是 key 又是 value）。
- 维度可不同：Q/K 维度 d_k，V 维度 d_v（实际 Transformer 中通常 $d_k = d_v = d_{\text{model}}/h$，见 ch09）。

2.3 Self-Attention vs Cross-Attention

Self-Attention：Q、K、V 来自同一序列。输入 $X \in \mathbb{R}^{n\times d}$，则 $Q, K, V \in \mathbb{R}^{n\times d_k}$（投影后）。每个位置可以直接关注序列中任何其他位置——不管距离多远，路径长度都是 1。这直接解决了 ch05 RNN 的长距离路径问题。

Cross-Attention：Q 来自 Decoder 上一层输出，K/V 来自 Encoder 最终输出。这对应 Bahdanau 的原始设定：Decoder 查询 Encoder 的记忆。

2.4 注意力权重的概率解释

α_i 是概率分布：非负、和为 1。注意力输出 $\sum_i \alpha_i v_i$ 就是 value 的期望。这个视角让我们能用概率工具分析注意力：

- 权重熵高 → 分布均匀 → “广泛关注”，对所有 key 一视同仁。
- 权重熵低 → 分布尖锐 → “聚焦于少数几个位置”。

训练好的语言模型里，不同位置、不同层的注意力熵差异很大，对应不同的注意力模式。

2.5 通用 attention 函数的形式

把打分函数 s 一般化：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{s(Q, K)}{\sqrt{d_k}}\right) V$$

不同的 s 对应不同变体：

- $s(q, k) = q^\top k$ → 缩放点积注意力（ch08）。
- $s(q, k) = v^\top \tanh(W_1 q + W_2 k)$ → 加性注意力（ch06）。

除以 $\sqrt{d_k}$ 的原因 ch08 专门推导，这里先记住它是为了防止打分过大导致 softmax 饱和。

3. 直觉理解

直觉：Q/K/V 就像图书馆检索——Q 是你要找的书的主题，K 是每本书的书名标签，V 是书的内容。你先把每个书名和你的主题做匹配（打分），匹配度高的书你多看几页（权重高），最后把所有书的内容按阅读比例混合成一份笔记输出。

Self-Attention 就是图书馆里每本书都在同时检索其他所有书——“我是一本讲猫的书，我想和讲动物学、讲宠物、讲生物学的书都聊聊”。Cross-Attention 是你拿着自己的笔记（Q）去图书馆书架（K/V）里找参考资料。

4. 常见误区

误区一：以为 Q/K/V 是三个不同的输入。它们来自同一个输入 X，只是通过三个不同投影矩阵变换到了三个不同子空间。

误区二：以为注意力”选择”了最相关的 value。它不是 hard selection（选一个），而是 soft mixture（所有 value 都参与，按权重混合）。权重可以接近 one-hot，但数学上始终是加权平均。

误区三：以为投影矩阵 W_Q, W_K, W_V 是多余的。没有它们，query 和 key 被锁死在原始 token 空间，模型学不到”该用什么标准衡量匹配”。

5. 代码实现

下面实现一个通用注意力函数，支持 dot 和 additive 两种打分。工程注意：additive 分支里的投影向量应当是传入的参数而不是函数内随机生成（否则每次调用都不一样，无法训练）；`masked_fill` 把被屏蔽位置填成大负数，配合 ch04 的 softmax 产生近零权重。

```
import torch
import torch.nn.functional as F

def general_attention(Q, K, V, score='dot', scale=None, mask=None):
    """
    Q: (..., n_q, d_k)
    K: (..., n_k, d_k)
    V: (..., n_k, d_v)
    score: 'dot' 或 'additive'
    """
    if score == 'dot':
        scores = torch.matmul(Q, K.transpose(-2, -1))  # (..., n_q, n_k)
        if scale is not None:
            scores = scores / scale
    elif score == 'additive':
        # 简化版: Q.unsqueeze(-2) + K.unsqueeze(-3) 后过 tanh，再和可学向量 w 做点积
        d_k = Q.size(-1)
        w = torch.randn(d_k, device=Q.device) * 0.1  # 演示用，实际应作为参数传入
        combined = torch.tanh(Q.unsqueeze(-2) + K.unsqueeze(-3))  # (..., n_q, n_k, d_k)
        scores = combined @ w  # (..., n_q, n_k)

    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    alpha = F.softmax(scores, dim=-1)  # (..., n_q, n_k)
    output = torch.matmul(alpha, V)    # (..., n_q, d_v)
    return output, alpha

# 测试 Self-Attention
torch.manual_seed(0)
X = torch.randn(1, 4, 8)  # batch=1, seq=4, dim=8
W_Q = torch.randn(8, 8) * 0.1
W_K = torch.randn(8, 8) * 0.1
W_V = torch.randn(8, 8) * 0.1

Q = X @ W_Q
K = X @ W_K
V = X @ W_V

out, alpha = general_attention(Q, K, V, score='dot')
print("Self-Attention 输出 shape:", tuple(out.shape))
print(" 注意力权重矩阵（每行和为 1）:")
print(alpha[0].detach().round(decimals=3))
print(" 每行权重和:", alpha[0].sum(dim=-1).detach().round(decimals=4).tolist())
```

6. 小结

1. Q/K/V 抽象来自数据库检索：Q 是查询，K 是索引，V 是内容；通用公式 softmax$(s(Q,K))V$。
2. 三个投影矩阵把”怎么匹配”（Q/K）和”提取什么”（V）解耦，三者来自同一输入但处于不同子空间。
3. Self-Attention 中 Q/K/V 来自同一序列，任意两位置路径长度为 1；Cross-Attention 中 Q 来自 Decoder、K/V 来自 Encoder。
4. 注意力输出是 value 的加权期望，权重是概率分布——软检索而非硬选择。

7. 延伸阅读/进一步思考

- Q/K/V 三要素的明确区分见 Vaswani 等 2017《Attention Is All You Need》（ch08/ch09 的主线）；在那之前 Bahdanau 版本里 key 和 value 是同一个向量。
- 想理解”为什么把 key 和 value 拆开是重要一步”，可对比：Dense Retrieval / 双塔模型里 query 塔和 doc 塔也是分开投影的，思想同源。
- 注意力权重的可解释性研究（“attention is not explanation”之争）提醒我们：高注意力权重不等于因果重要性，ch29 可视化时会再讨论。

8. 思考题

基础题

1. 如果把 Q 和 K 投影到同一个空间（即 $W_Q = W_K$），从”匹配标准”角度看会损失什么?

进阶题

2. 注意力权重的熵越高，说明模型在做什么？熵越低呢？分别对应什么语义场景?
3. 为什么 Cross-Attention 中 Q 来自 Decoder 而 K/V 来自 Encoder? 反过来（Q 来自 Encoder、K/V 来自 Decoder）对应什么任务?

开放题

4. 通用注意力公式里打分函数 s 可以替换成任意可学习函数。除了点积和加性，你能想到第三种打分函数吗?（提示：考虑余弦相似度、核方法、或把 key 拆成多块。）这种替换会如何影响 ch08 的缩放分析?

第 8 讲 Scaled Dot-Product Attention：完整推导

> **本讲问题**：为什么点积注意力要除以 $\sqrt{d_k}$? 不除会发生什么? 这一个小小的除法，是 Transformer 训练稳定的关键工程细节。本讲从方差出发把它推到底。

1. 数学工具/预备知识

ch04 的 Softmax 饱和区。单独 Softmax 的雅可比 $J_{ij} = p_i(\delta_{ij} - p_j)$，当某个 $p_i \to 1$、其余 $\to 0$ 时，整个雅可比趋近 0，梯度消失。

期望与方差。对随机变量 X,$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$。独立变量之和的方差等于方差之和:$\mathrm{Var}(\sum_l X_l) = \sum_l \mathrm{Var}(X_l)$。

ch07 的 Q/K/V 抽象。 Q, K, V 的定义和打分 $S_{ij} = q_i^\top k_j$。

2. 完整推导

2.1 维度标注

设 $Q \in \mathbb{R}^{n \times d_k}$，$K \in \mathbb{R}^{m \times d_k}$，$V \in \mathbb{R}^{m \times d_v}$。缩放点积注意力（SDPA）定义为:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

打分矩阵 $S = QK^\top \in \mathbb{R}^{n \times m}$，每个元素 $S_{ij} = q_i^\top k_j = \sum_{l=1}^{d_k} q_{il} k_{jl}$。

2.2 为什么除以 $\sqrt{d_k}$：方差推导

假设 q_i 和 k_j 的各分量是独立同分布的随机变量，均值为 0、方差为 1（这是 Xavier/He 初始化附近的典型假设，ch20 会讲为什么这样初始化）。计算 $S_{ij} = \sum_{l=1}^{d_k} q_{il} k_{jl}$ 的期望和方差。

期望:

$$\mathbb{E}[S_{ij}] = \sum_{l=1}^{d_k} \mathbb{E}[q_{il}k_{jl}] = \sum_{l=1}^{d_k} \mathbb{E}[q_{il}]\mathbb{E}[k_{jl}] = 0$$

第二步用了独立性（$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$）和零均值。

方差：因为 $\mathbb{E}[S_{ij}] = 0$，$\text{Var}(S_{ij}) = \mathbb{E}[S_{ij}^2]$。展开：

$$\text{Var}(S_{ij}) = \sum_{l=1}^{d_k} \text{Var}(q_{il}k_{jl})$$

跨 l 独立，方差可加。对单个乘积 $q_{il}k_{jl}$，由独立性和零均值：

$$\text{Var}(q_{il}k_{jl}) = \mathbb{E}[q_{il}^2 k_{jl}^2] = \mathbb{E}[q_{il}^2]\mathbb{E}[k_{jl}^2] = 1 \cdot 1 = 1$$

（这里用了 $\mathbb{E}[q^2] = \text{Var}(q) + (\mathbb{E}q)^2 = 1 + 0 = 1$。）所以：

$$\text{Var}(S_{ij}) = \sum_{l=1}^{d_k} 1 = d_k$$

S_{ij} 的标准差是 $\sqrt{d_k}$。当 $d_k = 64$ 时，打分典型量级是 $\sqrt{64} = 8$；$d_k = 512$ 时是 $\sqrt{512} \approx 22.6$。维度越高，打分的”音量”越大。

2.3 Softmax 饱和区的梯度消失

把这些很大的 logits 喂给 Softmax。假设 S_{ij} 量级是 $O(\sqrt{d_k})$，那么 n 个分数中最大的那个会比其他值大很多（差值是 $O(\sqrt{d_k})$）。回忆 ch04：Softmax 输入差越大，输出越接近 one-hot。比如 $d_k = 512$ 时分数标准差 22，最大分数比平均高两三个标准差，softmax 后那个位置权重几乎就是 1。

一旦 Softmax 输出接近 one-hot，回忆 ch04 雅可比 $J_{ij} = p_i(\delta_{ij} - p_j) \to 0$——梯度几乎为零，模型学不动。更糟的是，注意力权重接近 one-hot 意味着加权求和退化成”只取一个 value”，模型的软检索能力退化。

除以 $\sqrt{d_k}$ 后：

$$\text{Var}\left(\frac{S_{ij}}{\sqrt{d_k}}\right) = \frac{1}{d_k}\text{Var}(S_{ij}) = \frac{d_k}{d_k} = 1$$

打分的标准差回到 1，分数之间的典型差值在 $O(1)$ 量级，Softmax 工作在非饱和区，权重分布合理、梯度正常流动。

2.4 与加性注意力的效率对比

加性注意力（ch06）用小 MLP 打分：$e_{ij} = v^\top \tanh(W_1 q_i + W_2 k_j)$。

- 加性：每个 (i, j) 对都要过一个 d_a 维的小网络，每对 $O(d_a d_k)$。
- 点积：一次矩阵乘法 $QK^\top$，全部 $n \times m$ 对一共 $O(nmd_k)$，而且矩阵乘法在 GPU 上高度优化。

当 d_k 不大（原论文 $d_k = 64$）时，点积注意力快得多，这是原论文选择点积的主因。加性注意力在小 d_k 下没有速度优势，又多参数，被淘汰。

2.5 直觉核对

把 2.2 的推导和 ch04 串起来：不缩放 → 点积方差 d_k → logits 过大 → softmax 饱和 → 雅可比趋零 → 梯度消失。缩放 $\sqrt{d_k}$ → 方差 1 → softmax 在线性区。这就是为什么缩放因子恰好是 $\sqrt{d_k}$ 而不是 d_k 或别的——它正好把方差拉回 1。

3. 直觉理解

> **直觉**：维度越高，点积的”数值音量”越大——就像一千个人同时说话，噪音叠加起来震耳欲聋。除以 $\sqrt{d_k}$ 就是给这个嘈杂的打分做音量归一化，让 Softmax 这个”耳朵”能正常分辨谁在说什么，而不是被最大的声音震聋（饱和成 one-hot，梯度消失）。

4. 常见误区

> **误区**一：以为除以 $\sqrt{d_k}$ 是为了”数值稳定防上溢”。数值稳定是靠减最大值实现的（ch04），除以 $\sqrt{d_k}$ 的目的是**控制 Softmax 输入分布、避免饱和导致梯度消失**。
>
> **误区**二：以为这个缩放是经验调出来的。它有严格的方差推导支撑——前提是 Q/K 分量零均值单位方差。
>
> **误区**三：以为缩放因子该用 d_k 而不是 $\sqrt{d_k}$。d_k 是方差，$\sqrt{d_k}$ 才是标准差；要把标准差拉回 1，除的必须是标准差。

5. 代码实现

下面从零实现 SDPA 并对照 PyTorch 官方 `F.scaled_dot_product_attention`。工程注意：官方实现会自动处理 mask 和数值稳定，手写版要自己 `masked_fill`；验证饱和现象时，让 d_k 很大就能看到原始点积分数量级是 $\sqrt{d_k}$。

```
import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q: (batch, n_heads, n_q, d_k)
    K: (batch, n_heads, n_k, d_k)
    V: (batch, n_heads, n_k, d_v)
    """
    d_k = Q.size(-1)
    # 第一步：点积打分 (b, h, n_q, n_k)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    # 第二步：缩放，把方差从 d_k 拉回 1
    scores = scores / (d_k ** 0.5)
    # 第三步：mask（如果有），把禁止位置填大负数
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    # 第四步：softmax 归一化
    alpha = F.softmax(scores, dim=-1)
    # 第五步：加权求和
    output = torch.matmul(alpha, V)
    return output, alpha

# 对照官方实现
torch.manual_seed(0)
b, h, n, d_k = 2, 4, 10, 64
Q = torch.randn(b, h, n, d_k)
K = torch.randn(b, h, n, d_k)
V = torch.randn(b, h, n, d_k)
```

```
out_mine, alpha_mine = scaled_dot_product_attention(Q, K, V)
out_official = F.scaled_dot_product_attention(Q, K, V)

print(" 手动 SDPA 输出 shape:", tuple(out_mine.shape))
print(" 与官方实现差异:", torch.allclose(out_mine, out_official, atol=1e-5))

# 验证：不除以 sqrt(d_k) 会导致 softmax 饱和（权重接近 one-hot）
d_k_large = 512
torch.manual_seed(0)
Q2 = torch.randn(1, 1, 1, d_k_large)
K2 = torch.randn(1, 1, d_k_large, d_k_large)  # 1 个 query，很多 key
raw = (Q2 @ K2.transpose(-2, -1)).squeeze()
scaled = raw / (d_k_large ** 0.5)
p_raw = F.softmax(raw, dim=-1)
p_scaled = F.softmax(scaled, dim=-1)
print(f"\nd_k={d_k_large} 时:")
print(f"  原始点积分数 std: {raw.std().item():.2f} (应 ≈sqrt({d_k_large})={d_k_large**0.5:.1f})")
print(f"  缩放后分数 std:   {scaled.std().item():.2f} (应 ≈1)")
print(f"  不缩放 softmax 最大权重: {p_raw.max().item():.4f} (接近 1= 饱和)")
print(f"  缩放后 softmax 最大权重: {p_scaled.max().item():.4f} (更分散)")
```

6. 小结

1. SDPA 公式：$\text{softmax}(QK^\top/\sqrt{d_k})V$。
2. 除以 $\sqrt{d_k}$ 的原因：Q/K 分量独立零均值单位方差时，点积方差为 d_k，不缩放会导致打分过大、Softmax 饱和、梯度消失。
3. 缩放后方差为 1，Softmax 工作在线性区。
4. 点积注意力比加性注意力计算高效，因为可以用高度优化的矩阵乘法。

7. 延伸阅读/进一步思考

- 缩放因子的原始论证见 Vaswani 等 2017《Attention Is All You Need》第 3.2.2 节；注意论文里也只是定性猜测了”大维度下点积方差变大”，本讲 2.2 是严格化。
- 现代变体（如 GQA、MQA）改变的是 K/V 的头数，不改变 $1/\sqrt{d_k}$ 这个缩放。
- 若 Q/K 经过 LayerNorm（ch12）后再接注意力，点积方差会被 LN 重新归一，缩放的必要性会变化——这是某些架构选择的讨论点。

8. 思考题

基础题

1. 如果 Q 和 K 的分量方差不是 1 而是 σ^2，推导点积 S_{ij} 的方差，并给出正确的缩放因子。

进阶题

2. 当 $d_k = 1$ 时缩放因子是 1。此时点积注意力 $\text{softmax}(qk^\top)v$ 等价于什么操作?
3. 为什么不直接对 Q/K 做 LayerNorm 来控制方差，而要在打分后除以 $\sqrt{d_k}$?（提示：考虑训练速度和参数量。）

开放题

4. 实验发现：即使除以 $\sqrt{d_k}$，当 d_k 继续增大（比如到 4096），训练仍会不稳。这说明”方差归一化到 1”可能还不够——从 softmax 的熵和梯度量级角度，你觉得还缺什么?（提示：分数 std=1 时，n 个分数的最大值期望大约是 $\sqrt{2\log n}$，它随 key 数量 n 增长，这又会造成新的饱和。）

第 9 讲 Multi-Head Attention：多头的本质与维度分析

本讲问题：为什么 Transformer 要把 d_{model} 切成好几份分别做注意力、再拼回来？一个大头和八个小头，表达能力有什么本质区别？本讲用参数量和计算量两笔账说清这件事。

1. 数学工具/预备知识

ch08 的 SDPA。 $\text{softmax}(QK^\top/\sqrt{d_k})V$。

参数量与计算量估算。一个 $\mathbb{R}^{m\times k}$ 矩阵有 mk 个参数；矩阵乘 $\mathbb{R}^{m\times k}\mathbb{R}^{k\times n}$ 约需 $2mnk$ 次浮点运算（乘加各算一次，ch15 展开）。

子空间投影。一个 d 维向量被矩阵 $W\in\mathbb{R}^{r\times d}$ 投到 r 维子空间，可以理解成”换一组基看它”。不同的 W 对应不同的观察角度。

2. 完整推导

2.1 单头注意力的局限

单头注意力 $\text{softmax}(QK^\top/\sqrt{d_k})V$ 中，每个位置只有一个权重分布。但实际句子里，一个词可能同时需要关注多种关系：“苹果”既要关注主语（谁吃了苹果）、又要关注宾语（苹果被怎么了）、还要关注修饰语（红苹果还是青苹果）。一个权重分布无法同时建模多种关系——softmax 是一个单一的概率分布，没法同时给出”对主语的关注”和”对修饰语的关注”两套独立权重。

2.2 MHA 完整公式

Multi-Head Attention 把 d_{model} 维特征切成 h 个头，每个头维度 $d_k=d_{\text{model}}/h$。每个头独立做 SDPA:

$$\text{head}_i=\text{Attention}(QW_Q^{(i)},\,KW_K^{(i)},\,VW_V^{(i)})$$

其中 $W_Q^{(i)}\in\mathbb{R}^{d_{\text{model}}\times d_k}$，$W_K^{(i)}\in\mathbb{R}^{d_{\text{model}}\times d_k}$，$W_V^{(i)}\in\mathbb{R}^{d_{\text{model}}\times d_v}$。

拼接所有头再过一个输出投影:

$$\text{MultiHead}(Q,K,V)=\text{Concat}(\text{head}_1,\dots,\text{head}_h)W_O$$

其中 $W_O\in\mathbb{R}^{hd_v\times d_{\text{model}}}$。原论文取 $h=8$，$d_k=d_v=d_{\text{model}}/h=512/8=64$。

工程上，$W_Q^{(i)}$ 这 h 个矩阵被合并成一个大矩阵 $W_Q\in\mathbb{R}^{d_{\text{model}}\times d_{\text{model}}}$，一次投影后再 reshape 切头，避免 h 次小矩阵乘。

2.3 参数量分析

设 $d_{\text{model}}=d$，h 个头，每头 $d_k=d/h$。

单头（直接用 d 维做 Q/K/V）： - $W_Q:d\times d$ - $W_K:d\times d$ - $W_V:d\times d$ - W_O 不需要（输出已经是 d 维）- 合计 $3d^2$

多头（h 头，每头 $d_k=d/h$）： - $W_Q:h\times d\times(d/h)=d^2$（合并成一个 $d\times d$ 矩阵）- $W_K:d^2$ - $W_V:d^2$ - $W_O:d\times d$（拼接后 $h\cdot d_k=d$ 维，再投回 d 维）- 合计 $4d^2$

多头比单头只多了一个 W_O（d^2 参数）。也就是说，**多头几乎不增加参数量**，但得到了 h 套独立的注意力权重分布。

2.4 计算量分析

对序列长度 n。注意力 FLOPs 主要在两处：$QK^\top$ 和 $\text{softmax}\cdot V$。

- 单头：$QK^\top$ 是 $n\times n\times d$，约 $2n^2d$；乘 V 又是 $2n^2d$；合计 $4n^2d$（投影 nd^2 另算）。

- 多头：每个头维度 $d_k = d/h$，每个头的注意力是 $4n^2 d_k$；h 个头合计 $h \cdot 4n^2(d/h) = 4n^2 d$。

注意力的 FLOPs 和单头完全一样——切头只是在不同子空间里做同样的事，总量被 h 和 $1/h$ 约掉了。投影部分（$Q/K/V/O$ 四个 $d \times d$ 矩阵乘 n 维序列）也是 $O(nd^2)$，和单头对比只是多了 W_O。

2.5 多头的本质

多头不是”做更多次注意力”，而是”在 h 个不同子空间里并行做注意力”。每个头的 $W_Q^{(i)}$ 把输入投到一个不同子空间，在那个子空间里算相似度。不同子空间对应不同类型的关系：有的头关注语法依赖，有的头关注语义指代，有的头关注局部相邻位置。W_O 负责把这些不同视角的结果融合回统一的 d 维表征。

3. 直觉理解

直觉：单头注意力像一只眼睛看世界——只能聚焦一个焦点。多头像八只眼睛，每只戴不同颜色的滤镜：一只看语法关系，一只看语义关系，一只看位置邻近。最后把八只眼睛看到的画面拼起来、再由 W_O 融合，比一只眼睛看得全面。

关键是：八只眼睛的总”像素数”和一只眼睛一样——只是分通道处理了。所以多头几乎不花额外算力，买到的是视角的多样性。

4. 常见误区

误区一：以为多头是”并行做更多次注意力所以更强”。多头总计算量和单头相同，优势在于**子空间多样性**。

误区二：以为头数越多越好。头数太多时每头维度太小（$d = 512, h = 64$ 则每头 8 维），单头表达能力不足，性能反而下降。

误区三：以为 W_O 可有可无。没有它，八个头的输出只是粗暴拼接，头与头之间没有信息交换；W_O 负责让各视角融合成统一表征。

5. 代码实现

下面手写 MHA。工程关键：`view(b, s, h, d_k).transpose(1, 2)` 切头后，合并头时要先 `transpose(1,2)` 再 `contiguous().view(...)`——`contiguous()` 是因为 transpose 后的张量不连续，直接 view 会报错。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, n_heads=8):
        super().__init__()
        assert d_model % n_heads == 0, "d_model 必须能被 n_heads 整除"
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads

        # 四个 d_model x d_model 矩阵：Q/K/V 投影 + 输出投影
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

    def split_heads(self, x):
        # x: (batch, seq, d_model) -> (batch, n_heads, seq, d_k)
        b, s, _ = x.shape
        return x.view(b, s, self.n_heads, self.d_k).transpose(1, 2)
```

```
    def forward(self, Q, K, V, mask=None):
        # 线性投影后切头
        q = self.split_heads(self.W_Q(Q))   # (b, h, n_q, d_k)
        k = self.split_heads(self.W_K(K))   # (b, h, n_k, d_k)
        v = self.split_heads(self.W_V(V))   # (b, h, n_k, d_k)

        # SDPA
        scores = torch.matmul(q, k.transpose(-2, -1)) / (self.d_k ** 0.5)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        alpha = F.softmax(scores, dim=-1)
        out = torch.matmul(alpha, v)        # (b, h, n_q, d_k)

        # 合并头: transpose 回 (b, n_q, h, d_k) 再 view 成 (b, n_q, d_model)
        b, h, n_q, _ = out.shape
        out = out.transpose(1, 2).contiguous().view(b, n_q, self.d_model)
        return self.W_O(out)

# 测试
mha = MultiHeadAttention(d_model=512, n_heads=8)
x = torch.randn(2, 10, 512)  # batch=2, seq=10
out = mha(x, x, x)
print(" 输入 shape:", tuple(x.shape))
print(" 输出 shape:", tuple(out.shape))
print(" 参数量:", sum(p.numel() for p in mha.parameters()))
# 四个 512x512 矩阵 + 四个 bias(512) ≈ 4*262144 + 4*512
```

6. 小结

1. MHA 把 d_{model} 切成 h 个子空间，每个独立做 SDPA，最后拼接并线性投影。
2. 多头总参数量（$4d^2$）和计算量与单头（$3d^2$）基本同阶，优势在于子空间多样性。
3. 不同头可学到不同注意力模式（语法、语义、位置等）。
4. 头数选择受限于 $d_k = d_{\text{model}}/h$ 不能太小；W_O 负责融合各头。

7. 延伸阅读/进一步思考

- 原论文 Vaswani 等 2017 是多头的出处；后续研究（如 Voita 等 2019）分析发现很多头其实是冗余的，可以剪枝。
- 现代大模型为了省 KV cache（ch23）发展出 MQA（所有头共享同一组 K/V）和 GQA（分组共享），是对”多头”的进一步工程压缩。
- 想理解”子空间多样性”的实证，可看注意力可视化研究（ch29）：不同头确实倾向于捕捉不同的语言现象。

8. 思考题

基础题

1. $d_{\text{model}} = 512$、$h = 8$ 时每头维度是多少？$h = 16$ 呢？为什么 h 不能取到 512？

进阶题

2. 为什么 MHA 中需要额外的输出投影 W_O？没有它，拼接八个头的输出会缺什么能力？
3. 多头注意力和”把输入复制 h 份分别过 h 个独立的单头注意力、每份用不同的 d_k “有什么区别？从参数共享角度分析。

开放题

4. MQA/GQA 把 h 个头的 K/V 共享成 1 组（或少数几组），Q 仍然多头。从参数量和 KV cache 大小角度分析：这为什么能显著加速推理？（预告 ch23。）它牺牲了什么？

第 10 讲 Transformer 总体架构：Encoder-Decoder 数据流全览

> **本讲问题**：把前面九讲的零件组装起来，一个完整的 Transformer 长什么样？数据从 token 到 logits 的每一步维度变化是什么？训练和推理的数据流有什么本质区别？

1. 数学工具/预备知识

ch09 的 Multi-Head Attention。 Self-Attention 和 Cross-Attention 的区别只在 Q/K/V 从哪来。

残差连接与 LayerNorm（ch12 详解）。每个子层后都跟一个 `x + Sublayer(x)` 再加 LayerNorm，保证深层网络梯度能流通。

Teacher Forcing。训练 Decoder 时，输入不是模型自己上一步的预测，而是真实的上一步 token——这样所有位置可以并行算损失。

2. 完整推导

2.1 原论文架构

Transformer 由 $N = 6$ 个 Encoder 层和 $N = 6$ 个 Decoder 层堆叠而成。

Encoder 每层两个子层： 1. Multi-Head Self-Attention（残差 + LayerNorm）2. Position-wise Feed-Forward Network（残差 + LayerNorm）

Decoder 每层三个子层： 1. Masked Multi-Head Self-Attention（残差 + LayerNorm，禁止看到未来位置，ch14）2. Encoder-Decoder Cross-Attention（Q 来自 Decoder，K/V 来自 Encoder）3. Position-wise Feed-Forward Network（残差 + LayerNorm）

2.2 逐子层维度账本

设 $d_{\text{model}} = 512$，$d_{ff} = 2048$，$n_{\text{heads}} = 8$，$d_k = 64$。

输入嵌入：token ID 查表。输入 $X_{\text{id}} \in \mathbb{R}^{n\times 1}$（整数 token IDs），乘嵌入矩阵 $E \in \mathbb{R}^{V\times d_{\text{model}}}$ 变成 $X \in \mathbb{R}^{n\times 512}$。

加位置编码：$X = X_{\text{embed}} + P_{\text{pos}}$（ch11），形状不变。

Encoder 第 l 层： - Self-Attention：$n \times 512 \to n \times 512$ - Add & Norm：$n \times 512$ - FFN：$n \times 512 \to n \times 2048 \to n \times 512$ - Add & Norm：$n \times 512$

6 层后 Encoder 输出 $Z \in \mathbb{R}^{n\times 512}$。

Decoder 第 l 层： - Masked Self-Attention：$m \times 512 \to m \times 512$ - Add & Norm：$m \times 512$ - Cross-Attention：Q 是 $m \times 512$，K/V 来自 Encoder 输出 $n \times 512$，输出 $m \times 512$ - Add & Norm：$m \times 512$ - FFN：$m \times 512 \to m \times 2048 \to m \times 512$ - Add & Norm：$m \times 512$

6 层后 Decoder 输出 $D \in \mathbb{R}^{m\times 512}$。

输出投影：$D \in \mathbb{R}^{m\times 512}$ 乘 $W_{\text{out}} \in \mathbb{R}^{512\times V}$ 得到 logits $\in \mathbb{R}^{m\times V}$，逐位置 softmax 得到下一个 token 概率。

2.3 训练：Teacher Forcing

训练时 Decoder 输入是**完整目标序列右移一位**：`<bos> y1 y2 ... y_{m-1}`。Self-Attention 上加 Causal Mask（ch14），保证位置 t 只能看到 $\leq t$。

损失：对每个位置 t，预测 y_t，用交叉熵（ch02）。

Teacher Forcing 的含义：即使模型在位置 t 预测错了，位置 $t+1$ 的输入仍然是**真实的** y_t，而不是模型自己预测的 $\hat{y}_t$。这让训练可以并行（所有位置同时算损失），代价是 train-test 不一致（推理时模型看到的是自己的错误预测，ch29 讨论 exposure bias）。

2.4 推理：自回归

推理时没有完整目标序列，必须一步一步生成：1. 输入 <bos>，得到第一个 token 概率分布，采样/贪心得 $\hat{y}_1$。2. 把 $\hat{y}_1$ 接在输入后面，得到第二个 token。3. 重复直到 <eos> 或达到最大长度。

每一步都要重新过所有层的注意力——这就是 KV Cache（ch23）要解决的问题。

3. 直觉理解

直觉：Encoder 像一个读完原文后做笔记的人——把整句话压成一组向量表示。Decoder 像边看笔记边翻译的人——每翻一个词，先回顾自己已经翻了什么（Self-Attention），再看原文笔记（Cross-Attention），然后输出下一个词。训练时是看着答案翻译（Teacher Forcing 并行），推理时是自己边想边翻（自回归串行）。

4. 常见误区

误区一：以为 Decoder 的 Cross-Attention 也是 Self-Attention。Cross-Attention 中 Q 来自 Decoder，K/V 来自 Encoder——两个不同来源。

误区二：以为 Encoder 和 Decoder 共享权重。原论文中它们独立；现代 GPT 系列只有 Decoder 没有 Encoder。

误区三：以为位置编码加在注意力打分上。它加在输入嵌入上，只在第一层进入网络。

5. 代码实现

下面搭一个最小 Transformer。工程注意：`nn.MultiheadAttention` 的 cross-attention 里 `query` 是 Decoder 状态、`key/value` 是 Encoder 输出；这里为清晰起见没接 causal mask（ch14 专门讲），只验证维度。

```
import torch
import torch.nn as nn

class TransformerBlock(nn.Module):
    """ 单层 Encoder 或 Decoder（简化版，Post-LN）"""
    def __init__(self, d_model=512, n_heads=8, d_ff=2048, is_decoder=False):
        super().__init__()
        self.is_decoder = is_decoder
        self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.cross_attn = (nn.MultiheadAttention(d_model, n_heads, batch_first=True)
                           if is_decoder else None)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(), nn.Linear(d_ff, d_model)
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model) if is_decoder else None

    def forward(self, x, enc_out=None):
        # Self-Attention + 残差 + Norm
        attn_out, _ = self.attn(x, x, x)
        x = self.norm1(x + attn_out)

        if self.is_decoder and enc_out is not None:
            # Cross-Attention: Q=decoder, K/V=encoder
            cross_out, _ = self.cross_attn(x, enc_out, enc_out)
            x = self.norm2(x + cross_out)
            x = self.norm3(x + self.ffn(x))
        else:
            x = self.norm2(x + self.ffn(x))
```

```
        return x

class MiniTransformer(nn.Module):
    def __init__(self, vocab_size=1000, d_model=512, n_heads=8, n_layers=6):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.encoder_layers = nn.ModuleList(
            [TransformerBlock(d_model, n_heads, is_decoder=False) for _ in range(n_layers)])
        self.decoder_layers = nn.ModuleList(
            [TransformerBlock(d_model, n_heads, is_decoder=True) for _ in range(n_layers)])
        self.head = nn.Linear(d_model, vocab_size)

    def forward(self, src, tgt):
        enc = self.embed(src)
        for layer in self.encoder_layers:
            enc = layer(enc)
        dec = self.embed(tgt)
        for layer in self.decoder_layers:
            dec = layer(dec, enc_out=enc)
        return self.head(dec)

# 测试维度
model = MiniTransformer()
src = torch.randint(0, 1000, (2, 15))  # batch=2, src_len=15
tgt = torch.randint(0, 1000, (2, 10))  # tgt_len=10
logits = model(src, tgt)
print("src shape:", tuple(src.shape))
print("tgt shape:", tuple(tgt.shape))
print(" 输出 logits shape:", tuple(logits.shape))  # (2, 10, 1000)
```

6. 小结

1. Transformer = Encoder 栈 + Decoder 栈, 每层由 Attention + Add&Norm + FFN + Add&Norm 组成；Decoder 多一层 Cross-Attention。
2. 维度账本：输入嵌入 $n \times d_{\text{model}}$，各层维度不变，最后投影到 $n \times V$。
3. 训练用 Teacher Forcing（并行），推理用自回归（串行）。
4. Cross-Attention 中 Q 来自 Decoder，K/V 来自 Encoder。

7. 延伸阅读/进一步思考

- 原论文 Vaswani 等 2017 是 Encoder-Decoder 架构的出处；之后 BERT 只留 Encoder（理解类任务），GPT 只留 Decoder（生成类任务），这是架构上的两条分支。
- Teacher Forcing 带来的 exposure bias 是经典话题，scheduled sampling 等方法试图缓解；ch29 训练小语言模型时会再遇到。
- Encoder-Decoder（如 T5、BART）至今仍用于翻译、摘要等”输入输出都是序列”的任务。

8. 思考题

基础题

1. GPT 系列只有 Decoder 没有 Encoder，它的 Cross-Attention 去哪里了?

进阶题

2. 为什么训练时可以并行计算所有位置的损失，而推理时必须一个一个生成?
3. 如果把 Encoder 层数从 6 加到 24，参数量大约翻几倍?（提示：每层参数量近似常数，看 ch15。）

开放题

4. Encoder-Decoder 结构有”编码一次、多次解码”的优势（翻译时 Encoder 跑一遍，Decoder 自回归）。GPT 类 Decoder-only 模型没有独立 Encoder，它怎么处理”长输入”？从 KV cache 和计算分配角度分析。

第 11 讲 位置编码：正余弦编码的完整推导

> **本讲问题**：Self-Attention 对顺序完全不敏感——把句子的词打乱它根本不在意。那 Transformer 怎么知道”狗咬人”和”人咬狗”是两回事？位置信息从哪里来？

1. 数学工具/预备知识

ch07 的 Self-Attention 置换不变性。注意力权重只取决于 Q/K 相似度，和绝对位置无关。

三角恒等式。 $\sin(\alpha+\beta)=\sin\alpha\cos\beta+\cos\alpha\sin\beta$，$\cos(\alpha+\beta)=\cos\alpha\cos\beta-\sin\alpha\sin\beta$。

向量点积与余弦相似度。两向量点积衡量方向相似度。

2. 完整推导

2.1 Self-Attention 的置换不变性

把输入序列 $X\in\mathbb{R}^{n\times d}$ 的行打乱成 X'（同样的行、不同顺序），Self-Attention 的输出也只是对应行被打乱，**每个输出向量的集合不变**。原因：注意力权重 α_{ij} 只取决于 q_i 和 k_j 的相似度，而 q_i,k_j 只来自 x_i,x_j 本身，和它们在序列里的绝对位置 i,j 无关。

这意味着：不加任何位置信息，“猫追狗”和”狗追猫”对 Transformer 是同一个句子。必须显式注入位置信息。

2.2 位置编码的设计目标

位置编码 $p_t\in\mathbb{R}^{d_{\text{model}}}$ 应满足：1. 每个位置有唯一编码。2. 模型能从中推断相对距离（位置 t 和 $t+k$ 的关系可学习）。3. 能泛化到训练时没见过的更长序列。

2.3 正余弦位置编码推导

原论文用固定正余弦编码：

$$PE_{(t,2i)}=\sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(t,2i+1)}=\cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right)$$

其中 t 是位置，i 是维度索引（$i=0,\dots,d_{\text{model}}/2-1$）。

关键性质：相对位置可计算。记 $\omega_i=1/10000^{2i/d_{\text{model}}}$。考虑位置 t 和 $t+k$（k 是相对偏移），用三角恒等式展开：

$$PE_{t+k,2i}=\sin(\omega_i(t+k))=\sin(\omega_i t)\cos(\omega_i k)+\cos(\omega_i t)\sin(\omega_i k)$$

$$PE_{t+k,2i+1}=\cos(\omega_i(t+k))=\cos(\omega_i t)\cos(\omega_i k)-\sin(\omega_i t)\sin(\omega_i k)$$

写成矩阵形式：

$$\begin{bmatrix} PE_{t+k,2i} \\ PE_{t+k,2i+1} \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix}}_{\text{旋转矩阵} R(\omega_i k)} \begin{bmatrix} PE_{t,2i} \\ PE_{t,2i+1} \end{bmatrix}$$

位置 $t+k$ 的编码等于位置 t 的编码**旋转一个与 k 有关的角度**。这意味着：相对偏移 k 对所有绝对位置 t 的影响是同一个线性变换 $R(\omega_i k)$——模型可以通过学一个线性变换，来关注”偏移 k 个位置”的邻居。这正是我们要的相对位置性质。

不同维度 i 对应不同频率 ω_i：i 小则 ω_i 大（高频，变化快），i 大则 ω_i 小（低频，变化慢）。高频维度捕捉局部位置细节，低频维度捕捉全局位置关系。

2.4 加入方式

位置编码直接**逐元素加到**输入嵌入上：

$$x_t^{\text{input}} = x_t^{\text{embed}} + PE_t$$

不经过线性投影，直接相加。这要求嵌入和位置编码维度相同（都是 d_{model}）。

2.5 位置编码向量的点积

两个位置 t_1, t_2 的位置编码点积：

$$PE_{t_1} \cdot PE_{t_2} = \sum_i \left[\sin(\omega_i t_1)\sin(\omega_i t_2) + \cos(\omega_i t_1)\cos(\omega_i t_2)\right]$$

用余弦差公式 $\cos(a-b) = \cos a \cos b + \sin a \sin b$：

$$= \sum_i \cos(\omega_i (t_1 - t_2))$$

点积只依赖距离 $|t_1 - t_2|$，不依赖绝对位置。距离越近，余弦相似度越高——这正是相对位置性质。

3. 直觉理解

> **直觉**：位置编码像给每个 token 发一个带时间戳的工牌——工牌上印着不同频率的正弦波。远处的位置看高频波对不上（差异大），近处的位置看所有波都对得上（相似）。更妙的是，两个工牌之间的”相位差”只取决于它们隔多远，不取决于站在哪里。

4. 常见误区

> **误区**一：以为位置编码是可学习参数。原论文用固定正余弦函数；后来 BERT 改成可学习位置嵌入（直接查表），但正余弦的相对位置性质是理论优势。
>
> **误区**二：把位置编码乘到注意力打分上。它加在输入嵌入上，只在第一层进入网络。
>
> **误区**三：以为频率 ω_i 的 10000 是随便取的。它是个超参，决定了频率范围；RoPE（ch22）沿用了这个设计。

5. 代码实现

下面生成正余弦编码并验证旋转性质。工程注意：用 `torch.exp` 配合 `log(10000)` 计算 `div_term`，避免直接算 $10000^{2i/d}$ 的数值问题；偶数维 sin、奇数维 cos。

```python
import torch
import math

def positional_encoding(seq_len, d_model):
    """ 生成正余弦位置编码，返回 (seq_len, d_model)"""
    PE = torch.zeros(seq_len, d_model)
    position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1)  # (seq, 1)
    # 频率: omega_i = 1/10000^(2i/d), 用 exp(log(10000)*(-2i/d)) 计算
    div_term = torch.exp(
        torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
    )  # (d_model/2,)
    PE[:, 0::2] = torch.sin(position * div_term)  # 偶数列
    PE[:, 1::2] = torch.cos(position * div_term)  # 奇数列
    return PE

# 生成并打印前几个位置
PE = positional_encoding(seq_len=10, d_model=8)
print(" 位置编码 (前 5 个位置, 前 8 维):")
print(PE[:5].numpy().round(3))

# 验证旋转性质: PE[t+k] 应等于 R(omega*k) PE[t]
t, k = 3, 2
omega = 1.0 / (10000 ** (0 / 8))  # i=0 时的频率
rotated = torch.tensor([
    PE[t,0]*math.cos(omega*k) + PE[t,1]*math.sin(omega*k),
    -PE[t,0]*math.sin(omega*k) + PE[t,1]*math.cos(omega*k)
])
print(f"\n位置 t={t} 的前两维: {PE[t,:2].tolist()}")
print(f" 旋转 k={k} 后预测: {rotated.tolist()}")
print(f" 实际位置 t+k={t+k}: {PE[t+k,:2].tolist()}")
print(f" 误差: {(rotated - PE[t+k,:2]).abs().max().item():.6f}")

# 验证点积只依赖距离
PE2 = positional_encoding(seq_len=20, d_model=16)
dot_1_3 = (PE2[1]*PE2[3]).sum()
dot_5_7 = (PE2[5]*PE2[7]).sum()
print(f"\n距离都是 2: PE[1]·PE[3]={dot_1_3:.4f}, PE[5]·PE[7]={dot_5_7:.4f} (应接近)")
```

6. 小结

1. Self-Attention 置换不变，必须显式注入位置信息。
2. 正余弦编码利用三角恒等式，使相对偏移 k 对应一个旋转矩阵——模型可通过线性变换学习相对位置。
3. 不同维度对应不同频率，高频捕捉局部、低频捕捉全局；位置编码点积只依赖相对距离。
4. 位置编码直接加到输入嵌入上。

7. 延伸阅读/进一步思考

- 原论文 Vaswani 等 2017 提出固定正余弦编码；BERT 改用可学习位置嵌入，效果差不多但失去了外推能力。
- RoPE（ch22，旋转位置编码）把”旋转”思想从输入端搬到注意力打分里，是现代 LLaMA 系列的选择，外推性更好。
- 想理解”为什么要相对位置”，可对比绝对位置嵌入：它能记住每个位置，但训练时没见过的位置（超长序列）就失效了。

8. 思考题

基础题

1. 为什么不直接用可学习位置嵌入（查表法）？固定正余弦编码有什么外推优势？

进阶题

2. 如果序列长度超过训练时见过的最大长度（比如训练最多 512，推理 1024），正余弦编码还能用吗？为什么？可学习嵌入呢？
3. RoPE（ch22）和正余弦编码有什么本质区别？（提示：一个把位置信息加在输入上，一个把位置信息乘在 Q/K 的相对旋转里。）

开放题

4. 正余弦编码的频率范围由 10000 控制。如果把它改小（比如 100），高频维度变多变少？这会影响模型区分相邻位置的能力吗？

第 12 讲 Add & Norm：残差连接与 LayerNorm 的原理

本讲问题：为什么 Transformer 能堆到几十上百层而不崩？残差连接到底修好了什么病？LayerNorm 又在归一化什么？这两个零件合起来，是深层 Transformer 训练稳定的基石。

1. 数学工具/预备知识

ch03 的梯度消失/爆炸。梯度沿层间连乘 c^L 指数衰减。

向量的均值与方差。 $\mu = \frac{1}{d}\sum_i x_i$， $\sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2$。

批归一化（BatchNorm）。对 batch 维做归一化，需要足够大的 batch 才稳定。

2. 完整推导

2.1 残差连接：梯度恒等通路

设子层函数为 $F(x)$，残差连接输出为：

$$\text{Layer}(x) = x + F(x)$$

反向传播时，损失 L 对输入 x 的梯度：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial (x + F(x))}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(1 + \frac{\partial F}{\partial x}\right)$$

关键是那个 **1**。即使 $\frac{\partial F}{\partial x} \to 0$（子层梯度消失），梯度仍有一个恒等通路 1 直接传回。不管子层多深多复杂，梯度至少能以系数 1 无损回传。

对比没有残差：$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial F}{\partial x}$，一旦 $\frac{\partial F}{\partial x} \to 0$，梯度就消失了。这从结构上解决了 ch03 的连乘衰减问题——每一层的梯度都拆成 $1+$ 一个小量，L 层连乘后那个 1 始终保留。

2.2 LayerNorm 公式

LayerNorm 对每个样本的特征维做归一化。设输入 $x \in \mathbb{R}^d$（单个样本的 d 维特征）：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

其中 $\gamma, \beta \in \mathbb{R}^d$ 是可学习缩放和偏置，ϵ 防止除零。

为什么不用 BatchNorm? BatchNorm 对 batch 维归一化，统计量依赖 batch 大小。序列长度变化时（不同句子长度不同），BatchNorm 的 batch 统计量不稳定，且 train/inference 行为不一致。LayerNorm 对特征维归一化，每个样本独立，与 batch 大小、序列长度都无关。

2.3 Pre-LN vs Post-LN

原论文用 **Post-LN**：先子层，再残差，再 Norm：

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

现代大模型普遍改用 **Pre-LN**：先 Norm，再子层，再残差：

$$\text{output} = x + \text{Sublayer}(\text{LayerNorm}(x))$$

梯度流分析：Post-LN 中 Norm 在残差通路外面，梯度要穿过 Norm 才能走恒等通路；Pre-LN 中 Norm 在子层内部，残差通路 x 完全不受 Norm 影响，梯度恒等通路更干净。这就是 Pre-LN 训练更稳定、能堆更深的原因（代价是最后一层输出没有被归一化，通常额外加一个最终 LN）。

2.4 为什么需要 LayerNorm

没有归一化时，网络加深后每层输出的均值方差不断漂移（covariate shift）。LayerNorm 强制每层输入均值 0、方差 1（再由 γ, β 恢复表达能力），让每层输入分布保持稳定，训练更容易。配合残差，它把 ch03 里”每层谱半径乘积”这个不受控的量重新拉回稳定区间。

3. 直觉理解

直觉：残差连接像高速公路——不管中间出口（子层）堵不堵，主路永远畅通（系数 1）。LayerNorm 像车速限制——不管谁上高速都先校准到统一速度，防止某辆车速度太快导致连环相撞。Pre-LN 是先校准再上高速，Post-LN 是先上高速再校准——显然前者更安全。

4. 常见误区

误区一：以为残差连接是”把输入复制一份”。它是 $x + F(x)$，输入加子层输出，不是简单复制。

误区二：以为 LayerNorm 和 BatchNorm 只是归一化维度不同。关键区别：BatchNorm 训练用 batch 统计量、推理用滑动平均，有 train/inference 不一致；LayerNorm 每样本独立，行为一致。

误区三：以为 γ, β 是多余的。初始 $\gamma = 1, \beta = 0$ 时 LayerNorm 只是标准化，γ, β 让网络能自己恢复需要的尺度和偏移。

5. 代码实现

下面手写 LayerNorm 对照官方实现，并演示残差的梯度恒等通路。注意 `torch.var(unbiased=False)` 用有偏方差（除以 d），和公式一致；PyTorch 官方 LayerNorm 也用有偏估计。

```
import torch
import torch.nn as nn

class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(d_model))
        self.beta = nn.Parameter(torch.zeros(d_model))
        self.eps = eps
```

```
    def forward(self, x):
        # x: (..., d_model)，对最后一维归一化
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)  # 有偏方差
        x_hat = (x - mean) / torch.sqrt(var + self.eps)
        return self.gamma * x_hat + self.beta

# 对照官方实现
torch.manual_seed(0)
x = torch.randn(2, 10, 512)

my_ln = LayerNorm(512)
official_ln = nn.LayerNorm(512)  # 默认 gamma=1, beta=0
out_mine = my_ln(x)
out_official = official_ln(x)
print(" 手动 LayerNorm 与官方差异:", torch.allclose(out_mine, out_official, atol=1e-5))

# 验证残差的梯度恒等通路
x = torch.randn(4, requires_grad=True)
f = lambda z: z * 0.1  # 梯度很小的子层
y = x + f(x)           # 残差
y.sum().backward()
print("\n残差连接下 x 的梯度:", x.grad.tolist(), "(应为 1 + 0.1 = 1.1)")

x2 = torch.randn(4, requires_grad=True)
y2 = f(x2)             # 无残差
y2.sum().backward()
print(" 无残差时 x 的梯度:", x2.grad.tolist(), "(应为 0.1)")
```

6. 小结

1. 残差连接 $x + F(x)$ 提供梯度恒等通路，$\partial L/\partial x = \partial L/\partial y \cdot (1 + \partial F/\partial x)$，从结构上缓解梯度消失。
2. LayerNorm 对特征维归一化，每样本独立，与 batch 无关。
3. Pre-LN（先 Norm 再子层）比 Post-LN 梯度流更干净，现代大模型普遍采用。
4. 归一化防止激活分布漂移，是深层网络训练稳定的必要条件。

7. 延伸阅读/进一步思考

- 残差连接出自 He 等 2015《Deep Residual Learning for Image Recognition》；LayerNorm 出自 Ba 等 2016《Layer Normalization》。
- Pre-LN vs Post-LN 的系统对比可参考 Xiong 等 2020《On Layer Normalization in the Transformer Architecture》。
- RMSNorm（仅缩放不中心化）是 LayerNorm 的简化变体，LLaMA 在用，计算更省。

8. 思考题

基础题

1. 如果残差改成 $x + 0 \cdot F(x)$（完全绕过子层），网络还能训练吗？这对应什么极端情况？

进阶题

2. LayerNorm 的 γ, β 初始化为 1 和 0，意味着训练开始时 LayerNorm 在做什么操作？
3. 为什么 Pre-LN 的原始 Transformer 性能略差于 Post-LN？（提示：考虑最后一层输出有没有被归一化。）

开放题

4. RMSNorm 只做 $\hat{x}_i = x_i / \sqrt{\text{mean}(x^2) + \epsilon} \cdot \gamma$，不减均值。从”归一化到底在稳定什么”的角度分析：去掉减均值这一步，会损失什么、省了什么？为什么大模型愿意接受这个 trade-off？

第 13 讲 前馈网络 FFN：逐位置 MLP 的作用

> **本讲问题**：注意力负责”信息交换”，FFN 负责什么？为什么中间层维度是 $4d_{\text{model}}$？这个 4 倍系数是怎么来的？为什么说大部分参数和知识都在 FFN 里？

1. 数学工具/预备知识

逐位置（position-wise）。同一个 MLP 被序列每个位置独立、相同地应用一次。

非线性激活函数。没有非线性，两层线性变换等价于一层。ReLU/GELU/SwiGLU 提供非线性表达能力。

参数量估算。 $W_1 \in \mathbb{R}^{d \times d_{ff}}$ 有 $d \cdot d_{ff}$ 个参数。

2. 完整推导

2.1 逐位置本质

FFN 对序列的**每个位置独立、相同**地应用同一个两层 MLP：

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

关键：同一个 W_1, W_2 被所有位置共享。和 Self-Attention 不同，FFN **不跨位置交换信息**——它只对每个位置的向量做独立变换。

这意味着分工：注意力层负责”位置之间通信”，FFN 负责”每个位置自己的信息加工”。两者交替，形成”交换信息 → 各自思考 → 再交换 → 再思考”的循环。

工程上，因为逐位置独立，可以把 batch 和 seq 两个维合并，写成一个大矩阵乘法，不需要 for 循环遍历位置。

2.2 为什么中间层是 $4d_{\text{model}}$

原论文取 $d_{ff} = 4d_{\text{model}} = 2048$（$d_{\text{model}} = 512$ 时）。这个 4 倍不是理论最优值，而是经验选择，但有合理解释：中间层要比输入维宽，才能在”升维 → 非线性 → 降维”中提供足够的容量。

FFN 参数量：- W_1：$512 \times 2048 = 1{,}048{,}576$ - W_2：$2048 \times 512 = 1{,}048{,}576$ - 合计约 2.1M 参数

对比 MHA 参数量（约 $4 \times 512 \times 512 = 1{,}048{,}576$，见 ch09），FFN 占整个 Transformer 层约 2/3 参数量。也就是说**大部分参数和知识存储在 FFN 里**，而不是注意力里。

2.3 事实存储器假说

研究发现，Transformer 的事实知识（“法国首都是巴黎”这类）主要存储在 FFN 的权重里。注意力路由负责”把对的信息送到对的位置”，FFN 负责”在那个位置上查询和调用知识”。这也是 FFN 需要更大维度的原因——它要存大量参数化知识。

2.4 激活函数演进

ReLU：$\max(0, x)$。简单，但负值区梯度为 0，可能导致”神经元死亡”。

GELU：$x\Phi(x)$，Φ 是标准正态 CDF。近似为 $0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$。BERT、GPT-2 起用 GELU 替代 ReLU，更平滑。

SwiGLU：$(\text{Swish}(xW_1) \odot (xW_2))W_3$。引入门控，PaLM、LLaMA 用。SwiGLU 通常把中间层从 $4d$ 调成 $\frac{8}{3}d$，因为它用三个矩阵而不是两个，参数量大致持平。

3. 直觉理解

直觉：注意力是开会讨论——每个人从别人那里获取信息。FFN 是会后各自整理笔记——把刚听到的信息消化、重组、存进自己的笔记本。开会本身不产生新知识，只是交换；真正的知识加工发生在独自思考的时候。FFN 维度设大 4 倍，是因为”笔记本”要够大能装下足够多的知识。

4. 常见误区

误区一：以为 FFN 和普通 MLP 一样是”全局变换”。FFN 逐位置独立，位置之间没有信息交换。这也是它能写成一个大矩阵乘法的原因。

误区二：以为 FFN 可有可无。消融实验表明去掉 FFN 性能大幅下降，它不是冗余的。

误区三：以为 FFN 的 4 倍是理论最优。它是经验值，不同模型从 2.67 到 8 都有。

5. 代码实现

下面对比 ReLU FFN 和 SwiGLU，并验证 FFN 的逐位置独立性：改一个位置的输入，只影响那个位置的输出。工程注意：SwiGLU 用三个矩阵，为保持参数量和 $4d$ 版持平，中间维取 $\frac{8}{3}d$。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleFFN(nn.Module):
    def __init__(self, d_model=512, d_ff=2048):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))

class SwiGLU(nn.Module):
    def __init__(self, d_model=512, d_ff=None):
        super().__init__()
        # 三矩阵保持参数量与两矩阵 4d 版相当: 3*(d*d_ff) ≈ 2*(d*4d)
        if d_ff is None:
            d_ff = int(8 * d_model / 3)
        self.w_gate = nn.Linear(d_model, d_ff)
        self.w_up = nn.Linear(d_model, d_ff)
        self.w_down = nn.Linear(d_ff, d_model)
    def forward(self, x):
        return self.w_down(F.silu(self.w_gate(x)) * self.w_up(x))

# 对比参数量
ffn = SimpleFFN(512, 2048)
swiglu = SwiGLU(512)
print(f"ReLU FFN 参数量: {sum(p.numel() for p in ffn.parameters()):,}")
print(f"SwiGLU 参数量:   {sum(p.numel() for p in swiglu.parameters()):,}")

# 验证逐位置: 改位置 0, 只影响位置 0 的输出
x = torch.randn(2, 5, 512)
out = ffn(x)
x_mod = x.clone()
x_mod[0, 0, :] += 100
out_mod = ffn(x_mod)
diff = (out_mod - out).abs()
print(f"\n输入 shape: {tuple(x.shape)}, 输出 shape: {tuple(out.shape)}")
print(f" 改变位置 0 后, 位置 0 输出差异: {diff[0,0].max().item():.4f}")
print(f" 改变位置 0 后, 位置 1 输出差异: {diff[0,1].max().item():.6f} (应 ≈0)")
```

6. 小结

1. FFN 对每个位置独立应用同一个两层 MLP，不跨位置交换信息。
2. 中间层 $d_{ff} = 4d_{\text{model}}$ 是经验选择，FFN 占整层约 2/3 参数量。
3. FFN 被认为是 Transformer 的”事实存储器”。
4. 激活从 ReLU 演进到 GELU 再到 SwiGLU，门控提升表达能力。

7. 延伸阅读/进一步思考

- 事实存储器假说可参考 Geva 等 2020《Transformer Feed-Forward Layers Are Key-Value Memories》。
- SwiGLU 出自 Shazeer 2020《GLU Variants Improve Transformer》；LLaMA、PaLM 均采用。
- 把 FFN 扩成多个专家就是 MoE（ch25）——逐位置地路由到不同 FFN，是”条件计算”。

8. 思考题

基础题

1. 如果把 FFN 中间层从 $4d$ 改成 d（瓶颈），会发生什么?

进阶题

2. 为什么 FFN 可以写成一个大矩阵乘法，而不用 for 循环遍历每个位置?
3. SwiGLU 用三个矩阵代替原来两个，为什么还能保持参数量相当?

开放题

4. MoE（ch25）把 FFN 换成多个专家网络、每个 token 只激活其中一两个。从”FFN 存事实”的角度分析：为什么把 FFN 做成稀疏多个专家能在不显著增加计算量的前提下扩大知识容量?

第 14 讲 Mask 机制：Padding Mask 与 Causal Mask

本讲问题：为什么 Decoder 的 Self-Attention 要把未来位置遮掉? Padding 是怎么不污染注意力的? 这两个 Mask 分别长什么样? 为什么必须加在 Softmax 之前?

1. 数学工具/预备知识

ch04 的 Softmax。把 logits 变成概率；给 logits 加 $-\infty$ 会使对应项 $\exp(-\infty) = 0$。

上三角/下三角矩阵。下三角矩阵 $M_{ij} = 1$ 当 $j \le i$，否则 0。

序列填充（padding）。 batch 里句子长度不一，短的补 <pad> 对齐到最长长度。

2. 完整推导

2.1 两种 Mask 的动机

Padding Mask：一个 batch 里句子长度不同，短句要补 <pad> 对齐。这些 <pad> 位置没有实际意义，不能让注意力关注到它们。

Causal Mask（因果掩码）：Decoder 自回归生成时，位置 t 只能看到 $1, \dots, t$，不能看到 $t+1, \dots$（否则偷看答案）。

2.2 Softmax 前注入 -∞

Mask 的实现：在 Softmax 之前，把要屏蔽位置的 score 设成 $-\infty$（工程上用大负数如 -10^9）：

$$\text{MaskedScores}_{ij} = \begin{cases} S_{ij}, & \text{允许关注} \\ -10^9, & \text{禁止关注} \end{cases}$$

经过 Softmax：

$$\alpha_{ij} = \frac{\exp(\text{MaskedScores}_{ij})}{\sum_k \exp(\text{MaskedScores}_{ik})}$$

被屏蔽位置 $\exp(-10^9) \approx 0$，权重趋近 0。数学上等于完全不关注。关键：必须在 Softmax **之前**加，因为 Softmax 会归一化；之后乘 0 会破坏归一化（权重和不再为 1）。

2.3 Causal Mask 的下三角结构

对序列长度 n，Causal Mask 是 $n \times n$ 矩阵，下三角为 1、严格上三角为 0：

$$\text{Mask}_{ij} = \begin{cases} 1, & j \le i \\ 0, & j > i \end{cases}$$

位置 i 可以看到所有 $j \le i$，看不到 $j > i$。

2.4 Padding Mask 的来源

batch 中两句长度 3 和 5，pad 后统一长度 5。Padding Mask 形状 `(batch, 1, seq_len)`，标记哪些是真实 token：

$$\text{PaddingMask}_{b,j} = \begin{cases} 1, & \text{位置} j \text{ 真实} \\ 0, & \text{位置} j \text{ 是 padding} \end{cases}$$

注意力打分中，padding 位置作为 key 时被屏蔽（任何 query 都不能关注 pad）。

2.5 交叉注意力的 Mask

Cross-Attention 中 Q 来自 Decoder，K/V 来自 Encoder。Encoder 侧可能有 padding，所以 Cross-Attention 的 Mask 来自 Encoder 侧 Padding Mask——Decoder 不能关注 Encoder 的 pad 位置。

3. 直觉理解

直觉：Padding Mask 像考试时把草稿纸（pad 位置）折起来——不能看草稿。Causal Mask 像做阅读理解题目一页一页发——你只能看到当前页和之前页，不能翻到后面看答案。两个 Mask 都在 Softmax 打分前把不该看的地方打 $-\infty$ 标签，让 Softmax 自动忽略。

4. 常见误区

误区一：以为 Mask 在 Softmax 之后乘 0。那样破坏归一化（权重和不再为 1）。正确做法是 Softmax **之前**把 logits 设 $-\infty$。

误区二：以为 Causal Mask 只在训练时需要。推理时也需要——KV cache 里已有历史，当前位置只能关注已生成 token。

误区三：以为 Mask 方向无所谓。Causal Mask 写反（上三角为 1）会让模型看到未来，训练直接作弊。

5. 代码实现

下面构造 Causal Mask 和 Padding Mask，并验证效果。工程注意：`torch.tril` 生成下三角；`masked_fill(mask==0, -1e9)` 里 mask 的 True 表示允许；bool 掩码在 GPU/CPU 间行为一致。

```python
import torch
import torch.nn.functional as F

# Causal Mask: 下三角全 1, 上三角全 0, True 表示允许关注
def causal_mask(seq_len):
    return torch.tril(torch.ones(seq_len, seq_len)).bool()

# Padding Mask
def padding_mask(seq_lens, max_len):
    """seq_lens: 每个样本真实长度列表"""
    mask = torch.zeros(len(seq_lens), 1, max_len, dtype=torch.bool)
    for i, l in enumerate(seq_lens):
        mask[i, 0, :l] = True
    return mask

# 测试 Causal Mask
n = 5
cm = causal_mask(n)
print("Causal Mask (True= 允许):")
print(cm.int())

# 验证: 位置 i 只能看到 j<=i
scores = torch.randn(1, 1, n, n)  # (batch, heads, q_len, k_len)
scores_masked = scores.masked_fill(~cm, -1e9)
alpha = F.softmax(scores_masked, dim=-1)
print("\n位置 0 的注意力权重 (应只在位置 0 非零):", alpha[0,0,0].detach().round())
print(" 位置 2 的权重 (应在 0,1,2 非零, 3,4 为 0):", alpha[0,0,2].detach().round(decimals=3))

# 验证 Padding Mask
pm = padding_mask([3, 5], max_len=5)
print("\nPadding Mask (batch=2, lens=[3,5]):")
print(pm[0,0].int(), "(第一个句子只有 3 个真实位置)")
print(pm[1,0].int(), "(第二个句子有 5 个)")
```

6. 小结

1. Padding Mask 屏蔽 <pad> 位置，避免无意义位置参与注意力。
2. Causal Mask 屏蔽未来位置，保证 Decoder 自回归性质。
3. Mask 在 Softmax 前注入 $-\infty$，利用 Softmax 数学特性自然产生零权重。
4. Cross-Attention 的 Mask 来自 Encoder 侧 Padding Mask。

7. 延伸阅读/进一步思考

- Causal Mask 是 Decoder-only 模型（GPT）训练并行化的关键：一次前向算出所有位置的预测，靠 mask 保证不偷看。
- KV cache（ch23）下，推理时 Causal Mask 退化成”当前 query 关注所有已缓存 key”，不再是完整 $n \times n$ 矩阵。
- FlashAttention（ch24）的一个优化就是把 mask 融合进 kernel，避免显式构造大掩码矩阵。

8. 思考题

基础题

1. 如果把 Causal Mask 从下三角改成上三角，模型会变成什么?

进阶题

2. 推理时 KV cache 中已有 t 个 token，新位置 t 的 Causal Mask 应该是什么形状?
3. 为什么不在输入嵌入层就把 pad 位置向量置零，而要在注意力里 Mask?

开放题

4. 滑窗注意力（Sliding Window Attention）把每个 query 的可见范围限制在附近 w 个位置（类似局部 mask）。从 ch16 复杂度和长程依赖两个角度分析：这种局部 mask 牺牲了什么、换来了什么?

第 15 讲 参数量与计算量：一层 Transformer 的 FLOPs 分析

> **本讲问题**：一个”7B 模型”的 7B 到底是什么？训练一次要花多少钱？推理一秒能跑多少 token？这些数字怎么从架构算出来？本讲把一层 Transformer 的参数和算力账算到底。

1. 数学工具/预备知识

矩阵乘法 FLOPs。$A \in \mathbb{R}^{m\times k}$ 乘 $B \in \mathbb{R}^{k\times n}$，要算 mn 个点积，每个点积 k 次乘加，约 $2mnk$ 次浮点运算（乘一次加一次各算一次）。

参数量。等于权重矩阵元素总数，一个 $\mathbb{R}^{m\times k}$ 矩阵有 mk 个参数。

前向 vs 反向。反向传播梯度计算量约为前向的 2 倍，所以训练（前向 + 反向）约为 $3\times$ 前向。

2. 完整推导

2.1 一层 Transformer 的参数量

设 $d_{\text{model}} = d$，FFN 中间层 $d_{ff} = 4d$。

MHA 参数量（ch09）：四个 $d \times d$ 矩阵 W_Q, W_K, W_V, W_O，合计 $4d^2$。偏置忽略（现代大模型通常不带偏置）。

FFN 参数量（ch13）：$W_1 : d \times 4d$，$W_2 : 4d \times d$，合计 $8d^2$。

LayerNorm 参数量：每层 2 个 LN（MHA 后一个，FFN 后一个），每个有 γ, β 各 d 个参数，合计 $4d$。

一层总参数量：$4d^2 + 8d^2 + 4d = 12d^2 + 4d \approx 12d^2$。

2.2 注意力 FLOPs

对序列长度 n。

- Q, K, V 投影：三个 $n \times d \times d$ 矩阵乘，$3 \times 2nd^2 = 6nd^2$。
- $QK^\top$：$n \times n \times d$，约 $2n^2d$。
- Softmax：$O(n^2)$，可忽略。
- Attention $\cdot V$：$n \times n \times d$，约 $2n^2d$。
- W_O 投影：$2nd^2$。

MHA 总 FLOPs：$8nd^2 + 4n^2d$。

2.3 FFN FLOPs

- W_1：$n \times d \times 4d$，约 $2n \cdot d \cdot 4d = 8nd^2$。
- W_2：$n \times 4d \times d$，约 $8nd^2$。

FFN 总 FLOPs：$16nd^2$。

2.4 一层总 FLOPs

$$\text{FLOPs}_{\text{layer}} = (8nd^2 + 16nd^2) + 4n^2d = 24nd^2 + 4n^2d$$

短序列 $n \ll d$ 时 $24nd^2$（线性项）主导；长序列 $n \gg d$ 时 $4n^2d$（二次项）主导。

2.5 原论文配置

$d = 512, n = 1024, L = 6$（Encoder 6 层 + Decoder 6 层）：- 每层参数量 $12 \times 512^2 \approx 3.7M$。- Encoder+Decoder 共 12 层：$12 \times 3.7M \approx 44M$。- 加嵌入层和输出投影（共享权重时约 $V \times d = 37000 \times 512 \approx 19M$）。- 总参数量约 65M。

2.6 推理成本

自回归推理每生成一个 token：前向一遍所有层，约 $2N_{\text{params}}$ FLOPs（乘加各算一次）。对 L 层模型，每 token 推理 FLOPs $\approx 2N_{\text{params}}$。

3. 直觉理解

直觉：参数量像书的字数，FLOPs 像读书花的时间。FFN 占大部分字数（$8d^2$ vs 注意力 $4d^2$），但读长句子时注意力部分（$n^2 d$）越来越费时——每个词要和所有词比较。这就是长序列推理特别贵的原因。

4. 常见误区

误区一：以为 FLOPs 和参数量成正比。短序列是的，但长序列注意力 n^2 项主导——序列翻倍，注意力成本翻四倍。

误区二：把训练和推理 FLOPs 混为一谈。训练要反向传播约 $3\times$ 前向；推理只做前向。

误区三：以为”7B”是 FLOPs。7B 指 7B 参数，不是计算量。

5. 代码实现

下面用代码核算参数量和 FLOPs。工程注意：FLOPs 是估算，实际含 softmax、LayerNorm、激活等小项，通常只数矩阵乘；训练 FLOPs 乘 3 是经验系数（前向 1 + 反向约 2）。

```python
def count_transformer_params(d_model=512, n_layers=6, vocab_size=37000):
    """ 统计 Transformer 参数量"""
    mha_params = 4 * d_model * d_model           # W_Q/K/V/O
    ffn_params = d_model * 4*d_model + 4*d_model * d_model  # W1, W2
    ln_params = 2 * 2 * d_model                   # 每层 2 个 LN, 每个 gamma+beta
    per_layer = mha_params + ffn_params + ln_params
    total = per_layer * n_layers + vocab_size * d_model  # 加嵌入
    return total, per_layer

def count_transformer_flops(d_model=512, seq_len=1024, n_layers=6):
    """ 估算一层训练 FLOPs (前向 + 反向 ≈3x)"""
    mha_flops = 8 * seq_len * d_model**2 + 4 * seq_len**2 * d_model
    ffn_flops = 16 * seq_len * d_model**2
    per_layer = mha_flops + ffn_flops
    total_train = per_layer * n_layers * 3
    return total_train, per_layer

# 原论文配置: Encoder 6 层 + Decoder 6 层 = 12 层
total_params, per_layer = count_transformer_params(n_layers=12)
print(f" 原论文 Transformer 总参数量 (12 层 + 嵌入): {total_params/1e6:.1f}M")
print(f" 每层参数量: {per_layer/1e6:.2f}M")

# FLOPs（按 12 层计）
total_flops, layer_flops = count_transformer_flops(n_layers=12)
print(f"\n每层训练 FLOPs: {layer_flops/1e9:.2f}G")
print(f"12 层训练 FLOPs (每 token): {total_flops/1e9:.2f}G")
print(f" 训练 10 亿 token 总 FLOPs: {total_flops*1e9/1e12:.0f}T")
```

```python
# 验证 GPT-3 量级: 175B 参数, 96 层, d=12288
d_gpt3 = 12288
per_layer_gpt3 = 12*d_gpt3**2 + 4*d_gpt3
print(f"\nGPT-3 每层参数量约: {per_layer_gpt3/1e6:.1f}M, 96 层约: {per_layer_gpt3*96/1e9:.1f}B")
```

6. 小结

1. 一层参数量约 $12d^2$，FFN 占 $8d^2$（2/3），MHA 占 $4d^2$（1/3）。
2. 一层 FLOPs $24nd^2 + 4n^2d$，短序列线性项主导，长序列 n^2 项主导。
3. 训练 FLOPs 约 $3\times$ 前向。
4. 推理每生成一个 token 成本约 $2N_{\text{params}}$。

7. 延伸阅读/进一步思考

- FLOPs 估算的经典参考是 Kaplan 等 2020《Scaling Laws for Neural Language Models》和 Hoffmann 等 2022《Chinchilla》，它们用类似估算拟合模型性能和算力的关系。
- Chinchilla 结论：模型参数量和训练 token 数应近似等比例增长；这指导了后来大模型的配比。
- 推理成本除了 FLOPs，还有内存带宽瓶颈（ch23 KV cache、ch24 FlashAttention），实际推理常常不是算不动而是搬不动数据。

8. 思考题

基础题

1. 把 FFN 中间层从 $4d$ 改成 $8d$，一层参数量增加多少？FLOPs 增加多少？

进阶题

2. 序列长度从 1024 翻倍到 2048，注意力 FLOPs 翻几倍？FFN FLOPs 翻几倍？
3. GPT-3 有 175B 参数、96 层、$d = 12288$，验证每层参数量是否约 $12d^2$。

开放题

4. 推理时每 token FLOPs $\approx 2N_{\text{params}}$，但实测速度常常远低于这个理论值。从”内存带宽 vs 计算”的角度解释：为什么大模型推理常常是 memory-bound 而不是 compute-bound？（预告 ch23/KV cache、ch24/FlashAttention。）

第 16 讲 复杂度对比：Transformer vs RNN vs CNN

> **本讲问题**：原论文凭什么说 Transformer 比 RNN 和 CNN 都好？“每层复杂度”“最大路径长度”“串行度”这三个指标到底在比什么？为什么同样是 $O(nd^2)$，Transformer 的墙钟时间反而短得多？

1. 数学工具/预备知识

在展开对比之前，先把本讲要用的三件工具讲清楚，后面每个公式都会用到它们。

(1) 大 O 记号在本讲里的含义。我们说一个操作是 $O(\cdot)$，指的是”当规模趋于无穷时，它的 FLOPs 或步数随哪个变量多项式增长”。本讲有三个独立变量：序列长度 n、隐藏维度 d（每个 token 向量的宽度）、卷积核宽度 k。比较复杂度时必须固定另外两个变量，只动一个——这一点后面会反复用到。

(2) 矩阵乘 FLOPs 的口径。一个 $m \times k$ 矩阵乘一个 $k \times n$ 矩阵，得到 $m \times n$，需要 $2mnk$ 次浮点运算（每个输出元素做 k 次乘加，乘和加各算一次，所以乘出来是 k 个乘法 k 个加法，共 $2k$ 次）。这是 ch15 已经建立的口径。

(3) 并行度与串行度。“串行度”指的是一张计算图里最长的那条依赖链有多长——你必须等前一步算完才能算下一步。串行度越高，GPU 里越多的计算单元必须闲着排队。并行度高意味着大量计算可以同时扔进 GPU 同时做。这是本讲最容易被忽略、却最重要的维度：**理论 FLOPs 少不代表跑得快，能塞满 GPU 才跑得快**。

（4）归纳偏置（inductive bias）。指模型架构本身自带的”先验假设”。比如 CNN 天然假设”局部相邻的东西相关”，RNN 天然假设”时间是有先后顺序的”。归纳偏置越强，模型在小数据上越容易学到东西（因为它已经被人为塞进了很多常识），但在数据极多、任务不匹配时反而受限（因为它被框死了）。

2. 完整推导

2.1 原论文 Table 1 逐格拆解

原论文《Attention Is All You Need》里有一张著名的对比表。它不是凭空给的结论，每一格都可以从结构推出来。我们逐格推导。

架构	每层复杂度	最大路径长度	串行度（训练时）
Self-Attention	$O(n^2 d)$	$O(1)$	低
Recurrent（RNN）	$O(nd^2)$	$O(n)$	高
Convolutional（CNN）	$O(ndk)$	$O(\log_k n)$	中

Self-Attention 为什么是 $O(n^2 d)$。一层注意力里，最贵的两步是 $QK^\top$ 和 $\text{softmax}(QK^\top/\sqrt{d_k})V$。$Q, K, V$ 都是 $n \times d$，$QK^\top$ 是 $n \times n$，每一项由 d 次乘加得到，所以这一步是 $2n^2 d$；再乘 V 又是 $2n^2 d$。合起来关于 n 是平方级、关于 d 是线性级，记作 $O(n^2 d)$。注意这里没有把 Q, K, V 各自的线性投影 $O(nd^2)$ 算进去——原论文那张表比较的是”序列依赖部分”的代价，投影部分三种架构都有，不影响相对比较。

RNN 为什么是 $O(nd^2)$。RNN 在每个时间步 t 做一次矩阵乘法 $h_t = \phi(W_{ih}x_t + W_{hh}h_{t-1})$，其中 W_{hh} 是 $d \times d$。一个时间步一次 $d \times d$ 矩阵乘，是 $O(d^2)$；一共有 n 个时间步，所以整层是 $O(nd^2)$。关于 n 是线性的——这是它表面上便宜的来源。

CNN 为什么是 $O(ndk)$。卷积在每个位置上用一个长度 k 的窗口扫一遍，每个位置做 k 次乘加、一共 d 个通道，单位置是 $O(dk)$，n 个位置就是 $O(ndk)$。k 通常是个小常数（3 或 5），所以它关于 n 也是线性的。

2.2 三个复杂度放在一起：临界点在哪

把 Self-Attention 的 $n^2 d$ 和 RNN 的 nd^2 比一比，谁便宜取决于谁大：

$$n^2 d \lessgtr nd^2 \quad \Longleftrightarrow \quad n \lessgtr d$$

- 当序列很短（$n < d$）：$n^2 d < nd^2$，注意力更便宜。
- 当序列很长（$n > d$）：$n^2 d > nd^2$，RNN 更便宜。

原论文当时翻译任务的 n（约 1 万 token 级）和 d（512）恰好使得 $n^2 d \approx nd^2$，两者打平——但这**不是** Transformer 赢的原因。真正的原因在下面两行。

2.3 最大路径长度：长距离依赖能不能学到

“最大路径长度”指序列里任意两个位置的信息要直接发生关系，中间必须经过多少层运算。这个量直接决定梯度能不能从远端一路传回来。

RNN 是 $O(n)$。位置 1 的信息要影响位置 n，必须依次经过 $h_1 \to h_2 \to \cdots \to h_n$，一共 $n-1$ 个时间步。反过来，损失对位置 1 的梯度也要沿着同一条链回传 $n-1$ 次，每次都乘上一个雅可比矩阵 $\partial h_t / \partial h_{t-1}$。当这个雅可比的谱半径小于 1 时，梯度每传一步就乘一次小于 1 的数，连乘 n 次后趋近于 0——这就是 ch03 讲的梯度消失。n 越大，位置 1 和位置 n 之间越学不到依赖。

CNN 是 $O(\log_k n)$。一个核宽 k 的卷积，一层能让距离 k 以内的位置直接交流。叠两层，感受野变成约 $2k-1$（第一层每个位置看到 k 个邻居，第二层再把这 k 个位置各自的 k 个邻居合起来，所以覆盖长度大约翻一番）。叠 L 层感受野大约是 k^L。要覆盖整个长度 n，需要 $k^L \gtrsim n$，即 $L \gtrsim \log_k n$。路径长度是对数级的——比 RNN 的线性好，但仍然要堆很多层才能把两个远端位置连起来。

Self-Attention 是 $O(1)$。任意两个位置 i, j，在同一个注意力层里直接算一个分数 $q_i^\top k_j$。它们之间的路径长度就是 1——不管序列多长，两个位置第一次见面就是零距离。梯度从位置 j 回流到位置 i 只经过一层。

几何/物理意义：可以把路径长度想象成”两地之间的交通时间”。RNN 是单条盘山公路，每过一站堵一次；CNN 是一层层修立交桥，修一层视野扩一圈；Self-Attention 是直接修了一条从任意点到任意点的直达高铁。路越短，信号（梯度）在途中衰减得越少，长距离关系就越能被模型利用。

2.4 归纳偏置：便宜不是白给的

这一段解释为什么 Transformer 表面上”没有先验”。

- **RNN 的归纳偏置**：时间步是严格有序的，且”越近的时间步越重要”。这是一个很强的先验——它天然适合”前面的词按顺序影响后面”这类任务。但代价是：如果两个相隔很远的词关系很强（比如一个长句子开头的主语和句尾的动词），RNN 也必须硬着头皮一步一步传过去。
- **CNN 的归纳偏置**：局部性——相邻的 token 更可能相关。它假设”语言是局部马尔可夫的”。这对短语有用，但对需要跨越整段长距离的指代（比如”它到底指的是张三还是李四”）无能为力，除非堆很多层。
- **Self-Attention 的归纳偏置**：几乎为零。它不对”谁该和谁近”做任何假设，让数据自己学。这是它灵活的原因，也是它吃数据的原因——没有先验帮它，它就得靠海量样本自己把”语法规则”背下来。

2.5 并行度：为什么这才是胜负手

现在回到最关键的一行——训练时的串行度。

- **RNN**：第 t 步必须等第 $t-1$ 步的隐藏状态算出来才能开始，整段序列必须一个时间步一个时间步跑。GPU 里几千个计算核心，同一时刻只有一个核心在干活，其余全部闲置。理论 FLOPs nd^2 虽然小，但墙钟时间是 n 倍串行开销。
- **CNN**：层与层之间必须串行，但同一层里所有位置的卷积是互相独立的，可以同时算。并行度中等。
- **Self-Attention**：所有位置的 Q, K, V 投影、$QK^\top$ 矩阵、softmax、乘 V，全部一次性并行算完。n 个位置在 GPU 上几乎是同时被处理的。

这就是为什么原论文敢在 $n \approx d$ 的持平点上选 Transformer：**它的 FLOPs 没有更小，但它能把 GPU 灌满**。RNN 那点理论优势在墙钟时间上完全被串行浪费掉了。

2.6 一个能说明问题的小任务：复制任务

考虑一个极端任务：给序列 $x_{1:n}$，要求模型原样输出一遍。

- **RNN**：必须把整段序列压缩进一个固定维度 d 的隐藏向量里。当 $n > d$ 时，隐藏向量装不下这么多信息，复制必然出错——这是信息瓶颈。
- **CNN**：远端位置要靠堆层才能”看到”彼此，层数不够就复制不到远处的 token。
- **Self-Attention**：位置 i 的输出可以直接 attend 到位置 i 自己的输入（加上位置编码后），路径长度 $O(1)$，一次就把原 token 取过来，能完美复制。

这个任务是检验”长距离信息能不能无损流过”的试金石。

2.7 注意力的隐藏代价：显存

复杂度表里没写的一项是显存。注意力要存一个 $n \times n$ 的分数矩阵（ch24 会讲它有多大）。当 n 到几千、几万时，这个矩阵本身就吃掉大量显存——这也是为什么长序列训练后来要靠 FlashAttention 把中间矩阵从显存里”擦掉”。RNN 和 CNN 不需要这个 $n \times n$ 矩阵，显存是 $O(n)$ 的。所以注意力的 $O(1)$ 路径长度和全并行是有代价的：它用 n^2 的显存换来了 $O(1)$ 的路径长度。理解这个 trade-off，才能理解后面 ch23、ch24 为什么存在。

3. 直觉理解

> **直觉**：RNN 像单线程程序——一行一行往下跑，顺序感很强，但慢，而且第 1000 行要记得第 1 行说了什么，全靠脑子（隐藏向量）硬背。CNN 像多核局部并行——每个核只盯着周围一小块，跑得快但视野窄，想看到远处得堆很多层楼。Self-Attention 像一场全员圆桌会议——所有人同时和所有人对视，谁和谁都能直接对话，速度最快，但正因为没有任何”谁该和谁聊”的默认规则，它需要最多的会议资料（数据）才能学会什么该聊什么不该聊。

4. 常见误区

误区一：以为 $O(n^2 d)$ 总比 $O(nd^2)$ 好。当 $n < d$ 时 RNN 更便宜；当 $n > d$ 时注意力更贵。比较复杂度必须先固定其他变量。

误区二：以为 FLOPs 小就一定跑得快。RNN 的 $O(nd^2)$ 里藏着 n 倍串行依赖，GPU 根本喂不饱；注意力虽然 FLOPs 多，但全并行，墙钟时间反而短。这就是”算力利用率”比”理论 FLOPs”更重要的原因。

误区三：以为没有归纳偏置就一定更好。小数据集上，CNN/RNN 的强先验反而是救命稻草——相当于老师先帮你划了重点；Transformer 这种”零先验”模型在小数据上会严重过拟合。Transformer 统治世界的前提是互联网级别的数据量。

5. 代码实现

下面这段代码实测两种”架构”在不同序列长度下的墙钟时间。为了公平，我们直接模拟计算量：注意力用一次 $QK^\top$ 乘 V，RNN 用逐时间步的循环。这个对比的重点不是绝对数字，而是看**随 n 增长时谁涨得快、谁能被并行吃掉**。

```
import time
import torch

torch.manual_seed(0)
d = 256          # 隐藏维度，固定不变（CPU 沙箱里取小一点，避免 OOM）
n_list = [64, 128, 256, 512]

print(f"{'seq_len':>8} | {'Attn(ms)':>10} | {'RNN 串行 (ms)':>12} | {'比值':>8}")
print("-" * 48)

for n in n_list:
    # --- 模拟 Self-Attention 的核心两次矩阵乘 ---
    # Q,K,V: (batch=4, head=4, n, d_k=32)。这部分可以整段并行扔给 GPU/CPU 矩阵引擎。
    Q = torch.randn(4, 4, n, 32)
    K = torch.randn(4, 4, n, 32)
    V = torch.randn(4, 4, n, 32)
    # 预热一次（排除第一次 kernels 分配显存的抖动）
    _ = torch.softmax(Q @ K.transpose(-2, -1) / (32 ** 0.5), dim=-1) @ V

    start = time.time()
    for _ in range(10):
        scores = Q @ K.transpose(-2, -1) / (32 ** 0.5)   # QK^T: O(n^2 d)
        out = torch.softmax(scores, dim=-1) @ V          # 再乘 V: 又一个 O(n^2 d)
    attn_time = (time.time() - start) / 10 * 1000

    # --- 模拟 RNN 逐时间步串行 ---
    # 每一步都必须等上一步的 h 算完，这个 for 循环天然串行，正好暴露 RNN 的短板。
    x = torch.randn(n, 4, d)
    W = torch.randn(d, d)
    start = time.time()
    for _ in range(3):
        h = torch.zeros(4, d)
        for t in range(n):                # 关键：这个 for 循环无法并行
            h = torch.tanh(x[t] @ W + h @ W)
    rnn_time = (time.time() - start) / 3 * 1000

    print(f"{n:>8} | {attn_time:>10.1f} | {rnn_time:>12.1f} | {rnn_time/attn_time:>8.1f}x")

print("\n说明：RNN 那列随 n 几乎线性上涨且无法加速；")
print(" 注意力虽然 FLOPs 是 n^2，但整段交给矩阵引擎并行，实测并不慢。")
print("（真实 GPU 上注意力的并行优势会比这里 CPU 上更夸张。）")
```

6. 小结

1. 三种架构每层复杂度：Self-Attention $O(n^2 d)$，RNN $O(nd^2)$，CNN $O(ndk)$；谁便宜取决于 n 和 d 谁大。
2. 最大路径长度决定长距离依赖能否学到：Attention $O(1)$，CNN $O(\log_k n)$，RNN $O(n)$。
3. 串行度才是胜负手：Attention 全并行能喂饱 GPU，RNN 强串行浪费算力。
4. 归纳偏置强弱与数据量需求成反比：Attention 零先验最灵活但最吃数据。

7. 思考题

基础题

1. 当序列长度 n 远小于 d 时，三种架构里哪种每层计算最便宜？为什么？

进阶题

2. 从”梯度连乘时雅可比谱半径”的角度，解释为什么 RNN 的 $O(n)$ 路径长度会导致远距离梯度消失，而 Self-Attention 的 $O(1)$ 不会。

开放题

3. 长序列（比如 $n = 128k$）下注意力的 $O(n^2)$ 会成为瓶颈。本讲提到的 FlashAttention（ch24）改的是”IO”而不是”FLOPs”，KV Cache（ch23）改的是”重复计算”。请你想一想：如果要从根上把 $O(n^2)$ 降成线性，有哪些技术路线（稀疏注意力、线性注意力、状态空间模型）？它们各自牺牲了什么？

8. 延伸阅读 / 进一步思考

- 原论文第 3.1 节的 Table 1 与讨论：Vaswani et al.*, Attention Is All You Need,* 2017。
- 归纳偏置与数据量关系：关于”CNN/Transformer 在不同数据规模下泛化差异”的实证研究，可作为”小模型别盲目堆大架构”的依据。
- 长距离依赖的经典背景：Bengio 等人关于梯度消失与长程依赖（LSTM、GRU 的动机）的工作。
- 想继续往下挖：稀疏注意力（Longformer、BigBird）、线性注意力（Performer、Linear Transformer）、状态空间模型（Mamba）是三条不同的”把 n^2 降下来”的路线，留作第二册。

第 17 讲 损失函数：交叉熵、困惑度与 Label Smoothing

本讲问题：语言模型的损失为什么是交叉熵而不是 MSE？“困惑度”到底在量什么？Label Smoothing 凭什么能泛化变好，它又是怎么悄悄给 logits 戴了个紧箍咒的？

1. 数学工具/预备知识

（1）熵与交叉熵的定义。对一个真实分布 p 和一个模型给出的分布 q，离散熵 $H(p) = -\sum_k p_k \log p_k$，交叉熵 $H(p, q) = -\sum_k p_k \log q_k$。它们之间差一个 KL 散度：$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q)$。ch02 已经建立过这层关系，本讲把它用到语言模型上。

（2）对数似然。模型给第 t 个真实 token 分配的概率是 $q(x_t)$。看到一串文本 $x_{1:T}$，“这串文本在模型眼里有多合理”就等于它的联合概率 $\prod_t q(x_t)$。我们希望这个乘积越大越好——这就是最大似然估计（MLE）的出发点。

（3）softmax 的梯度（ch04）。对 logits $z = (z_1, \ldots, z_V)$，softmax 输出 $q_k = e^{z_k} / \sum_j e^{z_j}$。本讲最关键的一条结论会反复用到：**交叉熵对 logits 的梯度恰好是** $q - y$，即预测分布减去真实标签。我们会在 2.2 把它推出来。

（4）one-hot 向量。真实 token 是词汇表里的某一个，我们用一个长度 V 的向量 y 表示：正确位置是 1，其余全是 0。这叫 one-hot 标签。

2. 完整推导

2.1 从最大似然到交叉熵

位置 t 处，模型根据前文 $x_{<t}$ 输出一组 logits $z_{1:V}$，经 softmax 得到概率分布 $q_t = \text{softmax}(z_{1:V})$。真实 token 记为 x_t，它在词汇表里的序号对应的 one-hot 标签是 y_t。

整段文本的似然是各位置预测概率的连乘：

$$P(x_{1:T}) = \prod_{t=1}^{T} q_t(x_t \mid x_{<t})$$

直接乘很多个小于 1 的数会下溢，所以取对数（对数单调递增，最大化对数似然等价）：

$$\log P(x_{1:T}) = \sum_{t=1}^{T} \log q_t(x_t)$$

我们要最大化它，等价于最小化它的负数。再除以 T 做平均，得到标准的训练损失：

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log q_t(x_t)$$

这正是 one-hot 标签下的交叉熵。写成向量形式更清楚：因为 $q_t(x_t) = q_{t,k^*}$（k^* 是正确类），而 one-hot 标签 y_t 只在 k^* 处为 1，所以 $\sum_k y_{t,k} \log q_{t,k} = \log q_{t,k^*}$。于是：

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{V} y_{t,k} \log q_{t,k}$$

为什么不用 MSE？ MSE 假设输出是连续值、误差服从高斯分布；而这里我们要的是一个概率分布，真实标签是离散类别。交叉熵直接度量”模型把概率分配给正确类的效率”，梯度干净，且在分类问题上不会像 MSE 那样被 softmax 饱和拖慢学习。

2.2 交叉熵对 logits 的梯度：$q - y$

这是本讲最重要的一步，推出来以后很多现象都会变得一目了然。

单个样本的损失 $\mathcal{L} = -\sum_k y_k \log q_k$，而 $q_k = e^{z_k} / \sum_j e^{z_j}$。对任意一个 logit z_i 求偏导：

$$\frac{\partial \mathcal{L}}{\partial z_i} = -\sum_k y_k \frac{\partial \log q_k}{\partial z_i}$$

用 softmax 的求导公式 $\partial q_k / \partial z_i = q_k(\delta_{ki} - q_i)$（ch04 推过），即 $\partial \log q_k / \partial z_i = \delta_{ki} - q_i$。代进去：

$$\frac{\partial \mathcal{L}}{\partial z_i} = -\sum_k y_k(\delta_{ki} - q_i) = -y_i + \sum_k y_k q_i = -y_i + q_i \underbrace{\sum_k y_k}_{=1} = q_i - y_i$$

写成向量就是 $\nabla_z \mathcal{L} = q - y$。

这个结果为什么重要？ 它说明：训练时 logits 收到的”推力”正好等于”预测概率减去标签概率”。如果模型对正确类预测了 $q_{k^*} = 0.8$，而标签是 1，那这一项的梯度就是 $0.8 - 1 = -0.2$——把正确类 logit 往上推；预测错得越离谱，梯度越大，推得越狠。这个形式干净、尺度适中，是交叉熵适合梯度下降的根本原因。

2.3 困惑度：把损失翻译成”犹豫程度”

困惑度（Perplexity, PPL）定义为损失的指数：

$$\mathrm{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log q_t(x_t)\right)$$

把指数和对数交换一下（$e^{-\frac{1}{T}\sum \log q} = \prod q^{-1/T}$）：

$$\mathrm{PPL} = \left(\prod_{t=1}^{T}\frac{1}{q_t(x_t)}\right)^{1/T}$$

这是所有 token 预测概率倒数的**几何平均**。它的直观含义是：**模型在每个位置上，平均相当于从多少个等概率选项里做选择**。

- $\mathrm{PPL} = 1$：每个 token 都被预测概率 1，模型”胸有成竹”。
- $\mathrm{PPL} = V$：模型在 V 个词上均匀分布（$q_t(x_t) = 1/V$，$\mathcal{L} = \log V$），等于瞎猜。
- PPL 越小越好。

为什么用 PPL 而不是直接报 CE? 因为 CE 取了对数，读者对”损失 5”没有感觉；指数还原成”在多少个候选词之间犹豫”就直观了。而且 PPL 可以跨数据集、跨人比较——CE 依赖词表大小，PPL 不依赖。

几何意义：CE 是在”对数概率空间”里量平均距离，PPL 是把它映射回原始概率尺度。CE 每降 1，PPL 就除以 e——这是一个乘法关系，不是加法关系。

2.4 Label Smoothing：把 one-hot 变软

原始标签是纯 one-hot：正确类 $y_{k^*} = 1$，其余 0。Label Smoothing 把它改成一个软目标：

$$y'_k = (1-\epsilon)\, y_k + \frac{\epsilon}{V}$$

即正确类保留 $(1-\epsilon)$ 的概率质量，剩下 ϵ 的质量均匀撒给全部 V 个类。原论文取 $\epsilon = 0.1$。新标签仍然是个合法分布（所有分量加起来等于 1，验证一下：$(1-\epsilon)\cdot 1 + \epsilon/V \cdot V = 1$）。

带 Label Smoothing 的损失：

$$\mathcal{L}_{\mathrm{LS}} = -\sum_k y'_k \log q_k$$

2.5 为什么 Label Smoothing 能正则化 logits 尺度

先回答一个问题：交叉熵损失 $\mathcal{L} = -\sum_k y'_k \log q_k$ 关于 q 的最小值在哪？由于 $-\log$ 是严格凸函数，最小值在 $q_k = y'_k$ 处取得（用 2.2 的梯度结论：$\nabla_z \mathcal{L} = q - y'$，令它为 0 即 $q = y'$）。

也就是说，模型被推向的最优输出是”正确类给 $1-\epsilon$、其他类各给 ϵ/V “，而不是”正确类给 1、其他给 0”。

这直接掐住了 logits 的脖子。假设没有 Label Smoothing，最优解是 $q_{k^*} \to 1$，这要求正确类 logit $z_{k^*} \to +\infty$、其余 $\to -\infty$——logits 的尺度可以无限涨大。softmax 在极端 logits 下会饱和（ch04），梯度趋近 0，而且大 logit 对输入扰动极度敏感，泛化差。

加了 Label Smoothing 后，目标本身就要求 $q_{k^*} = 1-\epsilon < 1$。于是当 z_{k^*} 大到使 $q_{k^*} = 1-\epsilon$ 时，梯度 $q_{k^*} - y'_{k^*} = 0$，损失停止下降——**logits 不再被推往无穷大**。这就是它对 logits 尺度的隐式正则化：不是显式加一个权重衰减项，而是通过改变目标分布，从根上杜绝了过度自信。

几何/物理意义：one-hot 目标把模型拽向 softmax 曲面的”角落”（某个分量为 1，其余为 0），角落处梯度为零且曲面尖锐；Label Smoothing 把目标移到角落往内一点的位置，模型收敛到一个更”圆润”的点，对输入扰动更鲁棒。

2.6 边界讨论：为什么大模型后来不用了

Label Smoothing 早期（Transformer 翻译模型）用得很多。但 GPT-3 之后的很多大模型训练**关掉了它**。原因是：Label Smoothing 的本质是一种”人为注入不确定性”的正则化，它在数据少、模型容易过拟合时有用；当数据量极大、模型本身就欠拟合时，再把标签软化反而会干扰模型去拟合真实的类别结构，白白损失一点拟合能力。所以它是一个”看数据量下菜碟”的正则化开关。

3. 直觉理解

直觉：交叉熵是”猜错的代价”，猜错越离谱罚得越狠；困惑度是这个代价的指数版——代价翻一倍，犹豫的候选词数翻 e 倍。Label Smoothing 像老师改卷时说：“即使你确定答案是 A，也给 B、C、D 各留 0.1 分情面。”这么做不是纵容错误，而是防止学生把所有宝全押在一个选项上、押到最后一被反问就崩盘。它教会学生”其他选项也各有几分道理”，考试时更稳。

4. 常见误区

误区一：以为 PPL 能压到 1。PPL 的下限不是 1，而是数据本身的熵——人类写的文本本身就有随机性，同一个上文可以接很多合理的下一个词，完美模型也预测不出”唯一正确答案”。

误区二：以为 Label Smoothing 是给数据加噪声。它不是随机扰动，而是**改变了目标分布本身**——从 one-hot 变成软分布，数学上等价于一个正则项，是确定性的。

误区三：以为 CE 越小越好到无上限。CE 低到一定程度就说明模型开始”记住”训练集了，泛化未必跟着变好——这正是 Label Smoothing 要防的。

5. 代码实现

下面把 2.5 的结论亲手验证一遍：(1) 手动实现 Label Smoothing 并和官方 `cross_entropy(..., label_smoothing=...)` 对拍；(2) 验证梯度 $q - y$；(3) 看 PPL 随 CE 怎么变。

```
import torch
import torch.nn.functional as F

# ---------- (1) 手动 Label Smoothing 与官方对拍 ----------
def label_smoothed_ce(logits, target, eps=0.1, vocab_size=1000):
    # logits: (batch, V), target: (batch,)
    log_probs = F.log_softmax(logits, dim=-1)
    # 正确类那一项: -log q_{k*}
    nll_loss = -log_probs.gather(dim=-1, index=target.unsqueeze(1)).squeeze(1)
    # 所有类别平均 log prob（即把 eps 的质量均匀分给所有类）
    smooth_loss = -log_probs.mean(dim=-1)
    # 混合: (1-eps)* 正确项 + eps* 所有项平均
    loss = (1 - eps) * nll_loss + eps * smooth_loss
    return loss.mean()

torch.manual_seed(0)
logits = torch.randn(32, 1000)
target = torch.randint(0, 1000, (32,))

my_loss = label_smoothed_ce(logits, target, eps=0.1)
official_loss = F.cross_entropy(logits, target, label_smoothing=0.1)
print(f" 手动 LS loss:   {my_loss.item():.4f}")
print(f" 官方 LS loss:   {official_loss.item():.4f}")
print(f" 两者一致: {torch.allclose(my_loss, official_loss, atol=1e-5)}")

# ---------- (2) 验证梯度 = q - y ----------
z = torch.randn(5, requires_grad=True)
y_idx = torch.tensor([2])
loss = F.cross_entropy(z.unsqueeze(0), y_idx)
```

```
loss.backward()
q = F.softmax(z, dim=0).detach()
y_onehot = torch.zeros(5); y_onehot[2] = 1.0
print(f"\n梯度 q - y: {z.grad.tolist()}")
print(f"q - y 手算: {(q - y_onehot).tolist()}")
print(f" 梯度一致: {torch.allclose(z.grad, q - y_onehot, atol=1e-6)}")

# ---------- (3) PPL 与 CE 的指数关系 ----------
for ce in [6.9, 5.0, 4.0, 2.3]:
    print(f"CE={ce:.1f} -> PPL={torch.exp(torch.tensor(ce)).item():.1f}")
print(f" 均匀分布 (V=1000) 的 PPL = {torch.exp(torch.log(torch.tensor(1000.0))).item():.0f}")
```

6. 小结

1. 语言模型损失 = 对 one-hot 标签的交叉熵 = 负对数似然，它来自最大似然估计。
2. 交叉熵对 logits 的梯度恰好是 $q - y$——预测减标签，这个干净形式是它好用的根源。
3. 困惑度 $= \exp(\mathrm{CE})$，是”平均在多少个等概率候选间犹豫”，越小越好，下限是数据自身熵。
4. Label Smoothing 把 one-hot 软化为 $(1-\epsilon)$ 正确类 $+\epsilon/V$ 均匀分布，从目标分布上掐住 logits 尺度，防止过度自信。
5. Label Smoothing 是正则化开关：小数据开，超大数据可关。

7. 思考题

基础题

1. 如果把 ϵ 设成 1.0，Label Smoothing 的目标分布变成什么？模型还学得到正确类吗？

进阶题

2. CE 从 6.9 降到 4.0，PPL 从多少降到多少？请用 PPL $= \exp(\mathrm{CE})$ 的关系手算，并解释为什么说”CE 是对数尺度上的损失”。

开放题

3. 大模型（如 GPT-3、LLaMA）训练时普遍关掉了 Label Smoothing。请结合”正则化是用来对抗过拟合的”这一原理，论证：当数据量远大于模型容量时，为什么关掉它反而更好？

8. 延伸阅读 / 进一步思考

- Label Smoothing 原始动机：Szegedy et al., *Rethinking the Inception Architecture for Computer Vision*, 2016（首次系统提出）。
- 为什么大模型不用 LS：Chinchilla、LLaMA 系列技术报告里关于正则化选择的讨论。
- 温度 softmax 与蒸馏里的温度是同一个量，留到 ch26 知识蒸馏再深入。
- 想继续挖：熵正则化（entropy regularization）、focal loss 等”改损失改标签”的一族方法。

第 18 讲 学习率调度：Warmup 与衰减的设计原理

> **本讲问题**：为什么大模型训练不能从一开始就用最大学习率？那个”先慢慢升温、再慢慢降温”的曲线，背后到底在补偿什么？原论文那个带 $d_{\text{model}}^{-0.5}$ 和 min 的奇怪公式，为什么峰值恰好落在 warmup 步数上？

1. 数学工具/预备知识

（1）梯度下降的更新规则。 $\theta \leftarrow \theta - \eta\, g_t$，其中 $g_t = \nabla L(\theta_t)$ 是当前梯度，η 是学习率。η 是每步走多大。这是 ch03 的基础。

(2) 为什么"太大的 η 会发散"。把损失在当前点附近二阶泰勒展开：$L(\theta-\eta g)\approx L(\theta)-\eta g^2+\frac{1}{2}\eta^2 g^\top H g$，$H$ 是 Hessian。要让损失下降，需要 $\eta g^2>\frac{1}{2}\eta^2 g^\top H g$，即 $\eta<2/(g^\top Hg/g^2)=2/\lambda_{\max}$（$\lambda_{\max}$ 是 Hessian 最大特征值）。η 一旦超过这个上限，更新反而把损失推高——发散。

(3) 训练初期为什么特别脆弱。随机初始化的权重落在一个完全陌生的区域，此时梯度方向随机性大、量级也不稳。第 1 步如果用大 η，可能一步就把权重踢出"可学习盆地"，之后再也回不来。这就是 Warmup 要解决的问题。

(4) 平方根函数与余弦函数的形状。后面会用到 $\text{step}^{-0.5}$（随步数缓慢下降的曲线）和 $0.5(1+\cos(\pi t))$（从 1 平滑降到 0 的钟形）。记住它们"后期下降越来越慢"的特点。

2. 完整推导

2.1 Warmup 在补偿什么

训练第 1 步，权重是随机的，梯度 g_1 的方向和大小都没校准过。如果直接用最大学习率 $\eta_{\max}$，更新量 $\eta_{\max}g_1$ 可能大到把权重直接推飞。

Warmup 的做法：前 W 步让学习率从 0 **线性**升到 $\eta_{\max}$：

$$\eta_{\text{step}}=\eta_{\max}\cdot\frac{\text{step}}{W},\qquad \text{step}=1,\dots,W$$

这样开头几步更新量很小，模型有时间"先在小步幅下摸清梯度的大致方向"，等动量（ch19 的 Adam 状态）积累稳定了，再加大步幅。

2.2 原论文公式逐块拆解

原论文没有直接写 $\eta_{\max}$，而是给出一个自归一化的公式：

$$\eta_{\text{step}}=d_{\text{model}}^{-0.5}\cdot\min\left(\text{step}^{-0.5},\ \text{step}\cdot W^{-1.5}\right)$$

其中 $W=4000$ 是 warmup 步数。这个 min 套了两条曲线，我们拆开看。

第一条曲线（warmup 段）：当 $\text{step}\le W$ 时，$\text{step}\cdot W^{-1.5}$ 更小，取它。它关于 step 是**线性增长**的（step 的一次方）。

第二条曲线（衰减段）：当 $\text{step}>W$ 时，$\text{step}^{-0.5}$ 更小，取它。它关于 step 是**平方根衰减**的。

为什么峰值恰好落在 $\text{step}=W$？ 让两条曲线相等，解出交点：

$$\text{step}^{-0.5}=\text{step}\cdot W^{-1.5}\implies \text{step}^{-1.5}=W^{-1.5}\implies \text{step}=W$$

所以 min 操作自动在 $\text{step}=W$ 处"换档"：之前线性升，之后平方根降，峰值就卡在 warmup 终点。峰值处的学习率为：

$$\eta_{\text{peak}}=d_{\text{model}}^{-0.5}\cdot W^{-0.5}=\frac{1}{\sqrt{d_{\text{model}}\,W}}$$

代入原论文 $d_{\text{model}}=512, W=4000$：$\eta_{\text{peak}}=1/\sqrt{512\times4000}\approx 7.0\times10^{-4}$，和实际用的 ~$7.6\times10^{-4}$ 吻合。

为什么前面要乘 $d_{\text{model}}^{-0.5}$？ 因为注意力打分的方差和 d_k 成正比（ch08：点积方差为 d_k）。隐藏维度 d_{model} 越大，梯度的整体量级会跟着涨。用 $d_{\text{model}}^{-0.5}$ 做归一化，让"峰值学习率"和维度自动匹配——换一个 d_{model} 不用重新手调学习率。

2.3 为什么后期要衰减

训练后期，权重已经落到最优点附近。此时如果还用大学习率，每一步都在最优点两边来回跳（震荡），永远停不下来。逐步降低学习率，等于”越接近目标越小心迈步”，让模型在最优点附近精细打磨。

从 2.1 节那个二阶展开看：后期 Hessian 的条件数可能变差，需要更小的 η 才能保证单调下降。

2.4 主流调度方案对比

Warmup + 线性衰减：升到 $\eta_{\max}$ 后线性降到 0。BERT 用这个。

Warmup + 余弦衰减：升到 $\eta_{\max}$ 后，按余弦曲线降到 0：

$$\eta_{\text{step}} = \eta_{\max} \cdot \frac{1}{2}\left(1 + \cos\left(\pi \cdot \frac{\text{step} - W}{T - W}\right)\right)$$

其中 T 是总步数。验证两端：$\text{step} = W$ 时括号里是 0，$\cos 0 = 1$，$\eta = \eta_{\max}$；$\text{step} = T$ 时括号里是 1，$\cos \pi = -1$，$\eta = 0$。余弦曲线在中段下降快、后期下降慢——后期长时间维持一个很小但非零的学习率，对收敛有好处。GPT 系列用这个。

Warmup + 常数：升到 $\eta_{\max}$ 后保持不变。早期小模型用，后期不衰减容易震荡。

2.5 学习率和 batch size 的关系

实践中有一条经验法则叫**线性缩放规则**：batch size 翻 k 倍，学习率也大约翻 k 倍。道理是：batch 越大，梯度估计越准（平均了更多样本），梯度方向的噪声越小，于是可以放心迈更大的步。batch 小的时候梯度噪声大，步子迈大了会被噪声带偏。

这也是为什么”调 batch size”和”调学习率”必须联动看，不能孤立地改一个。

2.6 为什么是平方根衰减，而不是线性

原论文衰减段用的是 $\text{step}^{-0.5}$，不是线性下降。选平方根有个隐含道理：随训练推进，我们对”最优点在哪”的估计越来越准，但准得越来越慢——信息量按步数的平方根增长，相应地学习率就该按平方根下降。平方根曲线前期降得快、后期降得极慢，正好匹配”先粗调后细调”的节奏。线性衰减是匀速刹车，后期反而降得太干脆，容易在还没收敛完时就停了。

现代大模型还出现了 **WSD（Warmup-Stable-Decay）** 曲线：先 warmup，再在最大学习率上稳定很长一段，最后才短距离衰减。它服务于”先把模型能力练出来，最后再精修”的新训练范式，和传统余弦曲线的”一直降”思路不同。

3. 直觉理解

> **直觉**：学习率像开车。起步时不能一脚油门踩到底（Warmup），否则会直接冲出路面——你还没摸清这台车的脾气呢。开上高速后可以全速前进（常数段/峰值段），可以尽情跑。快到目的地了必须踩刹车（衰减），否则会冲过终点线停不下来。不同的衰减曲线就是不同的刹车脚法：线性刹车是一脚踩死，余弦刹车是先重后轻、稳稳停下。

4. 常见误区

> **误区**一：以为 Warmup 是可选的小技巧。对大模型（>1B 参数），没有 Warmup 几乎必发散——这不是调优，是保命。
>
> **误区**二：以为学习率越大收敛越快。太大直接发散，太小收敛极慢，存在一个最优区间。Warmup + 衰减就是为了让你在不同阶段待在该待的区间里。
>
> **误区**三：以为衰减到 0 才好。余弦衰减到接近 0 即可，实践中常常留一个最小学习率（比如峰值的 10%）不降到 0，完全降到 0 反而会让后期训练停滞。

5. 代码实现

下面实现两种调度并画出它们的数值（沙箱里不依赖 GUI，直接打印关键点）。注意原论文公式里 step 必须从 1 开始，否则 $\text{step}^{-0.5}$ 在 $\text{step}=0$ 处除零。

```
import numpy as np

def original_transformer_lr(step, d_model=512, warmup=4000):
    """ 原论文调度: d_model^-0.5 * min(step^-0.5, step*warmup^-1.5)"""
    step = max(step, 1)   # 防止 step=0 时除零
    return d_model ** (-0.5) * min(step ** (-0.5), step * warmup ** (-1.5))

def cosine_lr(step, max_lr=1e-3, warmup=4000, total_steps=100000):
    """Warmup + 余弦衰减"""
    if step < warmup:
        return max_lr * step / warmup          # 线性升温
    progress = (step - warmup) / (total_steps - warmup)
    return max_lr * 0.5 * (1 + np.cos(np.pi * progress))

print("=== 原论文调度 (d=512, warmup=4000) ===")
for s in [1, 1000, 4000, 10000, 50000]:
    print(f"  step={s:>6}: lr={original_transformer_lr(s):.6f}")
print("  (峰值应出现在 step=4000)")

print("\n=== 余弦调度 (max_lr=1e-3, warmup=4000, total=100000) ===")
for s in [1, 4000, 25000, 50000, 100000]:
    print(f"  step={s:>6}: lr={cosine_lr(s):.6f}")
print("  (step=4000 应等于 max_lr=0.001, step=100000 应 ≈0)")
```

6. 小结

1. Warmup 防止训练初期梯度不稳导致发散，前 W 步线性升学习率。
2. 原论文调度 $\eta = d_{\text{model}}^{-0.5}\min(\text{step}^{-0.5}, \text{step}\cdot W^{-1.5})$，min 自动在 $\text{step}=W$ 处换档，峰值 $\eta_{\text{peak}} = 1/\sqrt{d_{\text{model}}W}$。
3. $d_{\text{model}}^{-0.5}$ 把学习率和隐藏维度自动匹配，换维度不用重调。
4. 主流调度：Warmup + 线性/余弦/平方根衰减；余弦因后期降得慢更常用。
5. 学习率与 batch size 联动：batch 大可以用更大学习率。

7. 思考题

基础题

1. 如果把 warmup 步数 W 设成 0（第 1 步就用峰值学习率），训练初期最可能出现什么现象？

进阶题

2. 验证原论文公式的换档点：手算证明当 $\text{step}=W$ 时，$\text{step}^{-0.5}$ 和 $\text{step}\cdot W^{-1.5}$ 恰好相等。再算一遍 $d=512, W=4000$ 时的峰值学习率。

开放题

3. 余弦衰减后期”下降慢”，线性衰减后期”匀速下降”。请从”训练后期需要长时间精细打磨”的角度，论证为什么现代大模型更偏爱余弦衰减。如果总训练步数比预期短很多，余弦曲线会停在一个还很高的学习率上，这会带来什么问题？

8. 延伸阅读 / 进一步思考

- 原论文第 5.3 节关于优化器与学习率调度的讨论。
- 线性缩放规则：Goyal et al., *Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour*, 2017。

- 余弦衰减：Loshchilov & Hutter, *SGDR: Stochastic Gradient Descent with Warm Restarts*, 2016（余弦退火与热重启的源头）。
- 进阶：WSD（Warmup-Stable-Decay）调度，是 LLaMA-3 之后训练长上下文时兴起的新曲线，可作为进一步阅读方向。

第 19 讲 Adam 优化器：从动量到自适应学习率

本讲问题：为什么几乎所有 Transformer 都用 Adam 而不是 SGD？Adam 到底”自适”了什么？偏差修正又在修什么”偏”？AdamW 为什么非要把权重衰减从梯度里拆出来？

1. 数学工具/预备知识

（1）梯度下降。 $\theta_t = \theta_{t-1} - \eta g_t$，$g_t = \nabla L(\theta_{t-1})$。每步只看当前梯度。

（2）指数移动平均（EMA）。 $m_t = \beta m_{t-1} + (1-\beta) g_t$，$\beta \in [0, 1)$。它把最近一串梯度加权求和，越近的权重越大，越远的按几何级数衰减。可以把 m_t 理解成”最近梯度的滑动平均”。

（3）几何级数求和。 $\sum_{i=0}^{n-1} r^i = (1-r^n)/(1-r)$。本讲推偏差修正时会反复用到它。

（4）二阶矩。 v_t 跟踪的是 g_t^2 的滑动平均，它衡量”这个参数的梯度最近有多大”。v_t 大说明梯度长期偏大，v_t 小说明长期偏小。

2. 完整推导

2.1 SGD + 动量：把梯度平均掉

朴素 SGD 每步只看当前 g_t。问题是：当损失曲面狭长（一个方向陡、另一个方向缓）时，SGD 会在陡的方向上剧烈来回震荡，在缓的方向上却几乎不前进。

动量（Momentum）解决这个问题：累积历史梯度的 EMA，让更新方向沿”平均方向”走：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta m_t$$

把 m_t 展开，$m_t = (1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i} g_i$。注意震荡方向上正负梯度会互相抵消（EMA 里一加一减），而持续方向上的梯度会叠加——所以动量相当于一个”低通滤波器”，滤掉高频震荡、保留持续趋势。β_1 常取 0.9。

2.2 RMSProp：给每个参数配自己的步幅

动量只管了”方向”。RMSProp 管”每个参数走多快”。它对梯度平方做 EMA：

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}$$

v_t 是梯度平方的滑动平均。看更新量里的 $g_t/\sqrt{v_t}$：

- 如果某参数梯度一直很大，v_t 大，$g_t/\sqrt{v_t} \approx$ 个位数——步幅被压小。
- 如果某参数梯度一直很小，v_t 小，$g_t/\sqrt{v_t}$ 相对变大——步幅被放大。

这就是”自适应”：**梯度大的参数走小步，梯度小的参数走大步**，把各方向的有效步幅拉到差不多。ϵ（10^{-8}）防止除零。

2.3 Adam：动量 + 自适应合体

Adam 同时维护一阶矩（动量）和二阶矩（自适应）：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \text{（一阶矩，方向）}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad \text{（二阶矩，步幅）}$$

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

默认超参：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$。

2.4 偏差修正：为什么 $m_0 = v_0 = 0$ 会出问题

问题出在初始化：$m_0 = 0$，$v_0 = 0$。开头几步 m_t、v_t 都被零起点拖着，系统性地**偏小**。
以 m_t 为例，把递推展开成求和：

$$m_t = (1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i} g_i$$

现在假设梯度是个常数 $g_i \equiv g$（理想情况，没有噪声），那应该有 $m_t \approx g$ 才对。代进去用几何级数求和：

$$m_t = g(1-\beta_1) \sum_{i=1}^{t} \beta_1^{t-i} = g(1-\beta_1) \frac{1-\beta_1^t}{1-\beta_1} = g(1-\beta_1^t)$$

看出来了吗？当 t 很小时，$\beta_1^t \approx 1$，所以 $m_t = g(1 - \text{接近}1) \approx 0$——明明真实梯度是 g，动量却被算成接近 0，被严重低估了。v_t 同理。
修正方法就是把这个 $(1-\beta^t)$ 的折扣除掉：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

更新规则随之变成：

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

为什么 t 大了以后修正就失效了？ 因为 $t \to \infty$ 时 $\beta^t \to 0$，$1-\beta^t \to 1$，$\hat{m}_t \to m_t$——初始的零起点影响被时间冲淡了，修正项趋近于 1。所以偏差修正是个”只在前几千步起作用”的补丁。

2.5 AdamW：把权重衰减从梯度里解耦出来

原始 Adam 里，权重衰减（weight decay，即把权重往 0 拉的正则化）是直接加进梯度的：$g_t \leftarrow g_t + \lambda \theta_{t-1}$。但这样有个问题：加进去的 $\lambda\theta$ 随后又被 $\sqrt{v_t}$ 除了一遍——于是不同参数实际受到的衰减强度被自适应学习率搅乱了，正则化变得不干净。
AdamW 干脆把权重衰减从梯度更新里**解耦**出来，单独乘一个学习率：

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

其中 $\lambda\theta_{t-1}$ 是独立的衰减项，每个参数都以同样的比例 $\eta\lambda$ 向 0 收缩，不受自适应学习率干扰。这让正则化效果更干净，是现代大模型的标配。

3. 直觉理解

直觉：SGD 像蒙眼下山——每步只看脚下那一点坡度，路窄就左右横跳。动量像一颗带惯性的滚石——顺着之前滚的方向继续走，被小坑洼绊一下也不会立刻改向，震荡被惯性磨平了。RMSProp 像给不同陡度配不同步幅——陡坡走小步、缓坡走大步，把横移压下去。Adam 就是”滚石 + 自动步幅”的合体。偏差修正则是因为这颗石头起步时是静止的，得把”启动延迟”补偿掉，否则头几步石头根本不滚。

4. 常见误区

误区一：以为 Adam 不需要调学习率。Adam 只是把每个参数之间的**相对**步幅自适应了，全局学习率 η 仍然是必须手调的头号超参。

误区二：以为 Adam 总比 SGD 好。在很多 CV 任务上，SGD+ 动量泛化反而更好——因为 Adam 的自适应步幅会让模型过早收敛到”尖锐”的极小值，而 SGD 倾向于停在”平坦”的极小值，平坦点泛化更好。Transformer 用 Adam 主要是因为它收敛快、对超参不敏感。

误区三：以为 AdamW 的权重衰减就是 L2 正则化。两者在 SGD 里等价，但在 Adam 里不等价——正是这个不等价催生了 AdamW。

5. 代码实现

下面手写一个最小 AdamW，并和 PyTorch 官方实现对拍，验证前向几步的参数更新完全一致。手写版严格按上面公式实现偏差修正与解耦衰减。

```
import torch

class ManualAdamW:
    """ 严格按公式实现：一阶矩 + 二阶矩 + 偏差修正 + 解耦权重衰减"""
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.01):
        self.params = [p.clone().detach() for p in params]   # 维护一份参数副本
        self.lr = lr
        self.beta1, self.beta2 = betas
        self.eps = eps
        self.weight_decay = weight_decay
        self.t = 0
        self.m = [torch.zeros_like(p) for p in self.params]  # 一阶矩
        self.v = [torch.zeros_like(p) for p in self.params]  # 二阶矩

    def step(self, grads):
        self.t += 1
        for i, (p, g) in enumerate(zip(self.params, grads)):
            # 更新矩估计
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * g
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * g ** 2
            # 偏差修正：除以 (1 - beta^t)
            m_hat = self.m[i] / (1 - self.beta1 ** self.t)
            v_hat = self.v[i] / (1 - self.beta2 ** self.t)
            # 更新 + 解耦权重衰减：衰减项独立于自适应步幅
            p.data -= self.lr * (m_hat / (v_hat.sqrt() + self.eps) + self.weight_decay * p.data)

# ===== 对拍测试 =====
torch.manual_seed(0)
y_target = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])

# 官方 AdamW
```

```
x = torch.randn(5, requires_grad=True)
opt_official = torch.optim.AdamW([x], lr=0.01, betas=(0.9, 0.999), weight_decay=0.01)

# 手动 AdamW（从相同初值出发）
x2 = x.detach().clone().requires_grad_(True)
opt_mine = ManualAdamW([x2], lr=0.01, betas=(0.9, 0.999), weight_decay=0.01)

for step in range(5):
    # 官方：反向传播自动求梯度
    loss = ((x - y_target) ** 2).sum()
    opt_official.zero_grad(); loss.backward(); opt_official.step()
    # 手动：梯度就是 2*(当前参数 - y_target)，从【当前】参数算，不能用初值
    g = 2 * (opt_mine.params[0] - y_target)
    opt_mine.step([g])

    same = torch.allclose(x.detach(), opt_mine.params[0], atol=1e-6)
    print(f"step {step+1}: 官方 ={x.data.tolist()[:2]}  手动 ={opt_mine.params[0].tolist()[:2]}
    ↪  一致 ={same}")
```

6. 小结

1. Adam = 动量（梯度一阶矩 EMA）+ RMSProp（梯度二阶矩 EMA）：一个管方向，一个管步幅。
2. 偏差修正 $\hat{m}_t = m_t/(1-\beta_1^t)$ 补偿零初始化在前几千步造成的低估，t 大后自动失效。
3. AdamW 把权重衰减从梯度更新里解耦，正则化不受自适应步幅干扰，是现代标配。
4. 默认超参：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$；全局学习率仍需手调。

7. 思考题

基础题

1. 如果把 β_1 设成 0（不要动量），Adam 的更新退化成什么？如果把 β_2 设成 0 呢？

进阶题

2. 用几何级数求和，推导出偏差修正项 $(1-\beta_1^t)$ 的来历：假设 $g_i \equiv g$，证明 $m_t = g(1-\beta_1^t)$，并说明为什么除以 $(1-\beta_1^t)$ 就还原成了 g。

开放题

3. 为什么 SGD+ 动量在 CV 上泛化常常优于 Adam，而 Transformer 却几乎只用 Adam？请从”平坦 vs 尖锐极小值”和”收敛速度 vs 泛化”两个角度讨论。最新的某些优化器（如 Lion、Adafactor）又是在 Adam 的哪一点上做的改进？

8. 延伸阅读 / 进一步思考

- Adam 原始论文：Kingma & Ba, *Adam: A Method for Stochastic Optimization*, 2015。
- AdamW 与解耦权重衰减：Loshchilov & Hutter, *Decoupled Weight Decay Regularization*, 2017。
- Adam 泛化问题：Wilson et al., *The Marginal Value of Adaptive Gradient Methods in Machine Learning*, 2017。
- 进阶优化器：Lion（Google，用符号动量）、Adafactor（节省二阶矩显存）、Sophia（二阶信息），可作为第二册内容。

第 20 讲 初始化与梯度稳定性：Xavier 与 He 初始化推导

本讲问题：为什么权重不能全初始化为零？为什么 Xavier 要让前向和反向的方差都不变？He 初始化里那个”2”到底是从哪冒出来的？初始化选错了，网络会在第一次前向就崩。

1. 数学工具/预备知识

（1）独立随机变量乘积的方差。若 X, Y 独立、均值都是 0，则 $\mathrm{Var}(XY) = \mathrm{Var}(X)\mathrm{Var}(Y)$。推导：$E[XY] = E[X]E[Y] = 0$，所以 $\mathrm{Var}(XY) = E[X^2Y^2] = E[X^2]E[Y^2] = \mathrm{Var}(X)\mathrm{Var}(Y)$。本讲的核心就是反复用这条。

（2）独立随机变量和的方差。若各项独立，则 $\mathrm{Var}(\sum_j X_j) = \sum_j \mathrm{Var}(X_j)$——方差相加，不交叉。

（3）前向与反向传播。前向是信号从输入传到输出；反向是梯度从输出传回来（ch03）。两者都要经过同样的权重矩阵，所以方差在两个方向上都要照顾到。

（4）ReLU 的二阶矩。对零均值高斯 $X \sim \mathcal{N}(0, \sigma^2)$，$\mathrm{ReLU}(X) = \max(0, X)$。负半轴被置零，剩下正半轴。它的二阶期望 $E[\mathrm{ReLU}(X)^2] = \sigma^2/2$——正好是原方差的一半。这就是 He 初始化里那个因子的来源。

2. 完整推导

2.1 为什么不能全零初始化

如果所有权重 $W = 0$，那么同一层的所有神经元算出一模一样的输出，梯度也一模一样——无论怎么更新，这一堆神经元永远做相同的事，学不出不同特征。这叫**对称性问题**。所以初始化必须给权重加随机扰动，打破对称。

2.2 前向方差怎么一步步演化

设第 $l-1$ 层输入 $h^{l-1} \in \mathbb{R}^{n_{l-1}}$，权重矩阵 $W^l \in \mathbb{R}^{n_l \times n_{l-1}}$，每个 W^l_{ij} 独立同分布、均值 0、方差 σ_w^2。第 i 个输出神经元：

$$h_i^l = \sum_{j=1}^{n_{l-1}} W_{ij}^l h_j^{l-1}$$

假设 h_j^{l-1} 均值 0、方差 σ_h^2，且和 W 独立。先用乘积方差公式看每一项 $W_{ij}^l h_j^{l-1}$：

$$\mathrm{Var}(W_{ij}^l h_j^{l-1}) = \mathrm{Var}(W_{ij}^l)\mathrm{Var}(h_j^{l-1}) = \sigma_w^2 \sigma_h^2$$

再用和的方差公式把 n_{l-1} 项加起来：

$$\mathrm{Var}(h_i^l) = \sum_{j=1}^{n_{l-1}} \sigma_w^2 \sigma_h^2 = n_{l-1}\sigma_w^2\sigma_h^2$$

我们希望信号每穿过一层，方差既不放大也不缩小（否则叠 100 层后信号要么爆炸成无穷大、要么萎缩成 0）。令 $\mathrm{Var}(h^l) = \mathrm{Var}(h^{l-1}) = \sigma_h^2$：

$$n_{l-1}\sigma_w^2 = 1 \implies \sigma_w^2 = \frac{1}{n_{l-1}}$$

这就是**前向传播保持方差稳定**的条件。

2.3 反向传播的对称条件

梯度 $\delta^l = \partial L / \partial h^l$ 从输出往回传。反向时，梯度要乘 W^l 的转置，走的方向和前向相反。用同样的方差推导（把 n_{l-1} 换成 n_l），得到：

$$\mathrm{Var}(\delta^{l-1}) = n_l \sigma_w^2 \, \mathrm{Var}(\delta^l)$$

要让反向方差也不变（梯度不随层数消失或爆炸）：

$$n_l \sigma_w^2 = 1 \implies \sigma_w^2 = \frac{1}{n_l}$$

2.4 Xavier：前向要 $1/n_{in}$，反向要 $1/n_{out}$，取个折中

前向要求 $\sigma_w^2 = 1/n_{l-1} = 1/n_{in}$，反向要求 $\sigma_w^2 = 1/n_l = 1/n_{out}$。两个要求打架，Glorot 的办法是取平均：

$$\sigma_w^2 = \frac{2}{n_{in} + n_{out}}$$

这就是 **Xavier 初始化**（也叫 Glorot 初始化）。它假设激活函数是线性的（或近似线性的 tanh/sigmoid），让前向和反向的方差都大致稳住。

几何意义：把每层看成一个”信号放大器”。Xavier 让这个放大器的增益约等于 1——信号穿过去不增不减，梯度传回来也不增不减。

2.5 He 初始化：ReLU 悄悄吃掉了一半方差

Xavier 假设激活函数是线性的。但 ReLU 会把负值全部置零——大约一半的神经元输出变成 0。这等于悄悄把方差砍了一半。

具体说，前向传播经过 ReLU 后，方差关系变成（用 2.1 预备知识里的 $E[\mathrm{ReLU}(X)^2] = \sigma^2/2$）：

$$\mathrm{Var}(h^l) = \frac{1}{2}\, n_{l-1} \sigma_w^2\, \mathrm{Var}(h^{l-1})$$

那个 $1/2$ 就是 ReLU 置零负半轴造成的。要让方差仍然不变：

$$\frac{1}{2} n_{l-1} \sigma_w^2 = 1 \implies \sigma_w^2 = \frac{2}{n_{in}}$$

这就是 **He 初始化**（何恺明初始化）。相比 Xavier，它把方差从 $2/(n_{in} + n_{out})$ 放大成 $2/n_{in}$，多出来的那一点正好补偿 ReLU 吃掉的一半。Transformer 用 ReLU/GELU，所以对应 He 初始化。

2.6 Transformer 里的实际做法

原论文：权重矩阵 $W \sim \mathcal{N}(0, d_{\text{model}}^{-0.5})$，偏置全 0，嵌入层 $\sim \mathcal{N}(0, 1)$。

现代实践更简单：直接用 PyTorch 默认初始化，再加上 LayerNorm（ch12）。LayerNorm 会在每层主动把激活归一化回单位方差，等于给”方差漂移”上了个保险，所以对初始化的敏感度比早期网络低很多。

3. 直觉理解

> **直觉**：初始化像给一长串接力赛安排起跑。每个运动员（神经元）传棒给下一个时，不能把棒扔得太快太猛（信号爆炸），也不能软乎乎传不到（信号消失）。前向是”信号从起点传到终点”，反向是”教练的反馈从终点喊回起点”，两个方向都得保证声音不衰减。Xavier 让两个方向音量差不多；后来发现 ReLU 这个运动员有个怪习惯——一半情况下直接把棒扔了（置零），所以得让他起跑时音量加倍补偿，这就是 He。

4. 常见误区

> **误区一**：以为初始化只是个”初始值”，训久了就无所谓。糟糕的初始化可能让第一步前向就把激活打出 inf，之后再也救不回来。

误区二：以为 Xavier 和 He 可以随便用。Xavier 适合 tanh/sigmoid，He 适合 ReLU/GELU。用反了——比如对 ReLU 用 Xavier——前向方差会逐层缩小，几十层后激活全变成 0，梯度也跟着消失。

误区三：以为 LayerNorm 能完全替代好初始化。LayerNorm 缓解敏感度，但糟糕的初始化仍可能在第一层就破坏训练。

5. 代码实现

下面模拟一个 50 层的全连接网络，分别用 Xavier 和 He 初始化，打印前向传播中各层激活的方差变化。用 ReLU 激活，应该看到：He 初始化的方差基本稳住，Xavier 的方差逐层缩小。

```
import torch

def check_forward_variance(weight_std, n_in=512, n_layers=50, batch=32):
    """ 模拟前向传播，观察各层激活方差如何变化（带 ReLU） """
    x = torch.randn(batch, n_in)
    vars_list = [x.var().item()]
    for _ in range(n_layers):
        W = torch.randn(n_in, n_in) * weight_std   # 固定标准差的权重
        x = torch.relu(x @ W)                       # 线性层 + ReLU
        vars_list.append(x.var().item())
    return vars_list

# Xavier: 假设线性激活，对 ReLU 来说标准差偏小
std_xavier = (2 / (512 + 512)) ** 0.5
vars_xavier = check_forward_variance(std_xavier)

# He: 补偿了 ReLU 置零一半，标准差应保持方差稳定
std_he = (2 / 512) ** 0.5
vars_he = check_forward_variance(std_he)

print(f"Xavier std={std_xavier:.4f} (对 ReLU 偏小):")
print(f"  第 1 层 ={vars_xavier[0]:.3f}  第 10 层 ={vars_xavier[10]:.4f}  第 50 层
 ↪  ={vars_xavier[50]:.6f}")

print(f"\nHe std={std_he:.4f} (补偿 ReLU):")
print(f"  第 1 层 ={vars_he[0]:.3f}  第 10 层 ={vars_he[10]:.4f}  第 50 层 ={vars_he[50]:.4f}")
print("\n预期：He 的方差基本稳住；Xavier 的方差逐层萎缩到接近 0。")
```

6. 小结

1. 全零初始化导致对称性问题，必须用随机权重打破对称。
2. Xavier：$\sigma_w^2 = 2/(n_{in} + n_{out})$，同时照顾前向和反向方差，适合 tanh/sigmoid。
3. He：$\sigma_w^2 = 2/n_{in}$，多出来的因子补偿 ReLU/GELU 置零负半轴损失的一半方差。
4. 选错初始化会让前向信号爆炸或消失；LayerNorm 能缓解但不能替代好初始化。

7. 思考题

基础题

1. 如果对一个用 sigmoid 激活的网络错误地用了 He 初始化（方差偏大），前向传播的激活会发生什么？

进阶题

2. 用 $E[\text{ReLU}(X)^2] = \sigma^2/2$ 这一条，重新推导为什么 He 初始化的方差因子是 $2/n_{in}$ 而不是 $1/n_{in}$。GELU 也大约置零一半，它能用 He 初始化吗？

开放题

3. LayerNorm 在每层主动把激活拉回单位方差。请论证：为什么有了 LayerNorm 之后，现代 Transformer 对初始化的敏感度比早期 VGG/ResNet 低得多？这是否意味着初始化可以随便选？

8. 延伸阅读 / 进一步思考

- Xavier 原始推导：Glorot & Bengio, *Understanding the Difficulty of Training Deep Feedforward Neural Networks*, 2010。
- He 初始化：He et al., *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, 2015。
- 进阶：针对深层 Transformer 的初始化技巧（如缩小残差分支初始化、GPT-2 的缩放初始化），以及 LayerScale 等，可作为进一步阅读。

第 21 讲 混合精度训练：FP16 与 BF16 的数学与工程

本讲问题：为什么大模型训练不直接用 FP32？FP16 的动态范围不够会发生什么灾难？Loss Scaling 是怎么把下溢的梯度救回来的？BF16 又凭什么天生不需要这个补丁？

1. 数学工具/预备知识

（1）浮点数的三段式结构。一个浮点数拆成三部分：符号位 s（正负）、指数位 e（数量级）、尾数位 m（有效数字）。它表示的值约等于 $(-1)^s \times 2^{e-\text{bias}} \times (1.m)$。指数位决定”能表示多大或多小的数”（动态范围），尾数位决定”能分辨多精细的差别”（精度）。这是理解本讲的钥匙。

（2）动态范围。就是这个格式能表示的最大数除以最小正常值。指数位越多，动态范围越大。

（3）下溢（underflow）与上溢（overflow）。一个数小到比最小正常值还小，就被舍成 0，叫下溢；大到比最大还大，就变成 inf，叫上溢。两者都会让训练崩溃。

（4）链式法则中的梯度量级。反向传播时梯度往往是 10^{-3} 到 10^{-7} 量级的小数，正好踩在 FP16 的下溢边缘。

2. 完整推导

2.1 三种浮点格式的位分配

FP32（单精度）：1 符号 + 8 指数 + 23 尾数。- 动态范围：$\approx \pm 3.4 \times 10^{38}$ - 精度：约 7 位十进制有效数字。

FP16（半精度）：1 符号 + 5 指数 + 10 尾数。- 指数只有 5 位，bias $= 15$，所以最小正常值 $2^{-14} \approx 6 \times 10^{-5}$，最大约 65504。- 精度：约 3.3 位十进制有效数字。

BF16（bfloat16）：1 符号 + 8 指数 + 7 尾数。- 指数位和 FP32 一样是 8 位，所以**动态范围和 FP32 相同** $\approx \pm 3.4 \times 10^{38}$。- 但尾数只有 7 位，精度约 2.4 位十进制有效数字。

把三者放一起看，关键区别一目了然：

格式	指数位	尾数位	动态范围	精度
FP32	8	23	大	高
FP16	5	10	**小**	较高
BF16	8	7	大（同 FP32）	低

FP16 是”精度高但范围窄”，BF16 是”范围大但精度粗”。为什么要有两个半精度？因为它们解决不同的痛点。

2.2 FP16 的致命伤：梯度下溢

反向传播时，梯度通常很小（10^{-3} 到 10^{-7}）。FP16 的最小正常值约 6×10^{-5}，比这个更小的数一存就变成 0。

一旦某个参数的梯度被下溢成 0，它就再也收不到任何更新信号——这个参数”死”了。训练几百步后，一大批参数集体死亡，损失就卡在一个烂位置不动了。这就是早期 FP16 训练崩溃的原因。

2.3 Loss Scaling：先放大再除回去

解决思路很直接：既然梯度太小存不下，那就先把损失乘上一个大的 scale s。由链式法则，反向传播时梯度也跟着放大 s 倍：

$$\nabla(sL) = s\,\nabla L$$

放大后的 $s\nabla L$ 落进 FP16 能表示的范围，不会下溢。等更新参数时再把 s 除掉：

$$\theta \leftarrow \theta - \eta\frac{s\nabla L}{s} = \theta - \eta\nabla L$$

净效果和不放大完全一样——只是在”中间搬运梯度”这段时间里把数字放大到尺子能量的范围内。实际上 Adam 的状态更新仍用 FP32 做，所以只要梯度在反传途中别下溢就行。

实践中 scale 不是固定的，而是”动态调整”：如果这一步出现了 inf（上溢），就把 scale 砍半重试；连续几百步没出 inf，就把 scale 翻倍。这叫动态 Loss Scaling。

2.4 BF16 为什么天生不需要 Loss Scaling

BF16 的指数位和 FP32 一样是 8 位，最小正常值和 FP32 一样小（约 10^{-38}）。所以哪怕梯度小到 10^{-7}，BF16 也能正常存下，根本不会下溢——自然就不需要 Loss Scaling。代价是尾数只有 7 位，精度比 FP16 粗，但对训练来说”范围够”比”精度细”更重要。这就是为什么 Hopper 之后的 GPU 上大家直接用 BF16。

2.5 Master Weights：为什么不能全用半精度

即使前向和反向都用半精度算，**权重本身始终保留一份 FP32 副本**（master weights）。流程是：

1. 用半精度梯度去更新 FP32 的 master weights（累积更新在高精度账本上做）。
2. 前向时把 master weights 临时转成半精度喂进计算。

如果直接在半精度上累积更新，每次小更新都会被尾数舍入吃掉，几百步后参数就飘了。master weights 相当于一个精确的总账本，平时草稿纸用半精度算，正式记账永远用 FP32。

2.6 边界：FP8 的下一步

更新的硬件开始支持 FP8（E4M3/E5M2），进一步把计算压到 8 位。它延续了 BF16 的思路：优先保动态范围，再谈精度。这是混合精度这条工程路线的自然延伸。

2.7 那为什么不干脆全用 FP32

有人会问：既然半精度有这么多坑（下溢、要 master weights、要 loss scaling），为什么不全程 FP32？答案是性价比。FP32 训练：显存占一倍，Tensor Core 不加速，大模型根本塞不进去、也跑不快。混合精度用一点点数值上的小心思（loss scaling、master weights），换来显存减半、Tensor Core 加速两到三倍——这是训练十亿到万亿参数模型的必经之路。BF16 的出现更是把这套小心思里最麻烦的 loss scaling 也省了，所以现代大模型几乎清一色 BF16 起步。

3. 直觉理解

> **直觉**：FP32 是一把刻度精细、能量程大的标准卷尺。FP16 是一把短尺——刻度细但量不到很大或很小的数，量蚂蚁（小梯度）就出界了。BF16 是一把粗尺——能量程和卷尺一样长，但刻度稀疏。Loss Scaling 像把蚂蚁放到显微镜下放大一千倍量，量完再除回去。Master Weights 像公司的总账本——平时用草稿纸（半精度）快速算流水，但月底结账永远用精确的大账本（FP32），不然几轮舍入误差就把账做歪了。

4. 常见误区

误区一：以为混合精度就是”全部用 FP16”。实际上 master weights 永远是 FP32，只有前向/反向的计算激活和梯度用半精度。

误区二：以为 BF16 和 FP16 精度一样。BF16 尾数只有 7 位，精度比 FP16（10 位）粗，但动态范围大得多——这是用精度换范围，是划算的取舍。

误区三：以为半精度一定更快。半精度快主要因为两点：显存带宽省一半、Tensor Core 对半精度有专用硬件加速。在没有 Tensor Core 的硬件上，半精度可能并不快。

5. 代码实现

下面亲手复现 FP16 下溢：一个很小的梯度在 FP16 里变成 0，乘上 scale 后又能存下来，再除回去正好恢复。BF16 则直接就正常。

```
import torch

# ---------- 正常 FP32 梯度 ----------
x = torch.tensor(1.0, requires_grad=True)
y = (x * 1e-4) ** 2
y.backward()
print(f"FP32 梯度: {x.grad.item():.10f}")

# ---------- FP16: 梯度下溢成 0 ----------
x16 = torch.tensor(1.0, dtype=torch.float16, requires_grad=True)
y16 = (x16 * 1e-4) ** 2
y16.backward()
print(f"FP16 梯度 (下溢为 0): {x16.grad.item():.10f}")

# ---------- Loss Scaling 救回来 ----------
scale = 1000.0
x16s = torch.tensor(1.0, dtype=torch.float16, requires_grad=True)
y16s = ((x16s * 1e-4) ** 2) * scale   # 损失放大 scale 倍
y16s.backward()
print(f"FP16 + LossScaling 梯度 (放大{scale}x): {x16s.grad.item():.10f}")
# 注意: 除回 scale 必须在 FP32 master weights 上做, 不能在 FP16 里除 (否则又下溢)
recovered = x16s.grad.float() / scale
print(f"FP32 里除以 scale 后: {recovered.item():.10f}  (应 ≈FP32 的 {x.grad.item():.2e})")

# ---------- BF16: 不需要 scaling 也能存下小梯度 ----------
xb = torch.tensor(1.0, dtype=torch.bfloat16, requires_grad=True)
yb = (xb * 1e-4) ** 2
yb.backward()
print(f"\nBF16 梯度 (无需 scaling): {xb.grad.item():.10f}")
```

6. 小结

1. FP16 精度高但动态范围小（最小 6×10^{-5}），BF16 动态范围和 FP32 一样但精度粗。
2. FP16 训练必须做动态 Loss Scaling 防止小梯度下溢为 0，BF16 不需要。
3. Master Weights 始终保留 FP32 副本，保证参数累积更新不被舍入误差带偏。
4. 混合精度 = 计算用低精度 + 权重存储用高精度。

7. 思考题

基础题

1. FP16 的最大正常值是 65504。如果前向传播中某个激活值超过了 65504，会发生什么？（提示：上溢。）

进阶题

2. 从指数位数的角度解释：为什么 BF16 的最小正常值和 FP32 一样小，而 FP16 的最小正常值比它们大 10^{30} 倍？这直接解释了为什么 BF16 不需要 Loss Scaling。

开放题

3. Loss Scaling 是一个”救急”补丁。既然 BF16 不需要它，为什么早期还发明了 FP16 + Loss Scaling 这套复杂方案？请从硬件支持的时间线角度讨论：如果一上来就有 BF16，这段工程史会被删掉吗？

8. 延伸阅读 / 进一步思考

- 混合精度训练经典论文：Micikevicius et al., *Mixed Precision Training,* 2018（提出 Loss Scaling）。
- BF16 与 FP8：NVIDIA Hopper/Blackwell 架构的 FP8 支持，及其动态缩放（dynamic scaling）。
- 进阶：FlashAttention 2/3 对半精度的友好、分布式训练中的梯度通信压缩，可作为系统方向的延伸。

第 22 讲 RoPE 旋转位置编码：复数域的优雅推导

本讲问题：为什么现代大模型（LLaMA、Qwen 等）都抛弃了正余弦位置编码，改用 RoPE？“旋转”到底比”加法”好在哪？为什么它能让注意力分数只取决于相对位置？

1. 数学工具/预备知识

（1）复数乘法 = 旋转。一个复数 $z = re^{i\theta}$。把它乘上 $e^{i\phi}$，得到 $re^{i(\theta+\phi)}$——模长 r 不变，角度加了 ϕ。也就是说，**乘 $e^{i\phi}$ 等价于把这个复数在复平面上逆时针旋转 ϕ**。这是 RoPE 的全部几何基础。

（2）二维旋转矩阵。旋转角度 θ 的实矩阵形式是：

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

它作用在二维向量 $\begin{bmatrix} x \\ y \end{bmatrix}$ 上，正好和复数乘 $e^{i\theta}$ 一一对应。

（3）内积与共轭。两个复数的内积取实部：$\langle z, w \rangle = \mathrm{Re}(z\overline{w})$，其中 $\overline{w}$ 是共轭。共轭的作用是把指数上的符号翻号。

（4）ch11 的正余弦编码。它是把一个位置向量**加**到输入嵌入上，属于绝对位置编码。本讲要讲的 RoPE 是另一条路。

2. 完整推导

2.1 绝对位置编码的痛点

ch11 的正余弦编码是把位置信息**加在输入上**：$x_t + p_t$。问题在于：这是绝对位置——每个位置有一个固定的 p_t。当序列长度超过训练时见过的最大长度，模型就要用到从没见过的 p_t，外推出去完全失真，性能断崖式下跌。

2.2 RoPE 的核心思想

RoPE（Rotary Position Embedding）换了个思路：**不把位置加在输入上，而是在算注意力分数之前，把 Query 和 Key 各旋转一个和位置有关的角度**。这样点积的结果会自动只依赖相对位置。

把 d_k 维的 Q/K 向量两两分组，每组 2 维看成一个复数。比如 $q = (q_1, q_2, q_3, q_4, \dots)$ 就拆成 $(q_1 + iq_2,\ q_3 + iq_4,\ \dots)$。对位置 t 的 query，给每个复数乘上 $e^{it\theta_i}$（θ_i 是第 i 组的旋转频率）：

$$q'_t = q_t \cdot e^{it\theta_i}$$

同理，位置 s 的 key 变成 $k'_s = k_s \cdot e^{is\theta_i}$。

2.3 为什么点积只依赖相对位置

现在算旋转后两者的内积。用复数内积公式 $\langle z, w\rangle = \mathrm{Re}(z\overline{w})$：

$$\langle q'_t, k'_s\rangle = \mathrm{Re}\left(q_t e^{it\theta_i}\cdot\overline{k_s e^{is\theta_i}}\right)$$

关键一步：共轭把指数上的符号翻号，$\overline{e^{is\theta_i}} = e^{-is\theta_i}$。所以：

$$\langle q'_t, k'_s\rangle = \mathrm{Re}\left(q_t\overline{k_s}\cdot e^{i(t-s)\theta_i}\right)$$

看指数：$it\theta_i - is\theta_i = i(t-s)\theta_i$——**绝对位置 t 和 s 单独出现的地方全消掉了，只剩它们的差 $(t-s)$**。这就是 RoPE 最漂亮的性质：注意力分数只取决于相对距离 $t-s$，和它们站在哪里无关。

几何意义：把 Q 和 K 都看成复平面上的箭头。位置 t 的 query 箭头转了 $t\theta$，位置 s 的 key 箭头转了 $s\theta$。两个箭头之间的夹角差就是 $(t-s)\theta$——只跟”隔了多远”有关，跟”绝对站在哪”无关。所以模型天生就知道”这两个词隔了几个位置”。

2.4 用实矩阵实现：二维分块旋转

复数乘 $e^{it\theta_i}$ 对应一个二维旋转矩阵。对第 i 组维度 $(2i, 2i+1)$ 施加：

$$\begin{bmatrix}\cos(t\theta_i) & -\sin(t\theta_i)\\ \sin(t\theta_i) & \cos(t\theta_i)\end{bmatrix}\begin{bmatrix}q_{2i}\\ q_{2i+1}\end{bmatrix}$$

不同组用不同频率 θ_i：

$$\theta_i = 10000^{-2i/d_k}$$

低频组（θ_i 小）转得慢，负责”长距离粗粒度”的位置感知；高频组（θ_i 大）转得快，负责”短距离细粒度”。这套频率安排和 ch11 的正余弦编码同源。

2.5 为什么只转 Q 和 K，不转 V

注意力分数 $q^\top k$ 只涉及 Q 和 K，所以位置信息只需要进 Q、K。V 不参与点积——它只是被 softmax 出来的权重加权求和。如果也转 V，反而会给输出强行加一个和位置有关的旋转，没有任何好处，还破坏了”V 是内容本身”的纯粹性。

2.6 外推：好是好，但不是无限好

RoPE 的相对位置性质让它在一定长度范围内的外推能力远强于绝对编码。但当序列远超训练长度时仍会退化：高频组的旋转角度积累过大，不同相对位置的角度差变得难以区分。

现代做法：调整频率基来延长外推，如 NTK-aware scaling、YaRN。它们的思路都是”重新分配各频率组的角度预算”。

3. 直觉理解

> **直觉**：正余弦编码像给每个 token 发一张写着绝对门牌号的标签——标签是死的，一旦出了熟悉的街区，门牌号就对不上了。RoPE 像让每个 token 站在转盘上，按自己的位置转一个角度——两个 token 之间的夹角差只取决于它们隔了多远，不取决于它们站在操场的哪个角落。不管队伍拉多长，相对关系始终成立。

4. 常见误区

误区一：以为 RoPE 是加在输入上的。RoPE 是**乘在 Q/K 上**的——在注意力打分之前旋转它们，而不是在嵌入层加位置向量。

误区二：以为 RoPE 完美解决了外推。它只是比绝对编码外推好，超长序列（远超训练长度）仍然需要 NTK、YaRN 等额外缩放。

误区三：以为相对位置性质意味着模型不在乎绝对位置。其实通过”相对距离的累积”，模型仍能间接感知大致位置，只是不显式编码。

5. 代码实现

下面预计算 RoPE 频率表并实现旋转，然后**亲手验证**那个最关键的性质：同一对 Q/K 向量，放在 (10,12) 和 (50,52) 两处（相对距离都是 2），旋转后的点积应该相等。

```
import torch

def precompute_rope_freqs(d_k, max_len, theta_base=10000.0):
    """ 预计算每组维度的 cos/sin 频率表"""
    # 第 i 组的频率 theta_i = base^(-2i/d_k)
    freqs = 1.0 / (theta_base ** (torch.arange(0, d_k, 2).float() / d_k))
    positions = torch.arange(max_len).float()
    # 外积: positions[i] * freqs[j] -> (max_len, d_k//2)
    angles = torch.outer(positions, freqs)
    return torch.cos(angles), torch.sin(angles)

def apply_rope(x, cos_freqs, sin_freqs):
    """x: (batch, n_heads, seq_len, d_k)。把相邻两维当成一组做旋转"""
    x1 = x[..., 0::2]   # 偶数维: q_{2i}
    x2 = x[..., 1::2]   # 奇数维: q_{2i+1}
    # cos/sin: (seq_len, d_k//2) -> 广播到 (1,1,seq_len,d_k//2)
    cos = cos_freqs.unsqueeze(0).unsqueeze(0)
    sin = sin_freqs.unsqueeze(0).unsqueeze(0)
    # 旋转矩阵作用: [c -s; s c] @ [x1; x2]
    rot1 = x1 * cos - x2 * sin
    rot2 = x1 * sin + x2 * cos
    # 交错拼回原来的 (..., d_k) 布局
    return torch.stack([rot1, rot2], dim=-1).flatten(-2)

# ===== 验证：点积只依赖相对位置 =====
torch.manual_seed(0)
d_k = 8
cos_f, sin_f = precompute_rope_freqs(d_k, max_len=100)

q = torch.randn(1, 1, 1, d_k)   # 同一个 query 向量
k = torch.randn(1, 1, 1, d_k)  # 同一个 key 向量

# 情形 A: q 在位置 10, k 在位置 12, 相对距离 = 2
qA = apply_rope(q, cos_f[10:11], sin_f[10:11])
kA = apply_rope(k, cos_f[12:13], sin_f[12:13])
dotA = (qA * kA).sum().item()

# 情形 B: 同样的 q/k 向量, 放到位置 50 和 52, 相对距离仍 = 2
qB = apply_rope(q, cos_f[50:51], sin_f[50:51])
kB = apply_rope(k, cos_f[52:53], sin_f[52:53])
dotB = (qB * kB).sum().item()

print(f" 相对距离 =2, 位置 (10,12) 点积: {dotA:.6f}")
print(f" 相对距离 =2, 位置 (50,52) 点积: {dotB:.6f}")
print(f" 两者相等（相对位置性质）: {abs(dotA - dotB) < 1e-4}")
```

6. 小结

1. RoPE 在注意力打分前对 Q/K 施加与位置相关的旋转，使点积只依赖相对位置 $t-s$。
2. 旋转由二维旋转矩阵实现，不同维度组对应不同频率 $\theta_i = 10000^{-2i/d_k}$。
3. 只转 Q/K 不转 V，因为位置信息只需要进入点积。
4. 相比绝对位置编码，RoPE 外推能力更强；超长序列仍需 NTK、YaRN 等额外缩放。

7. 思考题

基础题

1. 为什么 RoPE 只旋转 Q 和 K，不旋转 V?（提示：V 进不进点积。）

进阶题

2. 当相对位置 $t-s=0$ 时，旋转角差为 0，RoPE 点积等于什么？这个结果说明位置 t 的 query 对”它自己”的打分有什么特点?

开放题

3. 如果把频率基从 10000 改成 1000000，高频组的频率会怎么变？这对”短距离敏感度”和”长距离外推能力”分别有什么影响？NTK、YaRN 又是怎么动态调整这个频率基的?

8. 延伸阅读 / 进一步思考

- RoPE 原始论文: Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021。
- 外推扩展：NTK-aware scaling、YaRN（Peng et al., 2023）、Position Interpolation。
- 其他位置编码路线对比：ALiBi（直接给注意力分数加一个和距离成反比的偏置），可作为对比阅读。

第 23 讲 KV Cache：自回归推理的加速本质

> **本讲问题**：为什么大模型推理不能像训练那样一次并行算完？KV Cache 到底缓存了什么？它把推理复杂度从什么量级降到了什么量级？又为什么省了计算却吃显存?

1. 数学工具/预备知识

（1）自回归解码。Decoder 一次只生成一个 token，然后把它接回输入再生成下一个（ch10）。位置 t 的输出只依赖位置 $1:t$。

（2）K/V 的计算只依赖自己。$K_t = W_K x_t,\ V_t = W_V x_t$。注意 K_t, V_t 只由第 t 个 token 自己的嵌入决定，和其他位置的 Query 无关——这是 KV Cache 能成立的根本原因。

（3）复杂度求和。本讲会用到 $\sum_{t=1}^{n} t = n(n+1)/2 \approx n^2/2$。

（4）显存 ≈ 参数量 × 字节数。一个 FP16 数占 2 字节，FP32 占 4 字节。

2. 完整推导

2.1 自回归解码为什么是 $O(n^2)$

生成第 t 个新 token 时，模型的输入是前面全部 $x_1, \dots, x_t$。如果每生成一个新 token 都把整个序列从头到尾重新算一遍:

- 第 1 步算长度 1 的序列,
- 第 2 步算长度 2 的序列,
- $\cdots$
- 第 n 步算长度 n 的序列。

关键观察：**生成第 t 步时，前 $t-1$ 个 token 的 K 和 V，和上一步（生成第 $t-1$ 步时）算出来的完全一样**。因为 $K_i = W_K x_i$ 只依赖 x_i 自己，而 $x_{1:t-1}$ 这一步和上一步没有任何变化。每次重算纯粹是重复劳动。

2.2 FLOPs 定量对比

无 Cache：第 t 步要处理长度 t 的序列，投影部分代价约 $O(td^2)$。把 n 步加起来：

$$\sum_{t=1}^{n} O(td^2) = O(d^2) \sum_{t=1}^{n} t = O(n^2 d^2)$$

这是关于 n 的二次方。

有 Cache：每一步只需要对**新 token** 算一次自己的 Q/K/V（代价 $O(d^2)$），然后拿新 Q 去和**已缓存的全部 K/V** 做注意力（代价 $O(td)$）。每步：

$$\text{每步 FLOPs} = O(d^2 + td)$$

总 FLOPs：

$$\sum_{t=1}^{n} O(d^2 + td) = O(nd^2 + d \sum_{t=1}^{n} t) = O(nd^2 + n^2 d)$$

对比一下：无 Cache 是 $O(n^2 d^2)$，有 Cache 是 $O(nd^2 + n^2 d)$。当 d 很大时，nd^2 项主导，复杂度从关于 n 的二次（$n^2 d^2$）降到了**关于 n 近似线性**（nd^2）。这就是为什么 KV Cache 是推理加速的第一功臣。

2.3 显存占用推导

KV Cache 要把所有历史 token 的 K 和 V 存下来。设 L 层、h 个头、头维度 $d_k = d/h$、序列长度 n。每层要存：

$$\text{每层 KV} = 2 \times h \times d_k \times n = 2 \times d \times n$$

（前面的 2 是 K、V 两份；$h \times d_k = d$ 是整层的宽度。）L 层全部：

$$\text{KV Cache 大小} = 2Ldn \times \text{bytes}$$

用 FP16 存（每个数 2 字节）：

$$\text{显存} = 2Ldn \times 2 = 4Ldn \text{ bytes}$$

代入 LLaMA-7B（$L = 32, d = 4096$），序列长度 2048：

$$\text{显存} = 4 \times 32 \times 4096 \times 2048 \approx 1.07 \times 10^9 \text{ bytes} \approx 1 \text{ GB}$$

序列拉到 32k，这一个数就涨 16 倍——KV Cache 是长上下文推理的显存大头。

2.4 Memory-Bound 与 MQA/GQA

推理时瓶颈往往不是计算（Compute-Bound），而是**显存带宽**：每生成一个 token，都要把整份 KV Cache 从显存搬到计算单元读一遍。KV Cache 越大，搬得越慢。这就是 Memory-Bound。

既然 KV Cache 吃显存又吃带宽，就有了两个省它的办法：

- **MQA（Multi-Query Attention）**：所有头共享同一份 K/V。KV Cache 直接缩小 h 倍，但表达力略降。
- **GQA（Grouped-Query Attention）**：把头分成若干组，组内共享 K/V，是 MQA 和普通 MHA 的折中。LLaMA-2 70B、LLaMA-3 都用 GQA。

3. 直觉理解

直觉：KV Cache 像上课做笔记。老师每讲一段，你就把这段的要点（K/V）记在本子上。下次写新段落时，不用把之前讲过的内容重新听一遍，直接翻笔记（缓存的 K/V）就行。没有 KV Cache，相当于每写一个字都把整堂课的录音重放一遍——蠢得要死。MQA/GQA 则是全班共用一份笔记：省纸，但笔记记得没那么细。

4. 常见误区

误区一：以为 KV Cache 缓存的是注意力输出。它缓存的是每层的 K 和 V——注意力输出是每次拿新 Q 和缓存 K/V 实时现算的。

误区二：以为 KV Cache 只省时间不占空间。它把计算量降下来了，但代价是显存——长序列时 KV Cache 可能比模型权重本身还占地方。

误区三：以为训练时也该开 KV Cache。训练时一次并行看到全部 token，每个位置的 Q/K/V 都要算，没有”历史”可缓存——缓存是为自回归的逐 token 生成服务的。

5. 代码实现

下面模拟逐 token 生成：先算前 3 个 token 的 K/V 存进缓存，再生成第 4 个 token 时直接复用缓存。最后验证：用缓存算出来的第 4 个 token 输出，和”把前 4 个 token 从头全量算一遍”的结果完全一致。

```
import torch
import torch.nn.functional as F

def full_attention(Q, K, V):
    """ 不带缓存: 一次性算完整序列的注意力"""
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / (d_k ** 0.5)
    return F.softmax(scores, dim=-1) @ V

def one_step_with_cache(q_new, k_cache, v_cache, k_new, v_new):
    """ 带缓存: 新 token 的 K/V 拼到缓存末尾, 再算注意力"""
    K = torch.cat([k_cache, k_new], dim=1)   # 历史 K + 新 K
    V = torch.cat([v_cache, v_new], dim=1)  # 历史 V + 新 V
    out = full_attention(q_new, K, V)
    return out, K, V

torch.manual_seed(0)
d = 64
x = torch.randn(1, 5, d)   # 模拟 5 个 token 的嵌入

# 第一步: 处理前 3 个 token, 它们的 K/V 进缓存
Q0 = x[:, :3, :]
K0, V0 = x[:, :3, :], x[:, :3, :]
_ = full_attention(Q0, K0, V0)
print(" 前 3 个 token 的 K/V 缓存 shape:", K0.shape)

# 第二步: 生成第 4 个 token, 复用缓存
q_new = x[:, 3:4, :]
k_new, v_new = x[:, 3:4, :], x[:, 3:4, :]
out4, K_upd, V_upd = one_step_with_cache(q_new, K0, V0, k_new, v_new)
print(" 第 4 个 token 输出 shape:", out4.shape)
print(" 更新后 K 缓存 shape:", K_upd.shape, "(应为 (1,4,d))")

# 验证: 缓存法 == 从头全量算
out_full = full_attention(x[:, :4, :], x[:, :4, :], x[:, :4, :])
match = torch.allclose(out_full[:, 3:4, :], out4, atol=1e-5)
print(" 缓存结果与全量计算一致:", match)
```

6. 小结

1. KV Cache 缓存每层历史 token 的 K/V，避免自回归生成时重复计算。
2. 无 Cache 推理 $O(n^2d^2)$，有 Cache 降到 $O(nd^2 + n^2d)$，约等于把 n 从二次变线性。
3. KV Cache 显存约 $4Ldn$ 字节（FP16），长序列时很大，是推理的显存/带宽瓶颈。
4. MQA/GQA 通过共享 K/V 缩小缓存，是推理优化的关键。

7. 思考题

基础题

1. batch size=1、序列长度 4096、LLaMA-7B（$L = 32, d = 4096$）的 KV Cache 用 FP16 存约多大？用本讲公式 $4Ldn$ 算一遍。

进阶题

2. MQA 把所有头的 K/V 合并成一份。它把 KV Cache 缩小了几倍？这样做会损失什么表达能力？为什么 GQA 是折中？

开放题

3. KV Cache 推理是 Memory-Bound 的——瓶颈在带宽不在算力。vLLM 的 PagedAttention 把 KV Cache 像操作系统虚拟内存一样分页管理，解决了什么问题？请从”显存碎片”的角度想一想。

8. 延伸阅读 / 进一步思考

- KV Cache 与推理优化：PagedAttention（Kwon et al.*, Efficient Memory Management for Large Language Model Serving with PagedAttention,* SOSP 2023）。
- MQA：Shazeer*, Fast Transformer Decoding: One Write-Head is All You Need,* 2019；GQA：Ainslie et al.*,* 2023。
- 进阶：连续批处理（continuous batching）、投机采样（speculative decoding），是推理系统方向的延伸。

第 24 讲 FlashAttention：IO 感知的精确注意力

> **本讲问题**：注意力的 FLOPs 明明那么多，为什么 GPU 上真正的瓶颈不是算不动，而是”搬不动数据”？FlashAttention 凭什么不改一个 FLOPs、不做任何近似，却能把注意力提速好几倍？

1. 数学工具/预备知识

（1）GPU 内存层级。GPU 里有两种存储：HBM（显存，容量大但带宽低，相当于冰箱）和 SRAM（片上缓存，容量小但带宽极高，相当于灶台）。计算单元（SM）只能直接用 SRAM 里的数据。

（2）Compute-Bound vs Memory-Bound。如果时间主要花在计算上，叫 Compute-Bound；如果主要花在把数据从 HBM 搬到 SRAM，叫 Memory-Bound。现代 GPU 的算力增长远快于显存带宽增长，所以大矩阵操作常常是 Memory-Bound。

（3）Online Softmax。标准 softmax 需要先看到全部分数才能归一化。但其实可以边看边维护一个”当前最大值”和”修正后的指数和”，逐块累加，最后归一化。这是 FlashAttention 的数学核心。

（4）为什么 softmax 要减最大值。直接算 $\exp(s)$ 容易上溢，减去当前最大值 $m = \max s$ 后，$\exp(s-m) \le 1$，数值稳定。

2. 完整推导

2.1 标准注意力为什么是 Memory-Bound

标准注意力流程在 HBM 和 SRAM 之间反复搬数据：

1. 从 HBM 读 Q,K,V 进 SRAM；
2. 算 $S = QK^\top$，把 S 写回 HBM；

3. 从 HBM 读回 S，算 softmax 得 A，把 A 写回 HBM；
4. 从 HBM 读回 A 和 V，算 AV，写回 HBM。

中间矩阵 S 和 A 都是 $n \times n$ 的。当序列很长时，这两个矩阵大到装不进 SRAM，只能在 HBM 里存着，一来一回搬好几趟。算力其实没占满，时间全耗在搬数据上——这就是 Memory-Bound。

2.2 标准注意力的 IO 复杂度

设 SRAM 容量为 M（几十 KB），Q/K/V 每行维度 d。标准注意力把 $n \times n$ 的中间矩阵物化在 HBM 里，每处理一个 query 块都要和全部 key 块交互，总 HBM 读写量约为：

$$\text{IO}_{\text{standard}} = O\left(n^2 + \frac{n^2 d}{M}\right)$$

关键是那个 n^2 项——它来自”把 $n \times n$ 的 scores 矩阵整个写进 HBM 再读回来”。n 一大，这一项就压垮一切。

2.3 Online Softmax：边算边归一化

FlashAttention 的核心洞察：**根本不需要把 $n \times n$ 的 scores 矩阵整个写进 HBM**。可以把 K/V 切成小块，在 SRAM 里逐块算，同时维护三个量：当前最大分数 m、修正后的指数和 l、累加输出 O。

假设处理到第 j 个 key 块时，手里有：- m_j：目前见过的最大 score；- $l_j = \sum_{i \le j} \exp(s_i - m_j)$：以 m_j 为基准修正后的指数和；- $O_j = \sum_{i \le j} \exp(s_i - m_j) v_i$：修正后的加权输出。

现在来一个新块，新分数是 $\{s'_i\}$。分四步更新：

第一步：更新最大值。

$$m_{j+1} = \max\left(m_j,\ \max_i s'_i\right)$$

第二步：把旧的和、旧的输出按新旧最大值的差重标定。因为之前的 $\exp(s_i - m_j)$ 是拿 m_j 当基准算的，现在基准换成了 m_{j+1}，得乘上 $\exp(m_j - m_{j+1})$ 修正：

$$l_j^{\text{old}} = l_j \cdot \exp(m_j - m_{j+1}), \qquad O_j^{\text{old}} = O_j \cdot \exp(m_j - m_{j+1})$$

第三步：把新块的贡献加进去。

$$l_{j+1} = l_j^{\text{old}} + \sum_i \exp(s'_i - m_{j+1})$$

$$O_{j+1} = O_j^{\text{old}} + \sum_i \exp(s'_i - m_{j+1}) v'_i$$

第四步（最后）：归一化。全部块处理完后，$\text{out} = O_{j+1} / l_{j+1}$。

整个过程只在 SRAM 里维护 m, l, O 三个向量，$n \times n$ 的中间矩阵**从来不需要写进 HBM**。数学上，这和先算完整 $A = \text{softmax}(S)$ 再乘 V 完全等价——只是计算顺序换了。

2.4 分块后的 IO 复杂度

FlashAttention 把 Q 切成 B_r 行的块、K/V 切成 B_c 列的块。对每个 Q 块：加载进 SRAM，遍历所有 K/V 块做上面的 online softmax 累积，最后把结果写回 HBM。中间产物全程留在 SRAM。

于是 IO 复杂度从 $O(n^2 + n^2 d/M)$ 降到：

$$\text{IO}_{\text{flash}} = O\left(\frac{n^2 d}{M}\right)$$

那个 n^2 的一次性写回项消失了。FLOPs 一个没少，但 HBM 访问量大幅下降——在 Memory-Bound 的场景下，这就是数倍提速。

2.5 为什么说它”精确”而不是”近似”

Online softmax 的重标定公式在数学上和一次性 softmax 完全等价（只是用了不同的基准最大值，最后都归一化掉）。它不丢精度，不分块近似，不丢任何 token。FlashAttention 提速纯粹来自”改 IO 顺序”，不是”降低计算精度”。这是它能直接替换标准注意力、且数值对得上的原因。

3. 直觉理解

直觉：标准注意力像在小厨房里做菜——每做一步就要把食材从冰箱（HBM）拿出来用一下，用完又塞回冰箱，下一步再拿出来，中间的碗碟堆在冰箱里。FlashAttention 像把这一顿需要的食材一次性摊开在灶台台面上（SRAM），在台面上一路做完，最后只把成品端出去。中间过程的临时碗碟从不离开台面——省掉了无数次开关冰箱门的时间。

4. 常见误区

误区一：以为 FlashAttention 是近似算法。它是**精确**注意力，数学结果和标准注意力完全一致，只是 IO 顺序不同。

误区二：以为 FlashAttention 减少了 FLOPs。它不减少 FLOPs，减少的是 HBM 访问次数。在 Memory-Bound 场景下，少搬数据比少算更值钱。

误区三：以为有了 FlashAttention 就不用关心 n^2 了。它只是把常数和 IO 大幅优化，n^2 的 FLOPs 本质还在，超长序列仍需稀疏/线性注意力配合。

5. 代码实现

下面实现一个简化版 FlashAttention：按块切分 Q 和 K/V，用 online softmax 逐块累积，最后和 `scaled_dot_product_attention` 对拍，验证误差极小。

```
import torch
import torch.nn.functional as F

def flash_attention_sim(Q, K, V, block=32):
    """ 简化版 FlashAttention: 分块 + online softmax。数学上等于标准注意力。"""
    b, h, n, d = Q.shape
    out = torch.zeros_like(Q)
    for i in range(0, n, block):
        q = Q[:, :, i:i+block, :]              # 当前 query 块 (b,h,Br,d)
        m = torch.full((b, h, block, 1), -1e9) # running max
        l = torch.zeros((b, h, block, 1))      # running 指数和
        O = torch.zeros((b, h, block, d))      # running 输出
        for j in range(0, n, block):
            k = K[:, :, j:j+block, :]          # key 块 (b,h,Bc,d)
            v = V[:, :, j:j+block, :]
            s = q @ k.transpose(-2, -1) / (d ** 0.5)   # 这块的 scores
            m_new = torch.maximum(m, s.max(dim=-1, keepdim=True).values)
            # 旧量按新旧最大值差重标定
            rescale = torch.exp(m - m_new)
            l = l * rescale
            O = O * rescale
            # 累加新块
```

```
            exp_s = torch.exp(s - m_new)
            l = l + exp_s.sum(dim=-1, keepdim=True)
            O = O + exp_s @ v
            m = m_new
        out[:, :, i:i+block, :] = O / l         # 最后归一化
    return out

torch.manual_seed(0)
Q = torch.randn(1, 2, 128, 64)
K = torch.randn(1, 2, 128, 64)
V = torch.randn(1, 2, 128, 64)

ref = F.scaled_dot_product_attention(Q, K, V)
out = flash_attention_sim(Q, K, V, block=32)
print(" 标准注意力 shape:", ref.shape)
print("Flash 模拟 shape:", out.shape)
print(" 两者最大逐元素误差:", (ref - out).abs().max().item(), "(应 ≈0)")
```

6. 小结

1. 标准注意力是 Memory-Bound：$n \times n$ 中间矩阵在 HBM 和 SRAM 间反复搬运。
2. Online Softmax 在 SRAM 里维护 running max/sum/output，逐块累积，从不物化完整 scores。
3. IO 复杂度从 $O(n^2 + n^2 d/M)$ 降到 $O(n^2 d/M)$，n^2 项消失。
4. FlashAttention 是精确算法，不降精度、不减 FLOPs，只改 IO 顺序。

7. 思考题

基础题

1. FlashAttention 减少的是 FLOPs 还是 HBM 访问量？为什么在 GPU 上减少后者反而更值钱？

进阶题

2. 如果 SRAM 无限大（能装下整个 $n \times n$ scores 矩阵），FlashAttention 还有意义吗？此时它退化成什么？

开放题

3. FlashAttention 的反向传播也得分块重算 scores（不存中间结果）。这意味着反向时要把前向的块再算一遍——你愿意为了省 HBM 多算一遍吗？这个 trade-off 怎么算才划算？

8. 延伸阅读 / 进一步思考

- FlashAttention 原始论文：Dao et al., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, 2022；FlashAttention-2/3。
- Online softmax：Milakov & Gimelshein, *Online normalizer calculation for softmax*, 2018。
- 进阶：FlashDecoding、与 PagedAttention 结合的推理引擎，以及稀疏注意力 + FlashAttention 的混合。

第 25 讲 MoE 混合专家：稀疏激活的条件计算

> **本讲问题**：为什么传闻 GPT-4 有 1.8T 参数但推理不算太慢？MoE 怎么做到”参数量巨大、每次却只算一小部分”？它又是怎么防止门控网络偷懒、把所有 token 都塞给同一个专家的？

1. 数学工具/预备知识

(1) 普通 FFN 的结构（ch13）。一个 FFN 是两层全连接：$x \to \text{ReLU}(W_1 x) \to W_2$。所有 token 都走同一个 FFN。

（2）门控网络（gating/router）。一个小的线性层，输入 token，输出”该把这个 token 送给哪个专家”的打分。

（3）稀疏激活。一次只激活 N 个候选里的 k 个，其余完全不参与计算。

（4）负载均衡。如果大家都挤向同一个专家，其余专家闲着，就浪费了。需要一个机制把 token 均匀摊开。

2. 完整推导

2.1 容量与计算解耦

普通 Transformer：每次前向，所有参数都参与计算。参数量 = 计算量，想加容量就得加计算。

MoE（Mixture of Experts）把 FFN 替换成 N 个并行的专家 FFN。每个 token 只送给其中 k 个专家（通常 $k = 2$）。于是：

- 总参数量 $\approx N\times$ 单专家参数量（容量随 N 涨）；
- 每次计算量 $\approx k\times$ 单专家参数量（成本随 k 涨）。

容量和计算解耦了——可以堆很大的记忆容量，而推理成本只跟那 k 个被激活的专家走。

2.2 门控网络：Top-k 路由

对每个 token，门控网络先输出 N 个 logits，再选分数最高的 k 个：

$$g(x) = \text{softmax}\big(\text{TopK}(\text{Gate}(x), k)\big)$$

具体说：$\text{Gate}(x) \in \mathbb{R}^N$ 是线性层输出的 N 个分数；TopK 选出最大的 k 个，其余置成 $-\infty$，再过 softmax，于是只有 k 个位置非零，且这 k 个权重加起来等于 1。

该 token 的输出是被选中专家的加权和：

$$\text{MoE}(x) = \sum_{i \in \text{TopK}} g_i(x) \cdot \text{Expert}_i(x)$$

2.3 负载均衡损失：防止赢家通吃

问题：门控网络可能很快学会”偷懒”——总把 token 送给少数几个它最有把握的专家，其余专家永远收不到 token，等于白建。这叫”专家坍缩”。

解决办法：加一个辅助负载均衡损失，鼓励每个专家被选中的频率大致相等：

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^{N} f_i \cdot P_i$$

其中 f_i 是专家 i 实际被路由到的 token 比例，P_i 是门控网络给专家 i 的平均打分。理想情况下 $f_i = P_i = 1/N$，此时 $N \sum_i (1/N)(1/N) = 1/N$，损失最小。只要某个专家被过度偏爱或冷落，$f_i$ 和 P_i 偏离 $1/N$，损失就上升——逼着门控网络均匀分配。

2.4 容量因子：工程上的取舍

实际工程里，每个专家一次最多处理多少 token 由”容量”决定：

$$\text{capacity} = \text{batch} \times \text{seq} \times \frac{k}{N} \times \text{capacity_factor}$$

如果某专家分到的 token 超过它的容量，多出来的 token 就被直接丢弃（走残差连接绕过这一层）。容量因子太大浪费计算，太小会丢 token 掉点。这是个工程权衡。

2.5 为什么 MoE 推理快、训练难

推理时只激活 k 个专家，所以虽然总参数巨大，单次前向的计算量只相当于 k 个专家——这就是”大参数、快推理”。

但训练很难：(1) 要跨 GPU 把 token 发到不同专家（All-to-All 通信）；(2) 要加负载均衡损失防止坍缩；(3) 不同专家更新步调不一致，训练不稳定。所以 MoE 只在超大规模数据下才划算。

3. 直觉理解

直觉：普通 FFN 像一个全科医生，什么病都自己看。MoE 像一家有 8 个专科医生的医院——每个病人（token）进门由分诊台（门控网络）分给最对口的 2 个医生看。医院总人力（参数量）是 8 个医生，但每个病人只看 2 个，医院运转成本（计算量）只相当于 2 个医生。负载均衡损失就是规定”分诊台不能把所有病人都塞给最火的那两个医生”，否则其他医生闲死、热门医生累死。

4. 常见误区

误区一：以为 MoE 推理时所有专家都算一遍。只激活 Top-k 个，其余完全不参与前向。

误区二：以为 MoE 一定比 dense 模型好。它训练难（负载均衡、All-to-All 通信）、显存占用大（所有专家权重都得驻留），只在超大规模下才体现优势。

误区三：以为 MoE 省显存。恰恰相反——所有专家的权重都得装在显存里，省的是计算量，不是显存。

5. 代码实现

下面实现一个干净的 MoE：门控选 Top-k，按专家分组喂 token，加权求和。最后对比”总参数量”和”每次激活参数量”。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    """ 一个普通的 FFN 专家"""
    def __init__(self, d_model=128, d_ff=256):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)
    def forward(self, x):
        return self.fc2(F.gelu(self.fc1(x)))

class MoE(nn.Module):
    def __init__(self, d_model=128, n_experts=8, top_k=2):
        super().__init__()
        self.experts = nn.ModuleList([Expert(d_model) for _ in range(n_experts)])
        self.gate = nn.Linear(d_model, n_experts)
        self.top_k = top_k
        self.n_experts = n_experts

    def forward(self, x):
        b, s, d = x.shape
        x_flat = x.reshape(-1, d)                       # (N_token, d)
        # 门控: 选 top_k 个专家
        logits = self.gate(x_flat)                      # (N_token, E)
        top_logits, top_idx = logits.topk(self.top_k, dim=-1)  # (N_token, k)
        top_probs = F.softmax(top_logits, dim=-1)       # 被选专家的权重
        output = torch.zeros_like(x_flat)
        for e in range(self.n_experts):
```

```
            # 哪些 (token, 槽位) 路由到了专家 e
            mask = (top_idx == e)                          # (N_token, k)
            if not mask.any():
                continue
            token_ids, slot_ids = mask.nonzero(as_tuple=True)
            expert_out = self.experts[e](x_flat[token_ids])
            w = top_probs[token_ids, slot_ids].unsqueeze(-1)
            output[token_ids] += w * expert_out
        return output.view(b, s, d)

torch.manual_seed(0)
moe = MoE(d_model=128, n_experts=8, top_k=2)
x = torch.randn(2, 10, 128)
out = moe(x)

total = sum(p.numel() for p in moe.parameters())
print(f" 输入 shape: {tuple(x.shape)}, 输出 shape: {tuple(out.shape)}")
print(f" 总参数量: {total/1e6:.2f}M")
print(f" 每次实际激活参数量 (top_k/8 = 1/4): {total*2/8/1e6:.2f}M")
print(f" 输出与输入同形状: {out.shape == x.shape}")
```

6. 小结

1. MoE 用多个并行专家 FFN 替代单个 FFN，每次只激活 Top-k 个。
2. 参数量随专家数 N 增长，计算量只随激活数 k 增长——容量与计算解耦。
3. 负载均衡损失 $N\sum_i f_i P_i$ 防止门控坍缩到少数专家。
4. 容量因子控制每专家 token 上限，超出则丢弃；MoE 省计算但不省显存。

7. 思考题

基础题

1. 如果把 k 设成 N（所有专家都激活并平均加权），这个 MoE 退化成了什么？

进阶题

2. 负载均衡损失 $N\sum_i f_i P_i$ 在理想均匀分配时取到什么值？为什么这个损失能逼门控网络别”赢家通吃”？

开放题

3. MoE 训练时需要 All-to-All 通信（把 token 发到不同 GPU 上的专家）。请从”模型并行”的角度想：为什么 MoE 天生适合专家并行（把不同专家放不同 GPU）？这种并行和张量并行有什么区别？

8. 延伸阅读 / 进一步思考

- MoE 经典：Shazeer et al., *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, 2017；Switch Transformer, 2021。
- 负载均衡与容量因子：Switch Transformer 论文的辅助损失与 token drop 机制。
- 进阶：DeepSeekMoE、专家并行的通信优化、MoE 的训练稳定性（aux-loss-free 路由），可作为第二册。

第 26 讲 量化与蒸馏：模型压缩的核心机理

本讲问题：为什么把 FP32 权重压成 INT8 后模型基本不掉点？知识蒸馏里说的”暗知识”到底是什么？温度 T 又是怎么把它调出来的？

1. 数学工具/预备知识

（1）线性量化。就是把一个连续的浮点数区间，等间距地映射到一个离散的整数区间。中间靠一个 scale（间隔）和一个 zero point（零点）连接。

（2）KL 散度（ch02）。$D_{\mathrm{KL}}(p\|q)=\sum_k p_k\log(p_k/q_k)$，衡量两个分布差多少。蒸馏用它让学生分布去贴近教师分布。

（3）软标签 vs 硬标签。硬标签是 one-hot（只有正确类为 1）；软标签是教师模型 softmax 出来的完整概率分布，包含所有类的相对可能性。

（4）温度 softmax（ch04）。$p_k=\exp(z_k/T)/\sum_j\exp(z_j/T)$。$T$ 越大，分布越平滑。

2. 完整推导

2.1 线性量化数学

把浮点区间 $[r_{\min},r_{\max}]$ 等间距映射到整数区间 $[q_{\min},q_{\max}]$（INT8 无符号版是 $[0,255]$）。用一个 scale s 和零点 z 连接：

$$r=s(q-z)$$

反过来说，给定浮点 r，量化到哪个整数 q：

$$q=\mathrm{round}(r/s)+z,\qquad s=\frac{r_{\max}-r_{\min}}{q_{\max}-q_{\min}}$$

z 的作用是让浮点 0 正好对应某个整数（因为权重里常有很多 0，这个数必须能精确表示）。反量化回浮点：$r_{\text{deq}}=s(q-z)$。

量化误差来自那个 round 操作，最大误差是 $s/2$——因为最近的整数和真值最多差半个间隔。

几何意义：量化就是把一条连续的数轴切成 2^8 个等宽台阶。每个台阶上的所有浮点都被”四舍五入”成同一个整数，代价就是那个台阶高度 s。

2.2 为什么 INT8 几乎不掉点

一个直觉是”少了精度肯定掉点”，但实践中 INT8 量化权重常常几乎不掉点。原因是：**权重本身高度冗余**——很多权重值聚集在 0 附近，且它们对最终输出的贡献被平均掉了。把一个权重从 FP32（2^{23} 个台阶）压到 INT8（256 个台阶），每个权重的绝对误差变大了，但大量小误差互相抵消，对最终 logits 的影响微乎其微。

2.3 PTQ 与 QAT

PTQ（训练后量化）：模型训练完，直接对权重量化，不再训练。简单快，但精度损失相对大。

QAT（量化感知训练）：训练过程里就在前向时模拟量化误差（假装权重是 INT8 的），让模型提前适应低精度。精度更好但要重新训练，成本高。

2.3.1 对称 vs 非对称、逐张量 vs 逐通道

量化还有两个工程细节值得说清楚。

对称 vs 非对称。上面公式里有个零点 z，允许量化区间不对称地套在浮点区间上，这叫非对称量化。如果权重本身以 0 为中心（对称分布），可以直接令 $z=0$，省掉一个参数，叫对称量化。实践中权重多用对称量化，激活因为分布偏斜常用非对称。

逐张量 vs 逐通道。一个 scale 是给整个张量共用，还是给权重矩阵的每一行单独一个 scale？逐通道（per-channel）让每一行用自己的量化区间，能适应不同行的数值范围差异，精度更好，是大模型权重量化的默认做法。

2.4 知识蒸馏：用大模型教小模型

教师模型（大模型）输出一个软概率分布 p^T，学生模型（小模型）不直接学 one-hot 标签，而是学着模仿这个软分布。蒸馏损失：

$$\mathcal{L}_{\text{distill}} = T^2 \, D_{\text{KL}}(p^T \| p^S)$$

为什么乘 T^2？ 因为 softmax 除以温度 T 后，梯度会缩小 T^2 倍（温度越高，软标签越平，梯度越弱）。乘回 T^2 是为了让蒸馏损失和普通交叉熵损失在量级上可比，不然温度一大蒸馏项就被淹没了。

2.5 暗知识：软标签里多出来的信息

硬标签（one-hot）只说”正确答案是 A”，对 B、C、D 一概说是 0——里面没有任何额外信息，熵为 0。

但教师模型的软标签不仅说”是 A”，还说了”B 比 C 更像 A”“D 最离谱”。这种**类间相似度信息**藏在软标签的概率比例里，硬标签完全没有。Hinton 称之为”暗知识”（dark knowledge）。学生跟着软标签学，不仅学到”答案是什么”，还学到”哪些错误答案更接近正确答案”——这就是它比直接学硬标签泛化更好的原因。

3. 直觉理解

> **直觉**：量化像把高清照片存成 JPEG——肉眼几乎看不出区别，但文件小了 4 倍。蒸馏像让学霸（教师）带学生——学霸不光告诉学生标准答案 A，还会说”B 其实也挺像，别看到它就选；D 明显错”。这种”错误答案之间的微妙差别”才是真正值钱的经验，比干巴巴的标准答案教得好。

4. 常见误区

> **误区一**：以为量化必然掉精度。对很多层，INT8 量化后精度几乎不变，因为权重冗余度高。但对某些敏感层（比如最后的输出投影）可能要特殊处理。
>
> **误区二**：以为蒸馏就是”小模型抄大模型的答案”。它学的是软标签里的暗知识，不是硬答案本身。
>
> **误区三**：以为 T 越大越好。T 太大，软标签太平，暗知识反而被稀释成均匀分布，学生学不到东西；T 太小，软标签又变回接近 one-hot，退化成硬标签。要折中。

5. 代码实现

下面演示两件事：(1) 把一组权重线性量化再反量化，看平均误差；(2) 实现蒸馏损失，对比硬标签 CE 和软标签蒸馏损失。

```
import torch
import torch.nn.functional as F

# ---------- (1) 线性量化 ----------
def quantize_tensor(x, num_bits=8):
    qmin, qmax = 0, 2 ** num_bits - 1
    rmin, rmax = x.min().item(), x.max().item()
    scale = (rmax - rmin) / (qmax - qmin)          # 台阶宽度
    zero_point = round(qmin - rmin / scale)         # 浮点 0 对应的整数
    q = torch.round(x / scale + zero_point).clamp(qmin, qmax)
    return q, scale, zero_point

def dequantize_tensor(q, scale, zero_point):
    return scale * (q - zero_point)                 # 还原回浮点

torch.manual_seed(0)
x = torch.randn(1000) * 3                           # 模拟权重
q, s, z = quantize_tensor(x, num_bits=8)
x_deq = dequantize_tensor(q, s, z)
err = (x - x_deq).abs().mean()
```

```
print(f"INT8 量化平均绝对误差: {err:.4f}")
print(f" 相对误差: {err / x.abs().mean():.2%}")

# ---------- (2) 知识蒸馏损失 ----------
teacher_logits = torch.randn(4, 10)   # 教师: 4 样本 10 类
student_logits = torch.randn(4, 10)
labels = torch.randint(0, 10, (4,))

T = 4.0
hard_loss = F.cross_entropy(student_logits, labels)          # 硬标签 CE
# 软标签: 教师/学生各自除以 T 后算 KL
soft_target = F.softmax(teacher_logits / T, dim=-1)
soft_pred = F.log_softmax(student_logits / T, dim=-1)
distill_loss = F.kl_div(soft_pred, soft_target, reduction='batchmean') * T ** 2
total = 0.5 * hard_loss + 0.5 * distill_loss
print(f"\n硬标签损失:     {hard_loss.item():.4f}")
print(f" 蒸馏损失 (带 T^2): {distill_loss.item():.4f}")
print(f" 总损失:         {total.item():.4f}")
```

6. 小结

1. 线性量化用 scale + zero_point 把浮点等间距映射到整数，最大误差 $s/2$。
2. PTQ 简单但掉点多，QAT 训练时模拟量化误差、精度更好。
3. 蒸馏用教师软标签教学生，软标签里的”类间相似度”就是暗知识。
4. 蒸馏损失 $= T^2\mathrm{KL}(p^T\|p^S)+$ 硬标签 CE，乘 T^2 是为了和 CE 量级对齐。

7. 思考题

基础题

1. 为什么 INT8 量化误差是 $s/2$? 这个 $s/2$ 和浮点区间宽度是什么关系?

进阶题

2. 蒸馏时温度 T 越大，软标签越均匀。请论证：$T\to\infty$ 时软标签趋近均匀分布，暗知识被稀释；$T\to 0$ 时又趋近 one-hot。所以 T 要取中间值。

开放题

3. 为什么 INT4 量化比 INT8 难很多? 从”台阶数变少、误差变大”和”权重分布不一定均匀”两个角度讨论。GPTQ、AWQ 这些 INT4 量化方法分别是怎么补救的?

8. 延伸阅读 / 进一步思考

- 蒸馏原始论文：Hinton et al., *Distilling the Knowledge in a Neural Network*, 2015。
- 量化：GPTQ、AWQ、LLM.int8() 等大模型权重量化方法。
- 进阶：量化与蒸馏结合、投机采样（用小学生模型草稿、大教师模型校验），可作为推理优化方向。

第 27 讲 手写 Attention：50 行 PyTorch 实现

本讲问题：前面讲了那么多公式，现在动手把注意力从零写出来。不调任何现成的注意力 API，不依赖黑盒，一行一行把 SDPA 和 MultiHeadAttention 搭出来，并用单元测试证明它没有”偷看未来”。

1. 数学工具/预备知识

- **ch08 的 Scaled Dot-Product Attention**：$\mathrm{softmax}(QK^\top/\sqrt{d_k})V$。
- **ch09 的多头**：把 d 维切成 h 份，每份 $d_k=d/h$，各自做注意力再拼回来。

- **ch14 的 Causal Mask**：把未来位置的分数换成 $-\infty$，softmax 后权重为 0。

这一讲不推新公式，重点是把公式正确翻译成能跑的代码，并想清楚每个 reshape 的维度含义。

2. 完整推导

2.1 SDPA 的五个步骤

Scaled Dot-Product Attention 拆开就是五步：

1. $QK^\top$：算每个 query 和每个 key 的相似度；
2. 除以 $\sqrt{d_k}$：防止点积过大导致 softmax 饱和（ch08）；
3. 加 mask：把未来/填充位置的分数设成 $-\infty$；
4. softmax：把分数归一化成权重；
5. 乘 V：按权重加权求和。

翻译成代码就是一个 `forward` 函数，下面 2.2 给出。

2.2 MultiHeadAttention 的数据流

多头就是把上面这套做 h 遍，只是在不同子空间里。关键在 reshape：

- 先用 W_Q, W_K, W_V 把 d 维投影到 d 维；
- 把最后一维 (d) 拆成 (h, d_k)，再把 head 维搬到前面，让注意力对每个头独立做；
- 注意力算完后，把头拼回去（transpose + reshape）；
- 最后用 W_O 投影回 d 维。

2.3 单元测试：因果掩码不能泄露未来

最关键的验证是：位置 0 的输出只能看到位置 0 及以前。具体做法：把位置 4 的输入改一下，如果位置 0 的输出变了，就说明它偷看到了位置 4——掩码失败。位置 4 自己的输出当然应该变。

3. 直觉理解

> **直觉**：把 Q/K/V 投影 → 分头 → 点积 → 缩放 → Mask → Softmax → 乘 V → 合并头 → 输出投影，就是全部。没有任何玄学，全是矩阵乘法加 reshape。真正容易错的不是公式，是维度搬来搬去时把 head 维放错位置。

4. 常见误区

> **误区一**：以为 MHA 需要很多特殊算子。它就是矩阵乘法 + reshape + softmax。
>
> **误区二**：以为 `transpose` 之后直接 `view` 就行。transpose 会让张量在内存里不连续，必须先 `.contiguous()` 再 `.view`，否则结果错乱。

5. 代码实现

下面从零写 `ScaledDotProductAttention` 和 `MultiHeadAttention`，并跑两个单元测试：因果无泄露、输出形状正确。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ScaledDotProductAttention(nn.Module):
    """ 缩放点积注意力: softmax(QK^T/sqrt(d_k)) V"""
    def forward(self, Q, K, V, mask=None):
        d_k = Q.size(-1)
        # 步骤 1+2: 点积并缩放。Q:(b,h,nq,dk) K:(b,h,nk,dk) -> scores:(b,h,nq,nk)
```

```
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)
        # 步骤 3: 把 mask=0 的位置分数换成 -inf, softmax 后权重为 0
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        # 步骤 4: softmax 归一化成注意力权重
        alpha = F.softmax(scores, dim=-1)
        # 步骤 5: 按权重加权 V
        return torch.matmul(alpha, V), alpha

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, n_heads=8):
        super().__init__()
        assert d_model % n_heads == 0, "d_model 必须能被头数整除"
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads          # 每个头的维度
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention()

    def split_heads(self, x):
        """(b, s, d_model) -> (b, n_heads, s, d_k): 把最后一维拆成头并换到前面"""
        b, s, _ = x.shape
        return x.view(b, s, self.n_heads, self.d_k).transpose(1, 2)

    def forward(self, Q, K, V, mask=None):
        b = Q.size(0)
        q = self.split_heads(self.W_Q(Q))     # (b,h,nq,dk)
        k = self.split_heads(self.W_K(K))
        v = self.split_heads(self.W_V(V))
        out, alpha = self.attn(q, k, v, mask)  # out:(b,h,nq,dk)
        # 合并头: transpose 回 (b,s,h,dk) 再展平成 (b,s,d_model)
        # transpose 后必须 contiguous() 才能 view
        out = out.transpose(1, 2).contiguous().view(b, -1, self.d_model)
        return self.W_O(out), alpha

# ===== 单元测试 =====
def test_causal_no_leak():
    """ 因果掩码: 改了位置 4, 位置 0 的输出绝不能变"""
    torch.manual_seed(42)
    mha = MultiHeadAttention(64, 4)
    x = torch.randn(1, 5, 64)
    # 下三角掩码: 位置 i 只能看 0..i
    mask = torch.tril(torch.ones(5, 5)).unsqueeze(0).unsqueeze(0).bool()
    out1, _ = mha(x, x, x, mask)
    # 把位置 4 的输入打乱
    x2 = x.clone(); x2[0, 4, :] += 100
    out2, _ = mha(x2, x2, x2, mask)
    diff0 = (out1[0, 0] - out2[0, 0]).abs().max().item()
    diff4 = (out1[0, 4] - out2[0, 4]).abs().max().item()
    print(f" 位置 0 输出差异: {diff0:.2e} (应 ≈0)")
    print(f" 位置 4 输出差异: {diff4:.2e} (应 >0)")
    assert diff0 < 1e-4, " 因果掩码泄露! 位置 0 看到了未来! "
    print(" 因果无泄露测试通过")

def test_output_shape():
    mha = MultiHeadAttention(512, 8)
    x = torch.randn(2, 10, 512)
```

```
    out, alpha = mha(x, x, x)
    assert out.shape == (2, 10, 512), f" 输出形状错: {out.shape}"
    assert alpha.shape == (2, 8, 10, 10), f" 注意力权重形状错: {alpha.shape}"
    print(" 输出形状测试通过")

test_causal_no_leak()
test_output_shape()
```

6. 小结

1. SDPA = 点积 → 缩放 → Mask → Softmax → 乘 V，五步。
2. MHA = 投影 → 分头 → SDPA → 合并头 → 输出投影。
3. transpose 后要 `.contiguous()` 才能 `view`，否则维度错乱。
4. 因果无泄露测试是验证 Causal Mask 正确性的标准方法。

7. 思考题

基础题

1. `split_heads` 里先 `view(b,s,n_heads,d_k)` 再 `transpose(1,2)`，为什么一定要 transpose 把头维换到前面？不换会怎样？

进阶题

2. 注意力权重 `alpha` 的形状是 `(batch, n_heads, seq, seq)`。请解释为什么 n_heads 维要放在 batch 和 seq 之间，而不是最后。

开放题

3. 这个实现里 Q、K、V 都来自同一个输入（自注意力）。怎么改成 Cross-Attention——让 Q 来自 decoder、K/V 来自 encoder？只需要改 forward 的哪几个参数？

8. 延伸阅读 / 进一步思考

- 本讲是 ch08、ch09、ch14 的代码落地，可对照原论文图 2。
- 进阶：把这份手写 MHA 换成 `F.scaled_dot_product_attention`（FlashAttention 后端），体会”同一个数学、不同工程实现”。
- ch28 会把这个 MHA 装进完整 Transformer Block。

第 28 讲 手写 mini-Transformer：完整模型搭建

> **本讲问题**：把所有零件组装起来——从 token ID 到 logits，一个完整的 Decoder-only Transformer 长什么样？权重共享为什么能省这么多参数？搭好后真的能训出 loss 下降吗？

1. 数学工具/预备知识

- **ch10 的整体架构**：嵌入 → 位置编码 → N 层 Block → 输出投影。
- **ch12 的残差 + LayerNorm**：Pre-LN 结构。
- **ch13 的 FFN**：两层 MLP。
- **ch14 的 Causal Mask**：下三角掩码。

这一讲把这些零件拼成一个能前向、能反向、能训练的完整模型。

2. 完整推导

2.1 Token 到 logits 的数据流

一个 Decoder-only 语言模型的前向数据流：

1. Token ID → Embedding 查表，得到 d 维向量；
2. 加位置编码；
3. 过 N 层 Block（Self-Attention + 残差，再接 FFN + 残差）；
4. 最后一次 LayerNorm；
5. 线性投影到词表大小 V，得到 logits。

2.2 Pre-LN 残差结构

每层 Block 用 Pre-LN（先归一化再进子层）：

$$x = x + \text{Attn}(\text{LN}(x)), \qquad x = x + \text{FFN}(\text{LN}(x))$$

残差通路保证梯度能直达输入（ch12），Pre-LN 让训练更稳。

2.3 权重共享

输入嵌入矩阵 $E \in \mathbb{R}^{V\times d}$ 和输出投影矩阵 $W_{\text{out}} \in \mathbb{R}^{d\times V}$ 共享同一份权重（互为转置）。这省掉了 $V \times d$ 个参数——对大词表来说这是一大笔。直觉是：“这个词长什么样”和”什么样的向量对应这个词”本质是同一份知识的正反两面。

2.4 参数量手算与代码对拍

搭完模型后，用代码数出真实参数量，再和手算公式对照，验证没有漏算或多算。

3. 直觉理解

> **直觉**：嵌入层和输出层共享权重像”进出门用同一把钥匙”——进去时查这个 token 对应什么向量，出来时把向量映射回 token 空间，同一份知识用两次。整个模型就是一串”加残差”的积木块叠起来。

4. 常见误区

> **误区**一：以为权重共享必须做。早期模型共享，后来大模型发现不共享效果更好——因为嵌入空间和输出空间的需求不完全一致。
>
> **误区**二：以为 Pre-LN 可以不要最后的 LayerNorm。去掉它，最后一层的输出可能幅度失控，logits 爆掉。

5. 代码实现

下面搭一个完整的 mini-Transformer，并做一个**迷你训练**：用随机数据跑几步，确认 loss 真的在下降（证明模型能训）。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Block(nn.Module):
    """Pre-LN Transformer Block: 自注意力 + FFN, 各带残差"""
    def __init__(self, d_model=128, n_heads=4, d_ff=512):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads, batch_first=True)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(d_ff, d_model)
        )
```

```
    def forward(self, x, mask):
        h = self.norm1(x)
        x = x + self.attn(h, h, h, attn_mask=mask, need_weights=False)[0]
        h = self.norm2(x)
        x = x + self.ffn(h)
        return x

class MiniTransformer(nn.Module):
    def __init__(self, vocab_size=1000, d_model=128, n_heads=4, n_layers=4, max_seq=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model)
        self.pos = nn.Embedding(max_seq, d_model)
        self.blocks = nn.ModuleList(
            [Block(d_model, n_heads, d_model * 4) for _ in range(n_layers)]
        )
        self.norm_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)
        self.head.weight = self.embed.weight      # 权重共享

    def forward(self, idx):
        b, s = idx.shape
        pos = torch.arange(s, device=idx.device).unsqueeze(0)
        x = self.embed(idx) + self.pos(pos)
        # 因果掩码: 上三角为 True 表示" 这里被屏蔽"
        mask = torch.triu(torch.ones(s, s, device=idx.device), diagonal=1).bool()
        for blk in self.blocks:
            x = blk(x, mask)
        return self.head(self.norm_f(x))

# ===== 搭模型 + 前向 =====
torch.manual_seed(0)
model = MiniTransformer(vocab_size=1000, d_model=128, n_heads=4, n_layers=4)
x = torch.randint(0, 1000, (2, 20))
logits = model(x)
print(f" 输入: {tuple(x.shape)}, 输出 logits: {tuple(logits.shape)}")

# ===== 参数量对拍 =====
total = sum(p.numel() for p in model.parameters())
# 逐项手算 (含 bias):
#   embed: 1000*128 ; pos: 64*128 ; head 共享 embed 不重复算
#   每 block: attn in_proj(3*128*128+3*128) + out_proj(128*128+128)
#             + ffn(128*512+512)+(512*128+128) + 两个 LN(2*2*128)
#   最后 norm_f: 2*128
manual = (1000*128 + 64*128
          + 4*((3*128*128+3*128) + (128*128+128) + (128*512+512) + (512*128+128) + 2*2*128)
          + 2*128)
print(f" 代码数参数量: {total/1e6:.3f}M, 手算: {manual/1e6:.3f}M, 一致: {total==manual}")

# ===== 迷你训练: 确认 loss 在下降 =====
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
y = torch.randint(0, 1000, (2, 20))   # 随机目标, 只验证训练回路通不通
losses = []
for step in range(30):
    out = model(x)
    loss = F.cross_entropy(out.reshape(-1, 1000), y.reshape(-1))
    opt.zero_grad(); loss.backward(); opt.step()
    losses.append(loss.item())
print(f"\n训练 loss: 第 1 步 ={losses[0]:.3f}, 第 30 步 ={losses[-1]:.3f}")
print(f"loss 下降了: {losses[-1] < losses[0]}")
```

6. 小结

1. 完整 mini-Transformer = 嵌入 + 位置编码 + N 层 Pre-LN Block + 最终 LN + 输出投影。
2. Pre-LN：先 Norm 再子层再残差，训练更稳。
3. 权重共享：嵌入和输出投影共用一份矩阵，省 $V \times d$ 参数。
4. 搭好后必须跑一个迷你训练，确认 loss 下降——证明前向反向回路真的通。

7. 思考题

基础题

1. 如果不共享权重，这个模型要多花多少参数？（提示：$V \times d$。）

进阶题

2. Pre-LN 为什么最后还要单独来一次 LayerNorm？如果把 `self.norm_f` 去掉，logits 的幅度可能出什么问题？

开放题

3. 这个模型是 Decoder-only。要改成 Encoder-Decoder 结构（加 Cross-Attention），需要在 Block 里加什么模块？参考 ch06 的 Seq2Seq 结构想一想。

8. 延伸阅读 / 进一步思考

- 本讲是 ch10 架构图的代码落地。
- 进阶：把这里的 `nn.MultiheadAttention` 换成 ch27 手写的 MHA，观察参数量是否一致。
- ch29 会用这个模型在真实小语料上训练并可视化注意力。

第 29 讲 从零训练：小语料上的语言模型与注意力可视化

> **本讲问题**：把前面所有零件拼起来，在一个小语料上真正训练一个字符级语言模型。loss 真的会下降吗？生成的文本像不像话？把注意力权重打印成热图，能看出模型到底在”看”哪里？

1. 数学工具/预备知识

- **ch19 的 AdamW**：优化器。
- **ch17 的交叉熵**：损失函数。
- **ch14 的 Causal Mask**：生成时不能看未来。
- **ch28 的 mini-Transformer**：本讲直接用它。

这一讲不推新公式，重点是把”数据 → 模型 → 训练 → 生成 → 可视化”整条链路跑通。

2. 完整推导

2.1 字符级数据管道

把一段文本切成单个字符，建立字符 ↔ID 对照表。用滑窗构造样本：输入是连续 L 个字符的 ID，目标是向右错一位的序列（输入 $x_{1:L}$，预测 $x_{2:L+1}$）。这就是”猜下一个字符”。

2.2 训练配置

- 优化器：AdamW；
- 损失：交叉熵（对字符 ID 分类）；
- 梯度裁剪：防止某一步梯度过大炸掉；
- 因果掩码：保证位置 t 只看 $0:t$。

2.3 生成与可视化

训练完后：给一个 prompt，逐字符自回归生成。同时取出某一层的注意力权重矩阵，打印成文本热图——行是”当前在生成哪个位置”，列是”它回头看了哪些位置”。

3. 直觉理解

> **直觉**：训练就是让模型在一堆文本上反复”猜下一个字”，猜错了就调权重。几百步后它就把这段文本的节奏背下来了。注意力热图告诉我们：模型在生成某个字时，眼神主要落在了哪几个字上。

4. 常见误区

> **误区一**：以为小模型训练没意义。即使是字符级小模型，也能学到基本的重复模式，热图能直观展示它关注什么。
>
> **误区二**：以为生成文本要很真实。小语料上的小模型只是”背个调调”，重点是验证整条链路通了，不是产出可用文本。

5. 代码实现

下面是完整可运行脚本：搭数据、建模型、训练（打印 loss 下降）、生成文本、打印注意力热图。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# ---------- 数据 ----------
text = "the quick brown fox jumps over the lazy dog. " * 30
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for i, c in enumerate(chars)}
V = len(chars)
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
print(f" 词表大小: {V}, 文本长度: {len(data)}")

# ---------- 模型（保存注意力权重以便可视化） ----------
class Block(nn.Module):
    def __init__(self, d=64, h=4):
        super().__init__()
        self.norm1 = nn.LayerNorm(d)
        self.attn = nn.MultiheadAttention(d, h, batch_first=True)
        self.norm2 = nn.LayerNorm(d)
        self.ffn = nn.Sequential(nn.Linear(d, 4*d), nn.GELU(), nn.Linear(4*d, d))
        self.attn_w = None   # 缓存最后一次的注意力权重
    def forward(self, x, mask):
        h = self.norm1(x)
        a, w = self.attn(h, h, h, attn_mask=mask, need_weights=True)
        self.attn_w = w.detach()
        x = x + a
        x = x + self.ffn(self.norm2(x))
        return x

class TinyLM(nn.Module):
    def __init__(self):
        super().__init__()
        self.emb = nn.Embedding(V, 64)
        self.pos = nn.Embedding(32, 64)
        self.blocks = nn.ModuleList([Block(64, 4) for _ in range(2)])
        self.norm = nn.LayerNorm(64)
```

```
        self.head = nn.Linear(64, V)
    def forward(self, idx):
        b, s = idx.shape
        x = self.emb(idx) + self.pos(torch.arange(s, device=idx.device))
        mask = torch.triu(torch.ones(s, s, device=idx.device), diagonal=1).bool()
        for blk in self.blocks:
            x = blk(x, mask)
        return self.head(self.norm(x))

# ---------- 训练 ----------
torch.manual_seed(0)
model = TinyLM()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
L = 32
losses = []
for step in range(300):
    i = torch.randint(0, len(data) - L - 1, (8,))
    x = torch.stack([data[j:j+L] for j in i])
    y = torch.stack([data[j+1:j+L+1] for j in i])
    loss = F.cross_entropy(model(x).reshape(-1, V), y.reshape(-1))
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    losses.append(loss.item())
print(f" 训练 loss: 第 1 步 ={losses[0]:.3f}, 第 150 步 ={losses[149]:.3f}, 第 300 步
 ↪  ={losses[-1]:.3f}")
print(f"loss 整体下降: {losses[-1] < losses[0]}")

# ---------- 生成 ----------
@torch.no_grad()
def generate(prompt, n_new=40):
    model.eval()
    x = torch.tensor([[stoi[c] for c in prompt]])
    for _ in range(n_new):
        logits = model(x[:, -32:])
        prob = F.softmax(logits[0, -1], dim=-1)
        nxt = torch.multinomial(prob, 1)
        x = torch.cat([x, nxt.unsqueeze(0)], dim=1)
    return ''.join(itos[i] for i in x[0].tolist())

print("\n生成结果:", repr(generate("the quick ", 40)))

# ---------- 注意力热图 (文本版) ----------
model.eval()
probe = torch.tensor([[stoi[c] for c in "the quick brown fox"[:16]]])
_ = model(probe)
w = model.blocks[0].attn_w[0]   # 第一层, 第 0 个头, (16,16)
labels = [itos[i] for i in probe[0].tolist()]
print("\n第一层第 0 头注意力热图 (行 = 当前位置, 列 = 它看的位置, ●= 权重大):")
print("     " + "".join(l for l in labels))
for r in range(16):
    row = "".join("●" if w[r, c] > 0.15 else ("·" if w[r, c] > 0.05 else " ")
                  for c in range(16))
    print(f"  {labels[r]:>2} : {row}")
```

6. 小结

1. 字符级 LM：把文本切字符、编号，用滑窗构造”输入 → 下一位”样本。
2. 训练用交叉熵 + AdamW + 梯度裁剪 + 因果掩码，小语料上 loss 能明显下降。

3. 生成是自回归：每次取最后位置的 logits 采样，再接回输入。
4. 注意力热图把权重矩阵可视化，能直观看出模型生成某字时回头看了哪些字。

7. 思考题

基础题

1. 字符级模型和词级（token 级）模型各有什么优劣？为什么大模型不用字符级？

进阶题

2. 训练 loss 不降反升，可能是哪些原因？（提示：学习率、梯度爆炸、mask 写错。）

开放题

3. 注意力热图里，如果某个位置对所有位置的权重都很均匀，说明什么？如果它几乎只盯着自己（对角线上权重大），又说明它学到了什么？

8. 延伸阅读 / 进一步思考

- 本讲对应 Andrej Karpathy 的 *nanoGPT* 教程，强烈推荐对照阅读。
- 进阶：把字符级换成 BPE tokenizer、把小语料换成真正的文本数据集、加学习率调度（ch18）。
- 注意力可视化的经典文献：Vaswani et al. 论文里的注意力图，以及 Clark et al. 关于 BERT 注意力头的分析。

第 30 讲 面试与进阶：高频考点与论文阅读路线

> **本讲问题**：30 讲学完了，面试会问什么？接下来往哪个方向深入？这一讲把整个知识体系串成一张因果地图——每个设计决策都不是拍脑袋，而是被前一个问题逼出来的。

1. 数学工具/预备知识

本讲是前 29 讲的汇总，不需要新数学工具。目标是把零散知识点串成”问题 → 动机 → 设计 → 代价”的因果链。

2. 完整推导

2.1 高频面试题连答

Q1：为什么 Self-Attention 要除以 $\sqrt{d_k}$？ 答：Q/K 分量独立零均值单位方差时，点积方差为 d_k。不缩放会导致打分过大、softmax 饱和、梯度消失。除以 $\sqrt{d_k}$ 把方差归一化为 1。（ch08）

Q2：残差连接解决了什么问题？ 答：梯度消失。残差 $x + F(x)$ 的反向梯度为 $\partial L/\partial y \cdot (1 + \partial F/\partial x)$，即使子层梯度趋零，恒等通路 1 仍在，梯度直达输入。（ch12）

Q3：LayerNorm 和 BatchNorm 的区别？ 答：LN 对特征维归一化、每个样本独立、与 batch 大小无关；BN 对 batch 维归一化、依赖 batch 统计量，train/inference 行为不一致。Transformer 用 LN。（ch12）

Q4：多头注意力为什么有效？参数量和单头比？ 答：多头在不同子空间并行做注意力，捕捉不同类型的关系。总参数量和单头基本相同（只是把 d 切 h 份），计算量也相同。（ch09）

Q5：Decoder 为什么需要 Causal Mask？ 答：自回归推理时位置 t 不能看到 $t+1$ 及以后，否则就是偷看答案。Mask 在 softmax 前把未来位置分数设为 $-\infty$。（ch14）

Q6：Adam 和 SGD 的区别？ 答：Adam 维护梯度一阶矩（动量）和二阶矩（自适应学习率），每个参数有自己的有效步幅；SGD 所有参数共享同一学习率。Adam 收敛快但可能泛化略差。（ch19）

Q7：KV Cache 加速原理？ 答：缓存历史 token 的 K/V，新 token 只需算自己的 Q/K/V 再和缓存做注意力。避免重复计算历史 K/V，推理复杂度从 $O(n^2 d^2)$ 降到 $O(nd^2 + n^2 d)$。（ch23）

Q8：FlashAttention 是近似算法吗？ 答：不是。它是精确注意力，只是通过分块和 Online Softmax 减少 HBM 访问次数，数学结果和标准注意力完全一致。（ch24）

Q9：RoPE 为什么比正余弦编码好？ 答：RoPE 把位置信息编码进 Q/K 的旋转中，点积只依赖相对位置，外推能力更强。正余弦是加在输入上的绝对位置编码，长序列外推差。（ch22）

Q10：MoE 的核心思想？ 答：稀疏激活——参数量随专家数增长，但每次推理只激活 Top-k 个专家，计算量不随参数量线性增长。（ch25）

Q11：为什么训练要用 Warmup？ 答：随机初始化时梯度方向随机、量级不稳，直接大学习率会一步把权重推飞。前几千步线性升温让动量状态稳定下来。（ch18）

Q12：Xavier 和 He 初始化怎么选？ 答：看激活函数。tanh/sigmoid 用 Xavier（$\sigma^2 = 2/(n_{in} + n_{out})$），ReLU/GELU 用 He（$\sigma^2 = 2/n_{in}$）——后者补偿了 ReLU 置零负半轴损失的一半方差。（ch20）

Q13：混合精度里 master weights 是干什么的？ 答：权重永远保留一份 FP32 副本做精确累积更新，前向/反向才转半精度。否则半精度累积更新会被舍入误差带偏。（ch21）

2.2 一条因果链串起全部

把 30 讲串起来看，你会发现每个设计都是被上一个问题逼出来的：

`Softmax` 会饱和 → 所以要缩放 `√d_k`（ch08）
梯度会消失 → 所以要残差 + `LayerNorm`（ch12）
自回归逐字生成慢 → 所以要 `KV Cache`（ch23）
显存带宽不够 → 所以要 `FlashAttention`（ch24）
绝对位置编码外推差 → 所以要 `RoPE`（ch22）
全参数训练贵 → 所以要 `MoE` 稀疏激活 / 量化蒸馏（ch25、ch26）

面试时与其背架构图，不如讲清楚这条链上每一环的”为什么”。

2.3 论文阅读路线图

1. **起点**：*Attention Is All You Need*（原始 Transformer）
2. **架构演进**：BERT（Encoder-only）→ GPT-2/3（Decoder-only）
3. **位置编码**：RoPE → ALiBi
4. **推理优化**：FlashAttention → PagedAttention
5. **训练优化**：AdamW → 学习率调度
6. **模型缩放**：Chinchilla scaling laws
7. **稀疏化**：Switch Transformer（MoE）
8. **长上下文**：NTK-aware scaling → YaRN

2.4 进阶方向

- **理论方向**：Transformer 表达能力、训练动力学、scaling law
- **系统方向**：分布式并行（张量/流水线/专家并行）、推理引擎
- **模型方向**：状态空间模型（Mamba）、线性注意力
- **应用方向**：RAG、Agent、多模态

3. 直觉理解

直觉：这 30 讲不是孤立的知识点，而是一条因果链。面试官真正想考的不是”你记不记得”，而是”你能不能从一个问题出发，自己推导出这个设计为什么必须长这样”。

4. 常见误区

误区：以为学完 30 讲就够了。这只是入门——Transformer 领域每周都有新论文。关键是建立”从问题出发推导设计”的思维方式，而不是背诵架构图。

5. 代码实现

下面是一个知识自测脚本：用代码快速验证你是否真的理解了前几讲的核心操作。

```python
import torch

def knowledge_check():
    checks = []
    # 矩阵乘法形状
    A = torch.randn(3, 4); B = torch.randn(4, 5)
    checks.append((" 矩阵乘法形状", (A @ B).shape == (3, 5)))
    # 交叉熵可算
    logits = torch.randn(1, 10); target = torch.tensor([3])
    loss = torch.nn.functional.cross_entropy(logits, target)
    checks.append((" 交叉熵 >0", loss.item() > 0))
    # softmax 归一化
    d_k = 64
    alpha = torch.softmax(torch.randn(1, 10, 10) / (d_k ** 0.5), dim=-1)
    checks.append(("softmax 归一化", torch.allclose(alpha.sum(-1), torch.ones(1, 10),
    atol=1e-5)))
    # 因果掩码
    mask = torch.tril(torch.ones(5, 5)).bool()
    checks.append((" 因果掩码正确", (mask[0, 1] == False) and (mask[1, 0] == True)))
    # 残差形状不变
    x = torch.randn(5); y = x + 0.1 * torch.randn(5)
    checks.append((" 残差形状不变", y.shape == x.shape))

    print("=== 知识自测 ===")
    ok = 0
    for name, passed in checks:
        print(f"  {'' if passed else ''} {name}")
        ok += passed
    print(f"\n通过: {ok}/{len(checks)}")

knowledge_check()
```

6. 小结

1. Transformer 的每个设计决策都有明确的数学动机，不是工程 hack。
2. 高频考点：缩放原因、残差作用、LN vs BN、KV Cache、FlashAttention 原理、RoPE。
3. 论文阅读从原始 Transformer 出发，沿架构 → 优化 → 系统三条线展开。
4. 核心能力：从问题出发推导设计，而不是背诵架构图。

7. 思考题

基础题

1. 用自己的话向一个完全不懂 AI 的人解释 Transformer 的核心思想（不超过三句话）。

进阶题

2. 把本讲 2.2 的因果链补完整：再往后延伸一环——为什么现在又开始研究 Mamba 这类线性/状态空间模型？它是对 Transformer 哪个痛点的回应？

开放题

3. 如果让你设计一个新的注意力机制，你会从什么问题出发？请说出一个真实存在的痛点（比如长序列 n^2、或显存带宽），并设想你会怎么动手。

8. 延伸阅读 / 进一步思考

- 原始论文：Vaswani et al., *Attention Is All You Need*, 2017。
- scaling law：Kaplan et al.（OpenAI）、Hoffmann et al.（Chinchilla）。
- 第二册方向：分布式训练、推理引擎、长上下文、多模态、RAG/Agent——本册是地基，后续是大厦。